
Chapter 4. SQL Syntax

This chapter describes the syntax of SQL. It forms the foundation for understanding the following chapters which will go into detail about how SQL commands are applied to define and modify data.

We also advise users who are already familiar with SQL to read this chapter carefully because it contains several rules and concepts that are implemented inconsistently among SQL databases or that are specific to PostgreSQL.

4.1. Lexical Structure

SQL input consists of a sequence of *commands*. A command is composed of a sequence of *tokens*, terminated by a semicolon (“;”). The end of the input stream also terminates a command. Which tokens are valid depends on the syntax of the particular command.

A token can be a *key word*, an *identifier*, a *quoted identifier*, a *literal* (or constant), or a special character symbol. Tokens are normally separated by whitespace (space, tab, newline), but need not be if there is no ambiguity (which is generally only the case if a special character is adjacent to some other token type).

For example, the following is (syntactically) valid SQL input:

```
SELECT * FROM MY_TABLE;  
UPDATE MY_TABLE SET A = 5;  
INSERT INTO MY_TABLE VALUES (3, 'hi there');
```

This is a sequence of three commands, one per line (although this is not required; more than one command can be on a line, and commands can usefully be split across lines).

Additionally, *comments* can occur in SQL input. They are not tokens, they are effectively equivalent to whitespace.

The SQL syntax is not very consistent regarding what tokens identify commands and which are operands or parameters. The first few tokens are generally the command name, so in the above example we would usually speak of a “SELECT”, an “UPDATE”, and an “INSERT” command. But for instance the UPDATE command always requires a SET token to appear in a certain position, and this particular variation of INSERT also requires a VALUES in order to be complete. The precise syntax rules for each command are described in Part VI.

4.1.1. Identifiers and Key Words

Tokens such as SELECT, UPDATE, or VALUES in the example above are examples of *key words*, that is, words that have a fixed meaning in the SQL language. The tokens MY_TABLE and A are examples of *identifiers*. They identify names of tables, columns, or other database objects, depending on the command they are used in. Therefore they are sometimes simply called “names”. Key words and identifiers have the same lexical structure, meaning that one cannot know whether a token is an identifier or a key word without knowing the language. A complete list of key words can be found in Appendix C.

SQL identifiers and key words must begin with a letter (a-z, but also letters with diacritical marks and non-Latin letters) or an underscore (_). Subsequent characters in an identifier or key word can be letters, underscores, digits (0-9), or dollar signs (\$). Note that dollar signs are not allowed in identifiers according to the letter of the SQL standard, so their use might render applications less portable. The SQL standard will not define a key word that contains digits or starts or ends with an underscore, so identifiers of this form are safe against possible conflict with future extensions of the standard.

The system uses no more than `NAMEDATALEN-1` bytes of an identifier; longer names can be written in commands, but they will be truncated. By default, `NAMEDATALEN` is 64 so the maximum identifier length is 63 bytes. If this limit is problematic, it can be raised by changing the `NAMEDATALEN` constant in `src/include/pg_config_manual.h`.

Key words and unquoted identifiers are case-insensitive. Therefore:

```
UPDATE MY_TABLE SET A = 5;
```

can equivalently be written as:

```
uPDaTE my_Table SeT a = 5;
```

A convention often used is to write key words in upper case and names in lower case, e.g.:

```
UPDATE my_table SET a = 5;
```

There is a second kind of identifier: the *delimited identifier* or *quoted identifier*. It is formed by enclosing an arbitrary sequence of characters in double-quotes (`"`). A delimited identifier is always an identifier, never a key word. So `"select"` could be used to refer to a column or table named `"select"`, whereas an unquoted `select` would be taken as a key word and would therefore provoke a parse error when used where a table or column name is expected. The example can be written with quoted identifiers like this:

```
UPDATE "my_table" SET "a" = 5;
```

Quoted identifiers can contain any character, except the character with code zero. (To include a double quote, write two double quotes.) This allows constructing table or column names that would otherwise not be possible, such as ones containing spaces or ampersands. The length limitation still applies.

Quoting an identifier also makes it case-sensitive, whereas unquoted names are always folded to lower case. For example, the identifiers `FOO`, `foo`, and `"foo"` are considered the same by PostgreSQL, but `"Foo"` and `"FOO"` are different from these three and each other. (The folding of unquoted names to lower case in PostgreSQL is incompatible with the SQL standard, which says that unquoted names should be folded to upper case. Thus, `foo` should be equivalent to `"FOO"` not `"foo"` according to the standard. If you want to write portable applications you are advised to always quote a particular name or never quote it.)

A variant of quoted identifiers allows including escaped Unicode characters identified by their code points. This variant starts with `U&` (upper or lower case `U` followed by ampersand) immediately before the opening double quote, without any spaces in between, for example `U&"foo"`. (Note that this creates an ambiguity with the operator `&`. Use spaces around the operator to avoid this problem.) Inside the quotes, Unicode characters can be specified in escaped form by writing a backslash followed by the four-digit hexadecimal code point number or alternatively a backslash followed by a plus sign followed by a six-digit hexadecimal code point number. For example, the identifier `"data"` could be written as

```
U&"d\0061t\+000061"
```

The following less trivial example writes the Russian word “slon” (elephant) in Cyrillic letters:

```
U&"\0441\043B\043E\043D"
```

If a different escape character than backslash is desired, it can be specified using the `UESCAPE` clause after the string, for example:

```
U&"d!0061t!+000061" UESCAPE '!'
```

The escape character can be any single character other than a hexadecimal digit, the plus sign, a single quote, a double quote, or a whitespace character. Note that the escape character is written in single quotes, not double quotes, after `UESCAPE`.

To include the escape character in the identifier literally, write it twice.

Either the 4-digit or the 6-digit escape form can be used to specify UTF-16 surrogate pairs to compose characters with code points larger than `U+FFFF`, although the availability of the 6-digit form technically makes this unnecessary. (Surrogate pairs are not stored directly, but are combined into a single code point.)

If the server encoding is not UTF-8, the Unicode code point identified by one of these escape sequences is converted to the actual server encoding; an error is reported if that's not possible.

4.1.2. Constants

There are three kinds of *implicitly-typed constants* in PostgreSQL: strings, bit strings, and numbers. Constants can also be specified with explicit types, which can enable more accurate representation and more efficient handling by the system. These alternatives are discussed in the following subsections.

4.1.2.1. String Constants

A string constant in SQL is an arbitrary sequence of characters bounded by single quotes (`'`), for example `'This is a string'`. To include a single-quote character within a string constant, write two adjacent single quotes, e.g., `'Dianne''s horse'`. Note that this is *not* the same as a double-quote character (`"`).

Two string constants that are only separated by whitespace *with at least one newline* are concatenated and effectively treated as if the string had been written as one constant. For example:

```
SELECT 'foo'
'bar';
```

is equivalent to:

```
SELECT 'foobar';
```

but:

```
SELECT 'foo'      'bar';
```

is not valid syntax. (This slightly bizarre behavior is specified by SQL; PostgreSQL is following the standard.)

4.1.2.2. String Constants with C-Style Escapes

PostgreSQL also accepts “escape” string constants, which are an extension to the SQL standard. An escape string constant is specified by writing the letter `E` (upper or lower case) just before the opening single quote, e.g., `E'foo'`. (When continuing an escape string constant across lines, write `E` only before the first opening quote.) Within an escape string, a backslash character (`\`) begins a C-like *backslash escape* sequence, in which the combination of backslash and following character(s) represent a special byte value, as shown in Table 4.1.

Table 4.1. Backslash Escape Sequences

Backslash Escape Sequence	Interpretation
<code>\b</code>	backspace
<code>\f</code>	form feed
<code>\n</code>	newline
<code>\r</code>	carriage return
<code>\t</code>	tab
<code>\o</code> , <code>\oo</code> , <code>\ooo</code> ($o = 0-7$)	octal byte value
<code>\xh</code> , <code>\xhh</code> ($h = 0-9, A-F$)	hexadecimal byte value
<code>\uxxxx</code> , <code>\Uxxxxxxxx</code> ($x = 0-9, A-F$)	16 or 32-bit hexadecimal Unicode character value

Any other character following a backslash is taken literally. Thus, to include a backslash character, write two backslashes (`\\`). Also, a single quote can be included in an escape string by writing `\'`, in addition to the normal way of `'`.

It is your responsibility that the byte sequences you create, especially when using the octal or hexadecimal escapes, compose valid characters in the server character set encoding. A useful alternative is to use Unicode escapes or the alternative Unicode escape syntax, explained in Section 4.1.2.3; then the server will check that the character conversion is possible.

Caution

If the configuration parameter `standard_conforming_strings` is `off`, then PostgreSQL recognizes backslash escapes in both regular and escape string constants. However, as of PostgreSQL 9.1, the default is `on`, meaning that backslash escapes are recognized only in escape string constants. This behavior is more standards-compliant, but might break applications which rely on the historical behavior, where backslash escapes were always recognized. As a workaround, you can set this parameter to `off`, but it is better to migrate away from using backslash escapes. If you need to use a backslash escape to represent a special character, write the string constant with an E.

In addition to `standard_conforming_strings`, the configuration parameters `escape_string_warning` and `backslash_quote` govern treatment of backslashes in string constants.

The character with the code zero cannot be in a string constant.

4.1.2.3. String Constants with Unicode Escapes

PostgreSQL also supports another type of escape syntax for strings that allows specifying arbitrary Unicode characters by code point. A Unicode escape string constant starts with `U&` (upper or lower case letter U followed by ampersand) immediately before the opening quote, without any spaces in between, for example `U&'foo'`. (Note that this creates an ambiguity with the operator `&`. Use spaces around the operator to avoid this problem.) Inside the quotes, Unicode characters can be specified in escaped form by writing a backslash followed by the four-digit hexadecimal code point number or alternatively a backslash followed by a plus sign followed by a six-digit hexadecimal code point number. For example, the string `'data'` could be written as

```
U&'d\0061t\+000061'
```

The following less trivial example writes the Russian word “slon” (elephant) in Cyrillic letters:

```
U&' \0441\043B\043E\043D'
```

If a different escape character than backslash is desired, it can be specified using the `UESCAPE` clause after the string, for example:

```
U&'d!0061t!+000061' UESCAPE '!'
```

The escape character can be any single character other than a hexadecimal digit, the plus sign, a single quote, a double quote, or a whitespace character.

To include the escape character in the string literally, write it twice.

Either the 4-digit or the 6-digit escape form can be used to specify UTF-16 surrogate pairs to compose characters with code points larger than U+FFFF, although the availability of the 6-digit form technically makes this unnecessary. (Surrogate pairs are not stored directly, but are combined into a single code point.)

If the server encoding is not UTF-8, the Unicode code point identified by one of these escape sequences is converted to the actual server encoding; an error is reported if that's not possible.

Also, the Unicode escape syntax for string constants only works when the configuration parameter `standard_conforming_strings` is turned on. This is because otherwise this syntax could confuse clients that parse the SQL statements to the point that it could lead to SQL injections and similar security issues. If the parameter is set to off, this syntax will be rejected with an error message.

4.1.2.4. Dollar-Quoted String Constants

While the standard syntax for specifying string constants is usually convenient, it can be difficult to understand when the desired string contains many single quotes, since each of those must be doubled. To allow more readable queries in such situations, PostgreSQL provides another way, called “dollar quoting”, to write string constants. A dollar-quoted string constant consists of a dollar sign (\$), an optional “tag” of zero or more characters, another dollar sign, an arbitrary sequence of characters that makes up the string content, a dollar sign, the same tag that began this dollar quote, and a dollar sign. For example, here are two different ways to specify the string “Dianne's horse” using dollar quoting:

```
$$Dianne's horse$$  
$SomeTag$Dianne's horse$SomeTag$
```

Notice that inside the dollar-quoted string, single quotes can be used without needing to be escaped. Indeed, no characters inside a dollar-quoted string are ever escaped: the string content is always written literally. Backslashes are not special, and neither are dollar signs, unless they are part of a sequence matching the opening tag.

It is possible to nest dollar-quoted string constants by choosing different tags at each nesting level. This is most commonly used in writing function definitions. For example:

```
$function$  
BEGIN  
    RETURN ($1 ~ $q$[\t\r\n\v\\]$q$);  
END;  
$function$
```

Here, the sequence `$q$[\t\r\n\v\\]$q$` represents a dollar-quoted literal string `[\t\r\n\v\\]`, which will be recognized when the function body is executed by PostgreSQL. But since the sequence does not match the outer dollar quoting delimiter `$function$`, it is just some more characters within the constant so far as the outer string is concerned.

The tag, if any, of a dollar-quoted string follows the same rules as an unquoted identifier, except that it cannot contain a dollar sign. Tags are case sensitive, so `$tag$String content$tag$` is correct, but `$TAG$String content$tag$` is not.

A dollar-quoted string that follows a keyword or identifier must be separated from it by whitespace; otherwise the dollar quoting delimiter would be taken as part of the preceding identifier.

Dollar quoting is not part of the SQL standard, but it is often a more convenient way to write complicated string literals than the standard-compliant single quote syntax. It is particularly useful when representing string constants inside other constants, as is often needed in procedural function definitions. With single-quote syntax, each backslash in the above example would have to be written as four backslashes, which would be reduced to two backslashes in parsing the original string constant, and then to one when the inner string constant is re-parsed during function execution.

4.1.2.5. Bit-String Constants

Bit-string constants look like regular string constants with a **B** (upper or lower case) immediately before the opening quote (no intervening whitespace), e.g., `B '1001 '`. The only characters allowed within bit-string constants are **0** and **1**.

Alternatively, bit-string constants can be specified in hexadecimal notation, using a leading **X** (upper or lower case), e.g., `X '1FF '`. This notation is equivalent to a bit-string constant with four binary digits for each hexadecimal digit.

Both forms of bit-string constant can be continued across lines in the same way as regular string constants. Dollar quoting cannot be used in a bit-string constant.

4.1.2.6. Numeric Constants

Numeric constants are accepted in these general forms:

```
digits
digits.[digits][e[+-]digits]
[digits].digits[e[+-]digits]
digitse[+-]digits
```

where *digits* is one or more decimal digits (0 through 9). At least one digit must be before or after the decimal point, if one is used. At least one digit must follow the exponent marker (e), if one is present. There cannot be any spaces or other characters embedded in the constant, except for underscores, which can be used for visual grouping as described below. Note that any leading plus or minus sign is not actually considered part of the constant; it is an operator applied to the constant.

These are some examples of valid numeric constants:

```
42
3.5
4.
.001
5e2
1.925e-3
```

Additionally, non-decimal integer constants are accepted in these forms:

```
0xhexdigits
0ooctdigits
0bbindigits
```

where *hexdigits* is one or more hexadecimal digits (0-9, A-F), *octdigits* is one or more octal digits (0-7), and *bindigits* is one or more binary digits (0 or 1). Hexadecimal digits and the radix prefixes can be in upper or lower case. Note that only integers can have non-decimal forms, not numbers with fractional parts.

These are some examples of valid non-decimal integer constants:

```
0b100101
0B10011001
0o273
0O755
0x42f
0XFFFF
```

For visual grouping, underscores can be inserted between digits. These have no further effect on the value of the constant. For example:

```
1_500_000_000
0b10001000_00000000
0o_1_755
0xFFFF_FFFF
1.618_034
```

Underscores are not allowed at the start or end of a numeric constant or a group of digits (that is, immediately before or after the decimal point or the exponent marker), and more than one underscore in a row is not allowed.

A numeric constant that contains neither a decimal point nor an exponent is initially presumed to be type `integer` if its value fits in type `integer` (32 bits); otherwise it is presumed to be type `bigint` if its value fits in type `bigint` (64 bits); otherwise it is taken to be type `numeric`. Constants that contain decimal points and/or exponents are always initially presumed to be type `numeric`.

The initially assigned data type of a numeric constant is just a starting point for the type resolution algorithms. In most cases the constant will be automatically coerced to the most appropriate type depending on context. When necessary, you can force a numeric value to be interpreted as a specific data type by casting it. For example, you can force a numeric value to be treated as type `real` (`float4`) by writing:

```
REAL '1.23' -- string style
1.23::REAL -- PostgreSQL (historical) style
```

These are actually just special cases of the general casting notations discussed next.

4.1.2.7. Constants of Other Types

A constant of an *arbitrary* type can be entered using any one of the following notations:

```
type 'string'
'string'::type
CAST ( 'string' AS type )
```

The string constant's text is passed to the input conversion routine for the type called *type*. The result is a constant of the indicated type. The explicit type cast can be omitted if there is no ambiguity as to the type the constant must be (for example, when it is assigned directly to a table column), in which case it is automatically coerced.

The string constant can be written using either regular SQL notation or dollar-quoting.

It is also possible to specify a type coercion using a function-like syntax:

```
typename ( 'string' )
```

but not all type names can be used in this way; see Section 4.2.9 for details.

The `::`, `CAST()`, and function-call syntaxes can also be used to specify run-time type conversions of arbitrary expressions, as discussed in Section 4.2.9. To avoid syntactic ambiguity, the *type* `'string'` syntax can only be used to specify the type of a simple literal constant. Another restriction on the *type* `'string'` syntax is that it does not work for array types; use `::` or `CAST()` to specify the type of an array constant.

The `CAST()` syntax conforms to SQL. The *type* `'string'` syntax is a generalization of the standard: SQL specifies this syntax only for a few data types, but PostgreSQL allows it for all types. The syntax with `::` is historical PostgreSQL usage, as is the function-call syntax.

4.1.3. Operators

An operator name is a sequence of up to NAMEDATALEN-1 (63 by default) characters from the following list:

```
+ - * / < > = ~ ! @ # % ^ & | ` ?
```

There are a few restrictions on operator names, however:

- `--` and `/*` cannot appear anywhere in an operator name, since they will be taken as the start of a comment.
- A multiple-character operator name cannot end in `+` or `-`, unless the name also contains at least one of these characters:

```
~ ! @ # % ^ & | ` ?
```

For example, `@-` is an allowed operator name, but `*-` is not. This restriction allows PostgreSQL to parse SQL-compliant queries without requiring spaces between tokens.

When working with non-SQL-standard operator names, you will usually need to separate adjacent operators with spaces to avoid ambiguity. For example, if you have defined a prefix operator named `@`, you cannot write `X*@Y`; you must write `X* @Y` to ensure that PostgreSQL reads it as two operator names not one.

4.1.4. Special Characters

Some characters that are not alphanumeric have a special meaning that is different from being an operator. Details on the usage can be found at the location where the respective syntax element is described. This section only exists to advise the existence and summarize the purposes of these characters.

- A dollar sign (\$) followed by digits is used to represent a positional parameter in the body of a function definition or a prepared statement. In other contexts the dollar sign can be part of an identifier or a dollar-quoted string constant.
- Parentheses (`( )`) have their usual meaning to group expressions and enforce precedence. In some cases parentheses are required as part of the fixed syntax of a particular SQL command.

- Brackets ([]) are used to select the elements of an array. See Section 8.15 for more information on arrays.
- Commas (,) are used in some syntactical constructs to separate the elements of a list.
- The semicolon (;) terminates an SQL command. It cannot appear anywhere within a command, except within a string constant or quoted identifier.
- The colon (:) is used to select “slices” from arrays. (See Section 8.15.) In certain SQL dialects (such as Embedded SQL), the colon is used to prefix variable names.
- The asterisk (*) is used in some contexts to denote all the fields of a table row or composite value. It also has a special meaning when used as the argument of an aggregate function, namely that the aggregate does not require any explicit parameter.
- The period (.) is used in numeric constants, and to separate schema, table, and column names.

4.1.5. Comments

A comment is a sequence of characters beginning with double dashes and extending to the end of the line, e.g.:

```
-- This is a standard SQL comment
```

Alternatively, C-style block comments can be used:

```
/* multiline comment
 * with nesting: /* nested block comment */
 */
```

where the comment begins with /* and extends to the matching occurrence of */. These block comments nest, as specified in the SQL standard but unlike C, so that one can comment out larger blocks of code that might contain existing block comments.

A comment is removed from the input stream before further syntax analysis and is effectively replaced by whitespace.

4.1.6. Operator Precedence

Table 4.2 shows the precedence and associativity of the operators in PostgreSQL. Most operators have the same precedence and are left-associative. The precedence and associativity of the operators is hard-wired into the parser. Add parentheses if you want an expression with multiple operators to be parsed in some other way than what the precedence rules imply.

Table 4.2. Operator Precedence (highest to lowest)

Operator/Element	Associativity	Description
.	left	table/column name separator
::	left	PostgreSQL-style typecast
[]	left	array element selection
+ -	right	unary plus, unary minus
COLLATE	left	collation selection
AT	left	AT TIME ZONE, AT LOCAL
^	left	exponentiation

Operator/Element	Associativity	Description
* / %	left	multiplication, division, modulo
+ -	left	addition, subtraction
(any other operator)	left	all other native and user-defined operators
BETWEEN IN LIKE ILIKE SIMILAR		range containment, set membership, string matching
< > = <= >= <>		comparison operators
IS ISNULL NOTNULL		IS TRUE, IS FALSE, IS NULL, IS DISTINCT FROM, etc.
NOT	right	logical negation
AND	left	logical conjunction
OR	left	logical disjunction

Note that the operator precedence rules also apply to user-defined operators that have the same names as the built-in operators mentioned above. For example, if you define a “+” operator for some custom data type it will have the same precedence as the built-in “+” operator, no matter what yours does.

When a schema-qualified operator name is used in the `OPERATOR` syntax, as for example in:

```
SELECT 3 OPERATOR(pg_catalog.+) 4;
```

the `OPERATOR` construct is taken to have the default precedence shown in Table 4.2 for “any other operator”. This is true no matter which specific operator appears inside `OPERATOR( )`.

Note

PostgreSQL versions before 9.5 used slightly different operator precedence rules. In particular, `<=`, `>=` and `<>` used to be treated as generic operators; `IS` tests used to have higher priority; and `NOT BETWEEN` and related constructs acted inconsistently, being taken in some cases as having the precedence of `NOT` rather than `BETWEEN`. These rules were changed for better compliance with the SQL standard and to reduce confusion from inconsistent treatment of logically equivalent constructs. In most cases, these changes will result in no behavioral change, or perhaps in “no such operator” failures which can be resolved by adding parentheses. However there are corner cases in which a query might change behavior without any parsing error being reported.

4.2. Value Expressions

Value expressions are used in a variety of contexts, such as in the target list of the `SELECT` command, as new column values in `INSERT` or `UPDATE`, or in search conditions in a number of commands. The result of a value expression is sometimes called a *scalar*, to distinguish it from the result of a table expression (which is a table). Value expressions are therefore also called *scalar expressions* (or even simply *expressions*). The expression syntax allows the calculation of values from primitive parts using arithmetic, logical, set, and other operations.

A value expression is one of the following:

- A constant or literal value
- A column reference

- A positional parameter reference, in the body of a function definition or prepared statement
- A subscripted expression
- A field selection expression
- An operator invocation
- A function call
- An aggregate expression
- A window function call
- A type cast
- A collation expression
- A scalar subquery
- An array constructor
- A row constructor
- Another value expression in parentheses (used to group subexpressions and override precedence)

In addition to this list, there are a number of constructs that can be classified as an expression but do not follow any general syntax rules. These generally have the semantics of a function or operator and are explained in the appropriate location in Chapter 9. An example is the `IS NULL` clause.

We have already discussed constants in Section 4.1.2. The following sections discuss the remaining options.

4.2.1. Column References

A column can be referenced in the form:

correlation.columnname

correlation is the name of a table (possibly qualified with a schema name), or an alias for a table defined by means of a `FROM` clause. The correlation name and separating dot can be omitted if the column name is unique across all the tables being used in the current query. (See also Chapter 7.)

4.2.2. Positional Parameters

A positional parameter reference is used to indicate a value that is supplied externally to an SQL statement. Parameters are used in SQL function definitions and in prepared queries. Some client libraries also support specifying data values separately from the SQL command string, in which case parameters are used to refer to the out-of-line data values. The form of a parameter reference is:

\$number

For example, consider the definition of a function, `dept`, as:

```
CREATE FUNCTION dept(text) RETURNS dept
  AS $$ SELECT * FROM dept WHERE name = $1 $$
LANGUAGE SQL;
```

Here the `$1` references the value of the first function argument whenever the function is invoked.

4.2.3. Subscripts

If an expression yields a value of an array type, then a specific element of the array value can be extracted by writing

expression[*subscript*]

or multiple adjacent elements (an “array slice”) can be extracted by writing

expression[*lower_subscript*:*upper_subscript*]

(Here, the brackets [] are meant to appear literally.) Each *subscript* is itself an expression, which will be rounded to the nearest integer value.

In general the array *expression* must be parenthesized, but the parentheses can be omitted when the expression to be subscripted is just a column reference or positional parameter. Also, multiple subscripts can be concatenated when the original array is multidimensional. For example:

```
mytable.arraycolumn[4]
mytable.two_d_column[17][34]
$1[10:42]
(arrayfunction(a,b))[42]
```

The parentheses in the last example are required. See Section 8.15 for more about arrays.

4.2.4. Field Selection

If an expression yields a value of a composite type (row type), then a specific field of the row can be extracted by writing

expression.fieldname

In general the row *expression* must be parenthesized, but the parentheses can be omitted when the expression to be selected from is just a table reference or positional parameter. For example:

```
mytable.mycolumn
$1.somecolumn
(rowfunction(a,b)).col3
```

(Thus, a qualified column reference is actually just a special case of the field selection syntax.) An important special case is extracting a field from a table column that is of a composite type:

```
(compositemcol).somefield
(mytable.compositemcol).somefield
```

The parentheses are required here to show that `compositemcol` is a column name not a table name, or that `mytable` is a table name not a schema name in the second case.

You can ask for all fields of a composite value by writing `.*`:

(*compositecol*).*

This notation behaves differently depending on context; see Section 8.16.5 for details.

4.2.5. Operator Invocations

There are two possible syntaxes for an operator invocation:

expression operator expression (binary infix operator)

operator expression (unary prefix operator)

where the *operator* token follows the syntax rules of Section 4.1.3, or is one of the key words AND, OR, and NOT, or is a qualified operator name in the form:

OPERATOR(*schema.operatorname*)

Which particular operators exist and whether they are unary or binary depends on what operators have been defined by the system or the user. Chapter 9 describes the built-in operators.

4.2.6. Function Calls

The syntax for a function call is the name of a function (possibly qualified with a schema name), followed by its argument list enclosed in parentheses:

function_name ([*expression* [, *expression* ...]])

For example, the following computes the square root of 2:

`sqrt(2)`

The list of built-in functions is in Chapter 9. Other functions can be added by the user.

When issuing queries in a database where some users mistrust other users, observe security precautions from Section 10.3 when writing function calls.

The arguments can optionally have names attached. See Section 4.3 for details.

Note

A function that takes a single argument of composite type can optionally be called using field-selection syntax, and conversely field selection can be written in functional style. That is, the notations `col(table)` and `table.col` are interchangeable. This behavior is not SQL-standard but is provided in PostgreSQL because it allows use of functions to emulate “computed fields”. For more information see Section 8.16.5.

4.2.7. Aggregate Expressions

An *aggregate expression* represents the application of an aggregate function across the rows selected by a query. An aggregate function reduces multiple inputs to a single output value, such as the sum or average of the inputs. The syntax of an aggregate expression is one of the following:

aggregate_name (*expression* [, ...] [*order_by_clause*]) [FILTER
(*WHERE filter_clause*)]

```
aggregate_name (ALL expression [ , ... ] [ order_by_clause ] )  
  [ FILTER ( WHERE filter_clause ) ]  
aggregate_name (DISTINCT expression [ , ... ] [ order_by_clause ] )  
  [ FILTER ( WHERE filter_clause ) ]  
aggregate_name ( * ) [ FILTER ( WHERE filter_clause ) ]  
aggregate_name ( [ expression [ , ... ] ] ) WITHIN GROUP  
  ( order_by_clause ) [ FILTER ( WHERE filter_clause ) ]
```

where *aggregate_name* is a previously defined aggregate (possibly qualified with a schema name) and *expression* is any value expression that does not itself contain an aggregate expression or a window function call. The optional *order_by_clause* and *filter_clause* are described below.

The first form of aggregate expression invokes the aggregate once for each input row. The second form is the same as the first, since *ALL* is the default. The third form invokes the aggregate once for each distinct value of the expression (or distinct set of values, for multiple expressions) found in the input rows. The fourth form invokes the aggregate once for each input row; since no particular input value is specified, it is generally only useful for the *count(*)* aggregate function. The last form is used with *ordered-set* aggregate functions, which are described below.

Most aggregate functions ignore null inputs, so that rows in which one or more of the expression(s) yield null are discarded. This can be assumed to be true, unless otherwise specified, for all built-in aggregates.

For example, *count(*)* yields the total number of input rows; *count(f1)* yields the number of input rows in which *f1* is non-null, since *count* ignores nulls; and *count(distinct f1)* yields the number of distinct non-null values of *f1*.

Ordinarily, the input rows are fed to the aggregate function in an unspecified order. In many cases this does not matter; for example, *min* produces the same result no matter what order it receives the inputs in. However, some aggregate functions (such as *array_agg* and *string_agg*) produce results that depend on the ordering of the input rows. When using such an aggregate, the optional *order_by_clause* can be used to specify the desired ordering. The *order_by_clause* has the same syntax as for a query-level *ORDER BY* clause, as described in Section 7.5, except that its expressions are always just expressions and cannot be output-column names or numbers. For example:

```
WITH vals (v) AS ( VALUES (1),(3),(4),(3),(2) )  
SELECT array_agg(v ORDER BY v DESC) FROM vals;  
      array_agg  
-----  
      {4,3,3,2,1}
```

Since *jsonb* only keeps the last matching key, ordering of its keys can be significant:

```
WITH vals (k, v) AS ( VALUES ('key0','1'), ('key1','3'),  
                              ('key1','2') )  
SELECT jsonb_object_agg(k, v ORDER BY v) FROM vals;  
      jsonb_object_agg  
-----  
      {"key0": "1", "key1": "3"}
```

When dealing with multiple-argument aggregate functions, note that the *ORDER BY* clause goes after all the aggregate arguments. For example, write this:

```
SELECT string_agg(a, ',' ORDER BY a) FROM table;
```

not this:

```
SELECT string_agg(a ORDER BY a, ',') FROM table; -- incorrect
```

The latter is syntactically valid, but it represents a call of a single-argument aggregate function with two `ORDER BY` keys (the second one being rather useless since it's a constant).

If `DISTINCT` is specified with an *order_by_clause*, `ORDER BY` expressions can only reference columns in the `DISTINCT` list. For example:

```
WITH vals (v) AS ( VALUES (1),(3),(4),(3),(2) )
SELECT array_agg(DISTINCT v ORDER BY v DESC) FROM vals;
array_agg
-----
{4,3,2,1}
```

Placing `ORDER BY` within the aggregate's regular argument list, as described so far, is used when ordering the input rows for general-purpose and statistical aggregates, for which ordering is optional. There is a subclass of aggregate functions called *ordered-set aggregates* for which an *order_by_clause* is *required*, usually because the aggregate's computation is only sensible in terms of a specific ordering of its input rows. Typical examples of ordered-set aggregates include rank and percentile calculations. For an ordered-set aggregate, the *order_by_clause* is written inside `WITHIN GROUP ( . . . )`, as shown in the final syntax alternative above. The expressions in the *order_by_clause* are evaluated once per input row just like regular aggregate arguments, sorted as per the *order_by_clause*'s requirements, and fed to the aggregate function as input arguments. (This is unlike the case for a non-`WITHIN GROUP` *order_by_clause*, which is not treated as argument(s) to the aggregate function.) The argument expressions preceding `WITHIN GROUP`, if any, are called *direct arguments* to distinguish them from the *aggregated arguments* listed in the *order_by_clause*. Unlike regular aggregate arguments, direct arguments are evaluated only once per aggregate call, not once per input row. This means that they can contain variables only if those variables are grouped by `GROUP BY`; this restriction is the same as if the direct arguments were not inside an aggregate expression at all. Direct arguments are typically used for things like percentile fractions, which only make sense as a single value per aggregation calculation. The direct argument list can be empty; in this case, write just `()` not `(*)`. (PostgreSQL will actually accept either spelling, but only the first way conforms to the SQL standard.)

An example of an ordered-set aggregate call is:

```
SELECT percentile_cont(0.5) WITHIN GROUP (ORDER BY income) FROM
households;
percentile_cont
-----
50489
```

which obtains the 50th percentile, or median, value of the `income` column from table `households`. Here, `0.5` is a direct argument; it would make no sense for the percentile fraction to be a value varying across rows.

If `FILTER` is specified, then only the input rows for which the *filter_clause* evaluates to true are fed to the aggregate function; other rows are discarded. For example:

```
SELECT
    count(*) AS unfiltered,
    count(*) FILTER (WHERE i < 5) AS filtered
FROM generate_series(1,10) AS s(i);
unfiltered | filtered
-----+-----
10 | 4
```

(1 row)

The predefined aggregate functions are described in Section 9.21. Other aggregate functions can be added by the user.

An aggregate expression can only appear in the result list or **HAVING** clause of a **SELECT** command. It is forbidden in other clauses, such as **WHERE**, because those clauses are logically evaluated before the results of aggregates are formed.

When an aggregate expression appears in a subquery (see Section 4.2.11 and Section 9.24), the aggregate is normally evaluated over the rows of the subquery. But an exception occurs if the aggregate's arguments (and *filter_clause* if any) contain only outer-level variables: the aggregate then belongs to the nearest such outer level, and is evaluated over the rows of that query. The aggregate expression as a whole is then an outer reference for the subquery it appears in, and acts as a constant over any one evaluation of that subquery. The restriction about appearing only in the result list or **HAVING** clause applies with respect to the query level that the aggregate belongs to.

4.2.8. Window Function Calls

A *window function call* represents the application of an aggregate-like function over some portion of the rows selected by a query. Unlike non-window aggregate calls, this is not tied to grouping of the selected rows into a single output row — each row remains separate in the query output. However the window function has access to all the rows that would be part of the current row's group according to the grouping specification (**PARTITION BY** list) of the window function call. The syntax of a window function call is one of the following:

```
function_name ([expression [, expression ... ]]) [ FILTER
  ( WHERE filter_clause ) ] OVER window_name
function_name ([expression [, expression ... ]]) [ FILTER
  ( WHERE filter_clause ) ] OVER ( window_definition )
function_name ( * ) [ FILTER ( WHERE filter_clause ) ]
  OVER window_name
function_name ( * ) [ FILTER ( WHERE filter_clause ) ] OVER
  ( window_definition )
```

where *window_definition* has the syntax

```
[ existing_window_name ]
[ PARTITION BY expression [, ...] ]
[ ORDER BY expression [ ASC | DESC | USING operator ] [ NULLS
  { FIRST | LAST } ] [, ...] ]
[ frame_clause ]
```

The optional *frame_clause* can be one of

```
{ RANGE | ROWS | GROUPS } frame_start [ frame_exclusion ]
{ RANGE | ROWS | GROUPS } BETWEEN frame_start AND frame_end
[ frame_exclusion ]
```

where *frame_start* and *frame_end* can be one of

```
UNBOUNDED PRECEDING
offset PRECEDING
CURRENT ROW
offset FOLLOWING
```

UNBOUNDED FOLLOWING

and *frame_exclusion* can be one of

EXCLUDE CURRENT ROW
EXCLUDE GROUP
EXCLUDE TIES
EXCLUDE NO OTHERS

Here, *expression* represents any value expression that does not itself contain window function calls.

window_name is a reference to a named window specification defined in the query's **WINDOW** clause. Alternatively, a full *window_definition* can be given within parentheses, using the same syntax as for defining a named window in the **WINDOW** clause; see the **SELECT** reference page for details. It's worth pointing out that **OVER wname** is not exactly equivalent to **OVER (wname . . .)**; the latter implies copying and modifying the window definition, and will be rejected if the referenced window specification includes a frame clause.

The **PARTITION BY** clause groups the rows of the query into *partitions*, which are processed separately by the window function. **PARTITION BY** works similarly to a query-level **GROUP BY** clause, except that its expressions are always just expressions and cannot be output-column names or numbers. Without **PARTITION BY**, all rows produced by the query are treated as a single partition. The **ORDER BY** clause determines the order in which the rows of a partition are processed by the window function. It works similarly to a query-level **ORDER BY** clause, but likewise cannot use output-column names or numbers. Without **ORDER BY**, rows are processed in an unspecified order.

The *frame_clause* specifies the set of rows constituting the *window frame*, which is a subset of the current partition, for those window functions that act on the frame instead of the whole partition. The set of rows in the frame can vary depending on which row is the current row. The frame can be specified in **RANGE**, **ROWS** or **GROUPS** mode; in each case, it runs from the *frame_start* to the *frame_end*. If *frame_end* is omitted, the end defaults to **CURRENT ROW**.

A *frame_start* of **UNBOUNDED PRECEDING** means that the frame starts with the first row of the partition, and similarly a *frame_end* of **UNBOUNDED FOLLOWING** means that the frame ends with the last row of the partition.

In **RANGE** or **GROUPS** mode, a *frame_start* of **CURRENT ROW** means the frame starts with the current row's first *peer* row (a row that the window's **ORDER BY** clause sorts as equivalent to the current row), while a *frame_end* of **CURRENT ROW** means the frame ends with the current row's last peer row. In **ROWS** mode, **CURRENT ROW** simply means the current row.

In the *offset* **PRECEDING** and *offset* **FOLLOWING** frame options, the *offset* must be an expression not containing any variables, aggregate functions, or window functions. The meaning of the *offset* depends on the frame mode:

- In **ROWS** mode, the *offset* must yield a non-null, non-negative integer, and the option means that the frame starts or ends the specified number of rows before or after the current row.
- In **GROUPS** mode, the *offset* again must yield a non-null, non-negative integer, and the option means that the frame starts or ends the specified number of *peer groups* before or after the current row's peer group, where a peer group is a set of rows that are equivalent in the **ORDER BY** ordering. (There must be an **ORDER BY** clause in the window definition to use **GROUPS** mode.)
- In **RANGE** mode, these options require that the **ORDER BY** clause specify exactly one column. The *offset* specifies the maximum difference between the value of that column in the current row and its value in preceding or following rows of the frame. The data type of the *offset* expression varies depending on the data type of the ordering column. For numeric ordering columns it is typically of the same type as the ordering column, but for datetime ordering columns it is an **interval**.

For example, if the ordering column is of type `date` or `timestamp`, one could write `RANGE BETWEEN '1 day' PRECEDING AND '10 days' FOLLOWING`. The *offset* is still required to be non-null and non-negative, though the meaning of “non-negative” depends on its data type.

In any case, the distance to the end of the frame is limited by the distance to the end of the partition, so that for rows near the partition ends the frame might contain fewer rows than elsewhere.

Notice that in both `ROWS` and `GROUPS` mode, `0 PRECEDING` and `0 FOLLOWING` are equivalent to `CURRENT ROW`. This normally holds in `RANGE` mode as well, for an appropriate data-type-specific meaning of “zero”.

The *frame_exclusion* option allows rows around the current row to be excluded from the frame, even if they would be included according to the frame start and frame end options. `EXCLUDE CURRENT ROW` excludes the current row from the frame. `EXCLUDE GROUP` excludes the current row and its ordering peers from the frame. `EXCLUDE TIES` excludes any peers of the current row from the frame, but not the current row itself. `EXCLUDE NO OTHERS` simply specifies explicitly the default behavior of not excluding the current row or its peers.

The default framing option is `RANGE UNBOUNDED PRECEDING`, which is the same as `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`. With `ORDER BY`, this sets the frame to be all rows from the partition start up through the current row's last `ORDER BY` peer. Without `ORDER BY`, this means all rows of the partition are included in the window frame, since all rows become peers of the current row.

Restrictions are that *frame_start* cannot be `UNBOUNDED FOLLOWING`, *frame_end* cannot be `UNBOUNDED PRECEDING`, and the *frame_end* choice cannot appear earlier in the above list of *frame_start* and *frame_end* options than the *frame_start* choice does — for example `RANGE BETWEEN CURRENT ROW AND offset PRECEDING` is not allowed. But, for example, `ROWS BETWEEN 7 PRECEDING AND 8 PRECEDING` is allowed, even though it would never select any rows.

If `FILTER` is specified, then only the input rows for which the *filter_clause* evaluates to true are fed to the window function; other rows are discarded. Only window functions that are aggregates accept a `FILTER` clause.

The built-in window functions are described in Table 9.67. Other window functions can be added by the user. Also, any built-in or user-defined general-purpose or statistical aggregate can be used as a window function. (Ordered-set and hypothetical-set aggregates cannot presently be used as window functions.)

The syntaxes using `*` are used for calling parameter-less aggregate functions as window functions, for example `count(*) OVER (PARTITION BY x ORDER BY y)`. The asterisk (`*`) is customarily not used for window-specific functions. Window-specific functions do not allow `DISTINCT` or `ORDER BY` to be used within the function argument list.

Window function calls are permitted only in the `SELECT` list and the `ORDER BY` clause of the query.

More information about window functions can be found in Section 3.5, Section 9.22, and Section 7.2.5.

4.2.9. Type Casts

A type cast specifies a conversion from one data type to another. PostgreSQL accepts two equivalent syntaxes for type casts:

```
CAST ( expression AS type )
expression::type
```

The `CAST` syntax conforms to SQL; the syntax with `::` is historical PostgreSQL usage.

When a cast is applied to a value expression of a known type, it represents a run-time type conversion. The cast will succeed only if a suitable type conversion operation has been defined. Notice that this is subtly different from the use of casts with constants, as shown in Section 4.1.2.7. A cast applied to an unadorned string literal represents the initial assignment of a type to a literal constant value, and so it will succeed for any type (if the contents of the string literal are acceptable input syntax for the data type).

An explicit type cast can usually be omitted if there is no ambiguity as to the type that a value expression must produce (for example, when it is assigned to a table column); the system will automatically apply a type cast in such cases. However, automatic casting is only done for casts that are marked “OK to apply implicitly” in the system catalogs. Other casts must be invoked with explicit casting syntax. This restriction is intended to prevent surprising conversions from being applied silently.

It is also possible to specify a type cast using a function-like syntax:

typename (*expression*)

However, this only works for types whose names are also valid as function names. For example, `double precision` cannot be used this way, but the equivalent `float8` can. Also, the names `interval`, `time`, and `timestamp` can only be used in this fashion if they are double-quoted, because of syntactic conflicts. Therefore, the use of the function-like cast syntax leads to inconsistencies and should probably be avoided.

Note

The function-like syntax is in fact just a function call. When one of the two standard cast syntaxes is used to do a run-time conversion, it will internally invoke a registered function to perform the conversion. By convention, these conversion functions have the same name as their output type, and thus the “function-like syntax” is nothing more than a direct invocation of the underlying conversion function. Obviously, this is not something that a portable application should rely on. For further details see `CREATE CAST`.

4.2.10. Collation Expressions

The `COLLATE` clause overrides the collation of an expression. It is appended to the expression it applies to:

expr `COLLATE` *collation*

where *collation* is a possibly schema-qualified identifier. The `COLLATE` clause binds tighter than operators; parentheses can be used when necessary.

If no collation is explicitly specified, the database system either derives a collation from the columns involved in the expression, or it defaults to the default collation of the database if no column is involved in the expression.

The two common uses of the `COLLATE` clause are overriding the sort order in an `ORDER BY` clause, for example:

```
SELECT a, b, c FROM tbl WHERE ... ORDER BY a COLLATE "C";
```

and overriding the collation of a function or operator call that has locale-sensitive results, for example:

```
SELECT * FROM tbl WHERE a > 'foo' COLLATE "C";
```

Note that in the latter case the `COLLATE` clause is attached to an input argument of the operator we wish to affect. It doesn't matter which argument of the operator or function call the `COLLATE` clause is attached to, because the collation that is applied by the operator or function is derived by considering all arguments, and an explicit `COLLATE` clause will override the collations of all other arguments. (Attaching non-matching `COLLATE` clauses to more than one argument, however, is an error. For more details see Section 23.2.) Thus, this gives the same result as the previous example:

```
SELECT * FROM tbl WHERE a COLLATE "C" > 'foo';
```

But this is an error:

```
SELECT * FROM tbl WHERE (a > 'foo') COLLATE "C";
```

because it attempts to apply a collation to the result of the `>` operator, which is of the non-collatable data type `boolean`.

4.2.11. Scalar Subqueries

A scalar subquery is an ordinary `SELECT` query in parentheses that returns exactly one row with one column. (See Chapter 7 for information about writing queries.) The `SELECT` query is executed and the single returned value is used in the surrounding value expression. It is an error to use a query that returns more than one row or more than one column as a scalar subquery. (But if, during a particular execution, the subquery returns no rows, there is no error; the scalar result is taken to be null.) The subquery can refer to variables from the surrounding query, which will act as constants during any one evaluation of the subquery. See also Section 9.24 for other expressions involving subqueries.

For example, the following finds the largest city population in each state:

```
SELECT name, (SELECT max(pop) FROM cities WHERE cities.state =
             states.name)
FROM states;
```

4.2.12. Array Constructors

An array constructor is an expression that builds an array value using values for its member elements. A simple array constructor consists of the key word `ARRAY`, a left square bracket `[`, a list of expressions (separated by commas) for the array element values, and finally a right square bracket `]`. For example:

```
SELECT ARRAY[1, 2, 3+4];
array
-----
{1, 2, 7}
(1 row)
```

By default, the array element type is the common type of the member expressions, determined using the same rules as for `UNION` or `CASE` constructs (see Section 10.5). You can override this by explicitly casting the array constructor to the desired type, for example:

```
SELECT ARRAY[1, 2, 22.7]::integer[];
array
-----
{1, 2, 23}
```

(1 row)

This has the same effect as casting each expression to the array element type individually. For more on casting, see Section 4.2.9.

Multidimensional array values can be built by nesting array constructors. In the inner constructors, the key word `ARRAY` can be omitted. For example, these produce the same result:

```
SELECT ARRAY[ARRAY[1,2], ARRAY[3,4]];
      array
-----
 {{1,2},{3,4}}
(1 row)
```

```
SELECT ARRAY[[1,2],[3,4]];
      array
-----
 {{1,2},{3,4}}
(1 row)
```

Since multidimensional arrays must be rectangular, inner constructors at the same level must produce sub-arrays of identical dimensions. Any cast applied to the outer `ARRAY` constructor propagates automatically to all the inner constructors.

Multidimensional array constructor elements can be anything yielding an array of the proper kind, not only a sub-`ARRAY` construct. For example:

```
CREATE TABLE arr(f1 int[], f2 int[]);

INSERT INTO arr VALUES (ARRAY[[1,2],[3,4]], ARRAY[[5,6],[7,8]]);

SELECT ARRAY[f1, f2, '{{9,10},{11,12}}'::int[]] FROM arr;
      array
-----
 {{{1,2},{3,4}},{5,6},{7,8}},{9,10},{11,12}}}
(1 row)
```

You can construct an empty array, but since it's impossible to have an array with no type, you must explicitly cast your empty array to the desired type. For example:

```
SELECT ARRAY[]::integer[];
      array
-----
 {}
(1 row)
```

It is also possible to construct an array from the results of a subquery. In this form, the array constructor is written with the key word `ARRAY` followed by a parenthesized (not bracketed) subquery. For example:

```
SELECT ARRAY(SELECT oid FROM pg_proc WHERE proname LIKE 'bytea%');
      array
-----
 {2011,1954,1948,1952,1951,1244,1950,2005,1949,1953,2006,31,2412}
(1 row)
```

```
SELECT ARRAY(SELECT ARRAY[i, i*2] FROM generate_series(1,5) AS
a(i));
```

```
array
```

```
-----
{{1,2},{2,4},{3,6},{4,8},{5,10}}
(1 row)
```

The subquery must return a single column. If the subquery's output column is of a non-array type, the resulting one-dimensional array will have an element for each row in the subquery result, with an element type matching that of the subquery's output column. If the subquery's output column is of an array type, the result will be an array of the same type but one higher dimension; in this case all the subquery rows must yield arrays of identical dimensionality, else the result would not be rectangular.

The subscripts of an array value built with `ARRAY` always begin with one. For more information about arrays, see Section 8.15.

4.2.13. Row Constructors

A row constructor is an expression that builds a row value (also called a composite value) using values for its member fields. A row constructor consists of the key word `ROW`, a left parenthesis, zero or more expressions (separated by commas) for the row field values, and finally a right parenthesis. For example:

```
SELECT ROW(1,2.5,'this is a test');
```

The key word `ROW` is optional when there is more than one expression in the list.

A row constructor can include the syntax *rowvalue*.*, which will be expanded to a list of the elements of the row value, just as occurs when the .* syntax is used at the top level of a `SELECT` list (see Section 8.16.5). For example, if table `t` has columns `f1` and `f2`, these are the same:

```
SELECT ROW(t.*, 42) FROM t;
SELECT ROW(t.f1, t.f2, 42) FROM t;
```

Note

Before PostgreSQL 8.2, the .* syntax was not expanded in row constructors, so that writing `ROW(t.*, 42)` created a two-field row whose first field was another row value. The new behavior is usually more useful. If you need the old behavior of nested row values, write the inner row value without .*, for instance `ROW(t, 42)`.

By default, the value created by a `ROW` expression is of an anonymous record type. If necessary, it can be cast to a named composite type — either the row type of a table, or a composite type created with `CREATE TYPE AS`. An explicit cast might be needed to avoid ambiguity. For example:

```
CREATE TABLE mytable(f1 int, f2 float, f3 text);

CREATE FUNCTION getf1(mytable) RETURNS int AS 'SELECT $1.f1'
LANGUAGE SQL;

-- No cast needed since only one getf1() exists
SELECT getf1(ROW(1,2.5,'this is a test'));
getf1
-----
```

```
      1
(1 row)

CREATE TYPE myrowtype AS (f1 int, f2 text, f3 numeric);

CREATE FUNCTION getf1(myrowtype) RETURNS int AS 'SELECT $1.f1'
LANGUAGE SQL;

-- Now we need a cast to indicate which function to call:
SELECT getf1(ROW(1,2.5,'this is a test'));
ERROR:  function getf1(record) is not unique

SELECT getf1(ROW(1,2.5,'this is a test')::mytable);
 getf1
-----
      1
(1 row)

SELECT getf1(CAST(ROW(11,'this is a test',2.5) AS myrowtype));
 getf1
-----
     11
(1 row)
```

Row constructors can be used to build composite values to be stored in a composite-type table column, or to be passed to a function that accepts a composite parameter. Also, it is possible to test rows using the standard comparison operators as described in Section 9.2, to compare one row against another as described in Section 9.25, and to use them in connection with subqueries, as discussed in Section 9.24.

4.2.14. Expression Evaluation Rules

The order of evaluation of subexpressions is not defined. In particular, the inputs of an operator or function are not necessarily evaluated left-to-right or in any other fixed order.

Furthermore, if the result of an expression can be determined by evaluating only some parts of it, then other subexpressions might not be evaluated at all. For instance, if one wrote:

```
SELECT true OR somefunc();
```

then `somefunc()` would (probably) not be called at all. The same would be the case if one wrote:

```
SELECT somefunc() OR true;
```

Note that this is not the same as the left-to-right “short-circuiting” of Boolean operators that is found in some programming languages.

As a consequence, it is unwise to use functions with side effects as part of complex expressions. It is particularly dangerous to rely on side effects or evaluation order in `WHERE` and `HAVING` clauses, since those clauses are extensively reprocessed as part of developing an execution plan. Boolean expressions (`AND/OR/NOT` combinations) in those clauses can be reorganized in any manner allowed by the laws of Boolean algebra.

When it is essential to force evaluation order, a `CASE` construct (see Section 9.18) can be used. For example, this is an untrustworthy way of trying to avoid division by zero in a `WHERE` clause:

```
SELECT ... WHERE x > 0 AND y/x > 1.5;
```

But this is safe:

```
SELECT ... WHERE CASE WHEN x > 0 THEN y/x > 1.5 ELSE false END;
```

A CASE construct used in this fashion will defeat optimization attempts, so it should only be done when necessary. (In this particular example, it would be better to sidestep the problem by writing $y > 1.5 * x$ instead.)

CASE is not a cure-all for such issues, however. One limitation of the technique illustrated above is that it does not prevent early evaluation of constant subexpressions. As described in Section 36.7, functions and operators marked IMMUTABLE can be evaluated when the query is planned rather than when it is executed. Thus for example

```
SELECT CASE WHEN x > 0 THEN x ELSE 1/0 END FROM tab;
```

is likely to result in a division-by-zero failure due to the planner trying to simplify the constant subexpression, even if every row in the table has $x > 0$ so that the ELSE arm would never be entered at run time.

While that particular example might seem silly, related cases that don't obviously involve constants can occur in queries executed within functions, since the values of function arguments and local variables can be inserted into queries as constants for planning purposes. Within PL/pgSQL functions, for example, using an IF-THEN-ELSE statement to protect a risky computation is much safer than just nesting it in a CASE expression.

Another limitation of the same kind is that a CASE cannot prevent evaluation of an aggregate expression contained within it, because aggregate expressions are computed before other expressions in a SELECT list or HAVING clause are considered. For example, the following query can cause a division-by-zero error despite seemingly having protected against it:

```
SELECT CASE WHEN min(employees) > 0
           THEN avg(expenses / employees)
           END
FROM departments;
```

The `min()` and `avg()` aggregates are computed concurrently over all the input rows, so if any row has `employees` equal to zero, the division-by-zero error will occur before there is any opportunity to test the result of `min()`. Instead, use a WHERE or FILTER clause to prevent problematic input rows from reaching an aggregate function in the first place.

4.3. Calling Functions

PostgreSQL allows functions that have named parameters to be called using either *positional* or *named* notation. Named notation is especially useful for functions that have a large number of parameters, since it makes the associations between parameters and actual arguments more explicit and reliable. In positional notation, a function call is written with its argument values in the same order as they are defined in the function declaration. In named notation, the arguments are matched to the function parameters by name and can be written in any order. For each notation, also consider the effect of function argument types, documented in Section 10.3.

In either notation, parameters that have default values given in the function declaration need not be written in the call at all. But this is particularly useful in named notation, since any combination of parameters can be omitted; while in positional notation parameters can only be omitted from right to left.

PostgreSQL also supports *mixed* notation, which combines positional and named notation. In this case, positional parameters are written first and named parameters appear after them.

The following examples will illustrate the usage of all three notations, using the following function definition:

```
CREATE FUNCTION concat_lower_or_upper(a text, b text, uppercase
boolean DEFAULT false)
RETURNS text
AS
$$
    SELECT CASE
        WHEN $3 THEN UPPER($1 || ' ' || $2)
        ELSE LOWER($1 || ' ' || $2)
    END;
$$
LANGUAGE SQL IMMUTABLE STRICT;
```

Function `concat_lower_or_upper` has two mandatory parameters, `a` and `b`. Additionally there is one optional parameter `uppercase` which defaults to `false`. The `a` and `b` inputs will be concatenated, and forced to either upper or lower case depending on the `uppercase` parameter. The remaining details of this function definition are not important here (see Chapter 36 for more information).

4.3.1. Using Positional Notation

Positional notation is the traditional mechanism for passing arguments to functions in PostgreSQL. An example is:

```
SELECT concat_lower_or_upper('Hello', 'World', true);
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

All arguments are specified in order. The result is upper case since `uppercase` is specified as `true`. Another example is:

```
SELECT concat_lower_or_upper('Hello', 'World');
concat_lower_or_upper
-----
hello world
(1 row)
```

Here, the `uppercase` parameter is omitted, so it receives its default value of `false`, resulting in lower case output. In positional notation, arguments can be omitted from right to left so long as they have defaults.

4.3.2. Using Named Notation

In named notation, each argument's name is specified using `=>` to separate it from the argument expression. For example:

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World');
concat_lower_or_upper
-----
hello world
(1 row)
```

Again, the argument `uppercase` was omitted so it is set to `false` implicitly. One advantage of using named notation is that the arguments may be specified in any order, for example:

```
SELECT concat_lower_or_upper(a => 'Hello', b => 'World', uppercase
=> true);
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

```
SELECT concat_lower_or_upper(a => 'Hello', uppercase => true, b =>
'World');
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

An older syntax based on `":="` is supported for backward compatibility:

```
SELECT concat_lower_or_upper(a := 'Hello', uppercase := true, b :=
'World');
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

4.3.3. Using Mixed Notation

The mixed notation combines positional and named notation. However, as already mentioned, named arguments cannot precede positional arguments. For example:

```
SELECT concat_lower_or_upper('Hello', 'World', uppercase => true);
concat_lower_or_upper
-----
HELLO WORLD
(1 row)
```

In the above query, the arguments `a` and `b` are specified positionally, while `uppercase` is specified by name. In this example, that adds little except documentation. With a more complex function having numerous parameters that have default values, named or mixed notation can save a great deal of writing and reduce chances for error.

Note

Named and mixed call notations currently cannot be used when calling an aggregate function (but they do work when an aggregate function is used as a window function).

Chapter 5. Data Definition

This chapter covers how one creates the database structures that will hold one's data. In a relational database, the raw data is stored in tables, so the majority of this chapter is devoted to explaining how tables are created and modified and what features are available to control what data is stored in the tables. Subsequently, we discuss how tables can be organized into schemas, and how privileges can be assigned to tables. Finally, we will briefly look at other features that affect the data storage, such as inheritance, table partitioning, views, functions, and triggers.

5.1. Table Basics

A table in a relational database is much like a table on paper: It consists of rows and columns. The number and order of the columns is fixed, and each column has a name. The number of rows is variable — it reflects how much data is stored at a given moment. SQL does not make any guarantees about the order of the rows in a table. When a table is read, the rows will appear in an unspecified order, unless sorting is explicitly requested. This is covered in Chapter 7. Furthermore, SQL does not assign unique identifiers to rows, so it is possible to have several completely identical rows in a table. This is a consequence of the mathematical model that underlies SQL but is usually not desirable. Later in this chapter we will see how to deal with this issue.

Each column has a data type. The data type constrains the set of possible values that can be assigned to a column and assigns semantics to the data stored in the column so that it can be used for computations. For instance, a column declared to be of a numerical type will not accept arbitrary text strings, and the data stored in such a column can be used for mathematical computations. By contrast, a column declared to be of a character string type will accept almost any kind of data but it does not lend itself to mathematical calculations, although other operations such as string concatenation are available.

PostgreSQL includes a sizable set of built-in data types that fit many applications. Users can also define their own data types. Most built-in data types have obvious names and semantics, so we defer a detailed explanation to Chapter 8. Some of the frequently used data types are `integer` for whole numbers, `numeric` for possibly fractional numbers, `text` for character strings, `date` for dates, `time` for time-of-day values, and `timestamp` for values containing both date and time.

To create a table, you use the aptly named `CREATE TABLE` command. In this command you specify at least a name for the new table, the names of the columns and the data type of each column. For example:

```
CREATE TABLE my_first_table (  
    first_column text,  
    second_column integer  
);
```

This creates a table named `my_first_table` with two columns. The first column is named `first_column` and has a data type of `text`; the second column has the name `second_column` and the type `integer`. The table and column names follow the identifier syntax explained in Section 4.1.1. The type names are usually also identifiers, but there are some exceptions. Note that the column list is comma-separated and surrounded by parentheses.

Of course, the previous example was heavily contrived. Normally, you would give names to your tables and columns that convey what kind of data they store. So let's look at a more realistic example:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric
```

```
);
```

(The `numeric` type can store fractional components, as would be typical of monetary amounts.)

Tip

When you create many interrelated tables it is wise to choose a consistent naming pattern for the tables and columns. For instance, there is a choice of using singular or plural nouns for table names, both of which are favored by some theorist or other.

There is a limit on how many columns a table can contain. Depending on the column types, it is between 250 and 1600. However, defining a table with anywhere near this many columns is highly unusual and often a questionable design.

If you no longer need a table, you can remove it using the `DROP TABLE` command. For example:

```
DROP TABLE my_first_table;  
DROP TABLE products;
```

Attempting to drop a table that does not exist is an error. Nevertheless, it is common in SQL script files to unconditionally try to drop each table before creating it, ignoring any error messages, so that the script works whether or not the table exists. (If you like, you can use the `DROP TABLE IF EXISTS` variant to avoid the error messages, but this is not standard SQL.)

If you need to modify a table that already exists, see Section 5.7 later in this chapter.

With the tools discussed so far you can create fully functional tables. The remainder of this chapter is concerned with adding features to the table definition to ensure data integrity, security, or convenience. If you are eager to fill your tables with data now you can skip ahead to Chapter 6 and read the rest of this chapter later.

5.2. Default Values

A column can be assigned a default value. When a new row is created and no values are specified for some of the columns, those columns will be filled with their respective default values. A data manipulation command can also request explicitly that a column be set to its default value, without having to know what that value is. (Details about data manipulation commands are in Chapter 6.)

If no default value is declared explicitly, the default value is the null value. This usually makes sense because a null value can be considered to represent unknown data.

In a table definition, default values are listed after the column data type. For example:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric DEFAULT 9.99  
);
```

The default value can be an expression, which will be evaluated whenever the default value is inserted (*not* when the table is created). A common example is for a `timestamp` column to have a default of `CURRENT_TIMESTAMP`, so that it gets set to the time of row insertion. Another common example is generating a “serial number” for each row. In PostgreSQL this is typically done by something like:

```
CREATE TABLE products (  
    product_no integer,
```

```
        product_no integer DEFAULT nextval('products_product_no_seq'),  
        ...  
);
```

where the `nextval()` function supplies successive values from a *sequence object* (see Section 9.17). This arrangement is sufficiently common that there's a special shorthand for it:

```
CREATE TABLE products (  
    product_no SERIAL,  
    ...  
);
```

The `SERIAL` shorthand is discussed further in Section 8.1.4.

5.3. Identity Columns

An identity column is a special column that is generated automatically from an implicit sequence. It can be used to generate key values.

To create an identity column, use the `GENERATED ... AS IDENTITY` clause in `CREATE TABLE`, for example:

```
CREATE TABLE people (  
    id bigint GENERATED ALWAYS AS IDENTITY,  
    ...  
);
```

or alternatively

```
CREATE TABLE people (  
    id bigint GENERATED BY DEFAULT AS IDENTITY,  
    ...  
);
```

See `CREATE TABLE` for more details.

If an `INSERT` command is executed on the table with the identity column and no value is explicitly specified for the identity column, then a value generated by the implicit sequence is inserted. For example, with the above definitions and assuming additional appropriate columns, writing

```
INSERT INTO people (name, address) VALUES ('A', 'foo');  
INSERT INTO people (name, address) VALUES ('B', 'bar');
```

would generate values for the `id` column starting at 1 and result in the following table data:

id	name	address
1	A	foo
2	B	bar

Alternatively, the keyword `DEFAULT` can be specified in place of a value to explicitly request the sequence-generated value, like

```
INSERT INTO people (id, name, address) VALUES (DEFAULT, 'C',  
    'baz');
```

Similarly, the keyword **DEFAULT** can be used in **UPDATE** commands.

Thus, in many ways, an identity column behaves like a column with a default value.

The clauses **ALWAYS** and **BY DEFAULT** in the column definition determine how explicitly user-specified values are handled in **INSERT** and **UPDATE** commands. In an **INSERT** command, if **ALWAYS** is selected, a user-specified value is only accepted if the **INSERT** statement specifies **OVERRIDING SYSTEM VALUE**. If **BY DEFAULT** is selected, then the user-specified value takes precedence. Thus, using **BY DEFAULT** results in a behavior more similar to default values, where the default value can be overridden by an explicit value, whereas **ALWAYS** provides some more protection against accidentally inserting an explicit value.

The data type of an identity column must be one of the data types supported by sequences. (See **CREATE SEQUENCE**.) The properties of the associated sequence may be specified when creating an identity column (see **CREATE TABLE**) or changed afterwards (see **ALTER TABLE**).

An identity column is automatically marked as **NOT NULL**. An identity column, however, does not guarantee uniqueness. (A sequence normally returns unique values, but a sequence could be reset, or values could be inserted manually into the identity column, as discussed above.) Uniqueness would need to be enforced using a **PRIMARY KEY** or **UNIQUE** constraint.

In table inheritance hierarchies, identity columns and their properties in a child table are independent of those in its parent tables. A child table does not inherit identity columns or their properties automatically from the parent. During **INSERT** or **UPDATE**, a column is treated as an identity column if that column is an identity column in the table named in the statement, and the corresponding identity properties are applied.

Partitions inherit identity columns from the partitioned table. They cannot have their own identity columns. The properties of a given identity column are consistent across all the partitions in the partition hierarchy.

5.4. Generated Columns

A generated column is a special column that is always computed from other columns. Thus, it is for columns what a view is for tables. There are two kinds of generated columns: stored and virtual. A stored generated column is computed when it is written (inserted or updated) and occupies storage as if it were a normal column. A virtual generated column occupies no storage and is computed when it is read. Thus, a virtual generated column is similar to a view and a stored generated column is similar to a materialized view (except that it is always updated automatically).

To create a generated column, use the **GENERATED ALWAYS AS** clause in **CREATE TABLE**, for example:

```
CREATE TABLE people (  
    ...,  
    height_cm numeric,  
    height_in numeric GENERATED ALWAYS AS (height_cm / 2.54)  
);
```

A generated column is by default of the virtual kind. Use the keywords **VIRTUAL** or **STORED** to make the choice explicit. See **CREATE TABLE** for more details.

A generated column cannot be written to directly. In **INSERT** or **UPDATE** commands, a value cannot be specified for a generated column, but the keyword **DEFAULT** may be specified.

Consider the differences between a column with a default and a generated column. The column default is evaluated once when the row is first inserted if no other value was provided; a generated column is updated whenever the row changes and cannot be overridden. A column default may not refer to other columns of the table; a generation expression would normally do so. A column default can use volatile

functions, for example `random()` or functions referring to the current time; this is not allowed for generated columns.

Several restrictions apply to the definition of generated columns and tables involving generated columns:

- The generation expression can only use immutable functions and cannot use subqueries or reference anything other than the current row in any way.
- A generation expression cannot reference another generated column.
- A generation expression cannot reference a system column, except `tableoid`.
- A virtual generated column cannot have a user-defined type, and the generation expression of a virtual generated column must not reference user-defined functions or types, that is, it can only use built-in functions or types. This applies also indirectly, such as for functions or types that underlie operators or casts. (This restriction does not exist for stored generated columns.)
- A generated column cannot have a column default or an identity definition.
- A generated column cannot be part of a partition key.
- Foreign tables can have generated columns. See `CREATE FOREIGN TABLE` for details.
- For inheritance and partitioning:
 - If a parent column is a generated column, its child column must also be a generated column of the same kind (stored or virtual); however, the child column can have a different generation expression.

For stored generated columns, the generation expression that is actually applied during insert or update of a row is the one associated with the table that the row is physically in. (This is unlike the behavior for column defaults: for those, the default value associated with the table named in the query applies.) For virtual generated columns, the generation expression of the table named in the query applies when a table is read.

- If a parent column is not a generated column, its child column must not be generated either.
- For inherited tables, if you write a child column definition without any `GENERATED` clause in `CREATE TABLE ... INHERITS`, then its `GENERATED` clause will automatically be copied from the parent. `ALTER TABLE ... INHERIT` will insist that parent and child columns already match as to generation status, but it will not require their generation expressions to match.
- Similarly for partitioned tables, if you write a child column definition without any `GENERATED` clause in `CREATE TABLE ... PARTITION OF`, then its `GENERATED` clause will automatically be copied from the parent. `ALTER TABLE ... ATTACH PARTITION` will insist that parent and child columns already match as to generation status, but it will not require their generation expressions to match.
- In case of multiple inheritance, if one parent column is a generated column, then all parent columns must be generated columns. If they do not all have the same generation expression, then the desired expression for the child must be specified explicitly.

Additional considerations apply to the use of generated columns.

- Generated columns maintain access privileges separately from their underlying base columns. So, it is possible to arrange it so that a particular role can read from a generated column but not from the underlying base columns.

For virtual generated columns, this is only fully secure if the generation expression uses only leakproof functions (see `CREATE FUNCTION`), but this is not enforced by the system.

- Privileges of functions used in generation expressions are checked when the expression is actually executed, on write or read respectively, as if the generation expression had been called directly from the query using the generated column. The user of a generated column must have permissions to call all functions used by the generation expression. Functions in the generation expression are executed with the privileges of the user executing the query or the function owner, depending on whether the functions are defined as `SECURITY INVOKER` or `SECURITY DEFINER`.
- Generated columns are, conceptually, updated after `BEFORE` triggers have run. Therefore, changes made to base columns in a `BEFORE` trigger will be reflected in generated columns. But conversely, it is not allowed to access generated columns in `BEFORE` triggers.
- Generated columns are allowed to be replicated during logical replication according to the `CREATE PUBLICATION` parameter `publish_generated_columns` or by including them in the column list of the `CREATE PUBLICATION` command. This is currently only supported for stored generated columns. See Section 29.6 for details.

5.5. Constraints

Data types are a way to limit the kind of data that can be stored in a table. For many applications, however, the constraint they provide is too coarse. For example, a column containing a product price should probably only accept positive values. But there is no standard data type that accepts only positive numbers. Another issue is that you might want to constrain column data with respect to other columns or rows. For example, in a table containing product information, there should be only one row for each product number.

To that end, SQL allows you to define constraints on columns and tables. Constraints give you as much control over the data in your tables as you wish. If a user attempts to store data in a column that would violate a constraint, an error is raised. This applies even if the value came from the default value definition.

5.5.1. Check Constraints

A check constraint is the most generic constraint type. It allows you to specify that the value in a certain column must satisfy a Boolean (truth-value) expression. For instance, to require positive product prices, you could use:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric CHECK (price > 0)  
);
```

As you see, the constraint definition comes after the data type, just like default value definitions. Default values and constraints can be listed in any order. A check constraint consists of the key word `CHECK` followed by an expression in parentheses. The check constraint expression should involve the column thus constrained, otherwise the constraint would not make too much sense.

You can also give the constraint a separate name. This clarifies error messages and allows you to refer to the constraint when you need to change it. The syntax is:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric CONSTRAINT positive_price CHECK (price > 0)  
);
```

So, to specify a named constraint, use the key word **CONSTRAINT** followed by an identifier followed by the constraint definition. (If you don't specify a constraint name in this way, the system chooses a name for you.)

A check constraint can also refer to several columns. Say you store a regular price and a discounted price, and you want to ensure that the discounted price is lower than the regular price:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric CHECK (price > 0),  
    discounted_price numeric CHECK (discounted_price > 0),  
    CHECK (price > discounted_price)  
);
```

The first two constraints should look familiar. The third one uses a new syntax. It is not attached to a particular column, instead it appears as a separate item in the comma-separated column list. Column definitions and these constraint definitions can be listed in mixed order.

We say that the first two constraints are column constraints, whereas the third one is a table constraint because it is written separately from any one column definition. Column constraints can also be written as table constraints, while the reverse is not necessarily possible, since a column constraint is supposed to refer to only the column it is attached to. (PostgreSQL doesn't enforce that rule, but you should follow it if you want your table definitions to work with other database systems.) The above example could also be written as:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric,  
    CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0),  
    CHECK (price > discounted_price)  
);
```

or even:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric CHECK (price > 0),  
    discounted_price numeric,  
    CHECK (discounted_price > 0 AND price > discounted_price)  
);
```

It's a matter of taste.

Names can be assigned to table constraints in the same way as column constraints:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric,  
    CHECK (price > 0),  
    discounted_price numeric,
```

```
CHECK (discounted_price > 0),  
CONSTRAINT valid_discount CHECK (price > discounted_price)  
);
```

It should be noted that a check constraint is satisfied if the check expression evaluates to true or the null value. Since most expressions will evaluate to the null value if any operand is null, they will not prevent null values in the constrained columns. To ensure that a column does not contain null values, the not-null constraint described in the next section can be used.

Note

PostgreSQL does not support **CHECK** constraints that reference table data other than the new or updated row being checked. While a **CHECK** constraint that violates this rule may appear to work in simple tests, it cannot guarantee that the database will not reach a state in which the constraint condition is false (due to subsequent changes of the other row(s) involved). This would cause a database dump and restore to fail. The restore could fail even when the complete database state is consistent with the constraint, due to rows not being loaded in an order that will satisfy the constraint. If possible, use **UNIQUE**, **EXCLUDE**, or **FOREIGN KEY** constraints to express cross-row and cross-table restrictions.

If what you desire is a one-time check against other rows at row insertion, rather than a continuously-maintained consistency guarantee, a custom trigger can be used to implement that. (This approach avoids the dump/restore problem because `pg_dump` does not reinstall triggers until after restoring data, so that the check will not be enforced during a dump/restore.)

Note

PostgreSQL assumes that **CHECK** constraints' conditions are immutable, that is, they will always give the same result for the same input row. This assumption is what justifies examining **CHECK** constraints only when rows are inserted or updated, and not at other times. (The warning above about not referencing other table data is really a special case of this restriction.)

An example of a common way to break this assumption is to reference a user-defined function in a **CHECK** expression, and then change the behavior of that function. PostgreSQL does not disallow that, but it will not notice if there are rows in the table that now violate the **CHECK** constraint. That would cause a subsequent database dump and restore to fail. The recommended way to handle such a change is to drop the constraint (using **ALTER TABLE**), adjust the function definition, and re-add the constraint, thereby rechecking it against all table rows.

5.5.2. Not-Null Constraints

A not-null constraint simply specifies that a column must not assume the null value. A syntax example:

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name text NOT NULL,  
    price numeric  
);
```

An explicit constraint name can also be specified, for example:

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name text CONSTRAINT products_name_not_null NOT NULL,  
    price numeric  
);
```

A not-null constraint is usually written as a column constraint. The syntax for writing it as a table constraint is

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric,  
    NOT NULL product_no,  
    NOT NULL name  
);
```

But this syntax is not standard and mainly intended for use by `pg_dump`.

A not-null constraint is functionally equivalent to creating a check constraint `CHECK (column_name IS NOT NULL)`, but in PostgreSQL creating an explicit not-null constraint is more efficient.

Of course, a column can have more than one constraint. Just write the constraints one after another:

```
CREATE TABLE products (  
    product_no integer NOT NULL,  
    name text NOT NULL,  
    price numeric NOT NULL CHECK (price > 0)  
);
```

The order doesn't matter. It does not necessarily determine in which order the constraints are checked.

However, a column can have at most one explicit not-null constraint.

The `NOT NULL` constraint has an inverse: the `NULL` constraint. This does not mean that the column must be null, which would surely be useless. Instead, this simply selects the default behavior that the column might be null. The `NULL` constraint is not present in the SQL standard and should not be used in portable applications. (It was only added to PostgreSQL to be compatible with some other database systems.) Some users, however, like it because it makes it easy to toggle the constraint in a script file. For example, you could start with:

```
CREATE TABLE products (  
    product_no integer NULL,  
    name text NULL,  
    price numeric NULL  
);
```

and then insert the `NOT` key word where desired.

Tip

In most database designs the majority of columns should be marked not null.

5.5.3. Unique Constraints

Unique constraints ensure that the data contained in a column, or a group of columns, is unique among all the rows in the table. The syntax is:

```
CREATE TABLE products (  
    product_no integer UNIQUE,  
    name text,  
    price numeric  
);
```

when written as a column constraint, and:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric,  
    UNIQUE (product_no)  
);
```

when written as a table constraint.

To define a unique constraint for a group of columns, write it as a table constraint with the column names separated by commas:

```
CREATE TABLE example (  
    a integer,  
    b integer,  
    c integer,  
    UNIQUE (a, c)  
);
```

This specifies that the combination of values in the indicated columns is unique across the whole table, though any one of the columns need not be (and ordinarily isn't) unique.

You can assign your own name for a unique constraint, in the usual way:

```
CREATE TABLE products (  
    product_no integer CONSTRAINT must_be_different UNIQUE,  
    name text,  
    price numeric  
);
```

Adding a unique constraint will automatically create a unique B-tree index on the column or group of columns listed in the constraint. A uniqueness restriction covering only some rows cannot be written as a unique constraint, but it is possible to enforce such a restriction by creating a unique partial index.

In general, a unique constraint is violated if there is more than one row in the table where the values of all of the columns included in the constraint are equal. By default, two null values are not considered equal in this comparison. That means even in the presence of a unique constraint it is possible to store duplicate rows that contain a null value in at least one of the constrained columns. This behavior can be changed by adding the clause **NULLS NOT DISTINCT**, like

```
CREATE TABLE products (  
    product_no integer UNIQUE NULLS NOT DISTINCT,
```

```
        name text,  
        price numeric  
    );  
  
or
```

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric,  
    UNIQUE NULLS NOT DISTINCT (product_no)  
);
```

The default behavior can be specified explicitly using **NULLS DISTINCT**. The default null treatment in unique constraints is implementation-defined according to the SQL standard, and other implementations have a different behavior. So be careful when developing applications that are intended to be portable.

5.5.4. Primary Keys

A primary key constraint indicates that a column, or group of columns, can be used as a unique identifier for rows in the table. This requires that the values be both unique and not null. So, the following two table definitions accept the same data:

```
CREATE TABLE products (  
    product_no integer UNIQUE NOT NULL,  
    name text,  
    price numeric  
);
```

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name text,  
    price numeric  
);
```

Primary keys can span more than one column; the syntax is similar to unique constraints:

```
CREATE TABLE example (  
    a integer,  
    b integer,  
    c integer,  
    PRIMARY KEY (a, c)  
);
```

Adding a primary key will automatically create a unique B-tree index on the column or group of columns listed in the primary key, and will force the column(s) to be marked **NOT NULL**.

A table can have at most one primary key. (There can be any number of unique constraints, which combined with not-null constraints are functionally almost the same thing, but only one can be identified as the primary key.) Relational database theory dictates that every table must have a primary key. This rule is not enforced by PostgreSQL, but it is usually best to follow it.

Primary keys are useful both for documentation purposes and for client applications. For example, a GUI application that allows modifying row values probably needs to know the primary key of a table

to be able to identify rows uniquely. There are also various ways in which the database system makes use of a primary key if one has been declared; for example, the primary key defines the default target column(s) for foreign keys referencing its table.

5.5.5. Foreign Keys

A foreign key constraint specifies that the values in a column (or a group of columns) must match the values appearing in some row of another table. We say this maintains the *referential integrity* between two related tables.

Say you have the product table that we have used several times already:

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name text,  
    price numeric  
);
```

Let's also assume you have a table storing orders of those products. We want to ensure that the orders table only contains orders of products that actually exist. So we define a foreign key constraint in the orders table that references the products table:

```
CREATE TABLE orders (  
    order_id integer PRIMARY KEY,  
    product_no integer REFERENCES products (product_no),  
    quantity integer  
);
```

Now it is impossible to create orders with non-NULL product_no entries that do not appear in the products table.

We say that in this situation the orders table is the *referencing* table and the products table is the *referenced* table. Similarly, there are referencing and referenced columns.

You can also shorten the above command to:

```
CREATE TABLE orders (  
    order_id integer PRIMARY KEY,  
    product_no integer REFERENCES products,  
    quantity integer  
);
```

because in absence of a column list the primary key of the referenced table is used as the referenced column(s).

You can assign your own name for a foreign key constraint, in the usual way.

A foreign key can also constrain and reference a group of columns. As usual, it then needs to be written in table constraint form. Here is a contrived syntax example:

```
CREATE TABLE t1 (  
    a integer PRIMARY KEY,  
    b integer,  
    c integer,  
    FOREIGN KEY (b, c) REFERENCES other_table (c1, c2)  
);
```

Of course, the number and type of the constrained columns need to match the number and type of the referenced columns.

Sometimes it is useful for the “other table” of a foreign key constraint to be the same table; this is called a *self-referential* foreign key. For example, if you want rows of a table to represent nodes of a tree structure, you could write

```
CREATE TABLE tree (  
    node_id integer PRIMARY KEY,  
    parent_id integer REFERENCES tree,  
    name text,  
    ...  
);
```

A top-level node would have NULL `parent_id`, while non-NULL `parent_id` entries would be constrained to reference valid rows of the table.

A table can have more than one foreign key constraint. This is used to implement many-to-many relationships between tables. Say you have tables about products and orders, but now you want to allow one order to contain possibly many products (which the structure above did not allow). You could use this table structure:

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name text,  
    price numeric  
);  
  
CREATE TABLE orders (  
    order_id integer PRIMARY KEY,  
    shipping_address text,  
    ...  
);  
  
CREATE TABLE order_items (  
    product_no integer REFERENCES products,  
    order_id integer REFERENCES orders,  
    quantity integer,  
    PRIMARY KEY (product_no, order_id)  
);
```

Notice that the primary key overlaps with the foreign keys in the last table.

We know that the foreign keys disallow creation of orders that do not relate to any products. But what if a product is removed after an order is created that references it? SQL allows you to handle that as well. Intuitively, we have a few options:

- Disallow deleting a referenced product
- Delete the orders as well
- Something else?

To illustrate this, let's implement the following policy on the many-to-many relationship example above: when someone wants to remove a product that is still referenced by an order (via `order_items`), we disallow it. If someone removes an order, the order items are removed as well:

```
CREATE TABLE products (  
    product_no integer PRIMARY KEY,  
    name text,  
    price numeric  
);  
  
CREATE TABLE orders (  
    order_id integer PRIMARY KEY,  
    shipping_address text,  
    ...  
);  
  
CREATE TABLE order_items (  
    product_no integer REFERENCES products ON DELETE RESTRICT,  
    order_id integer REFERENCES orders ON DELETE CASCADE,  
    quantity integer,  
    PRIMARY KEY (product_no, order_id)  
);
```

The default **ON DELETE** action is **ON DELETE NO ACTION**; this does not need to be specified. This means that the deletion in the referenced table is allowed to proceed. But the foreign-key constraint is still required to be satisfied, so this operation will usually result in an error. But checking of foreign-key constraints can also be deferred to later in the transaction (not covered in this chapter). In that case, the **NO ACTION** setting would allow other commands to “fix” the situation before the constraint is checked, for example by inserting another suitable row into the referenced table or by deleting the now-dangling rows from the referencing table.

RESTRICT is a stricter setting than **NO ACTION**. It prevents deletion of a referenced row. **RESTRICT** does not allow the check to be deferred until later in the transaction.

CASCADE specifies that when a referenced row is deleted, row(s) referencing it should be automatically deleted as well.

There are two other options: **SET NULL** and **SET DEFAULT**. These cause the referencing column(s) in the referencing row(s) to be set to nulls or their default values, respectively, when the referenced row is deleted. Note that these do not excuse you from observing any constraints. For example, if an action specifies **SET DEFAULT** but the default value would not satisfy the foreign key constraint, the operation will fail.

The appropriate choice of **ON DELETE** action depends on what kinds of objects the related tables represent. When the referencing table represents something that is a component of what is represented by the referenced table and cannot exist independently, then **CASCADE** could be appropriate. If the two tables represent independent objects, then **RESTRICT** or **NO ACTION** is more appropriate; an application that actually wants to delete both objects would then have to be explicit about this and run two delete commands. In the above example, order items are part of an order, and it is convenient if they are deleted automatically if an order is deleted. But products and orders are different things, and so making a deletion of a product automatically cause the deletion of some order items could be considered problematic. The actions **SET NULL** or **SET DEFAULT** can be appropriate if a foreign-key relationship represents optional information. For example, if the products table contained a reference to a product manager, and the product manager entry gets deleted, then setting the product's product manager to null or a default might be useful.

The actions **SET NULL** and **SET DEFAULT** can take a column list to specify which columns to set. Normally, all columns of the foreign-key constraint are set; setting only a subset is useful in some special cases. Consider the following example:

```
CREATE TABLE tenants (  
    tenant_id integer PRIMARY KEY
```

```
);

CREATE TABLE users (
    tenant_id integer REFERENCES tenants ON DELETE CASCADE,
    user_id integer NOT NULL,
    PRIMARY KEY (tenant_id, user_id)
);

CREATE TABLE posts (
    tenant_id integer REFERENCES tenants ON DELETE CASCADE,
    post_id integer NOT NULL,
    author_id integer,
    PRIMARY KEY (tenant_id, post_id),
    FOREIGN KEY (tenant_id, author_id) REFERENCES users ON DELETE
    SET NULL (author_id)
);
```

Without the specification of the column, the foreign key would also set the column `tenant_id` to null, but that column is still required as part of the primary key.

Analogous to `ON DELETE` there is also `ON UPDATE` which is invoked when a referenced column is changed (updated). The possible actions are the same, except that column lists cannot be specified for `SET NULL` and `SET DEFAULT`. In this case, `CASCADE` means that the updated values of the referenced column(s) should be copied into the referencing row(s). There is also a noticeable difference between `ON UPDATE NO ACTION` (the default) and `ON UPDATE RESTRICT`. The former will allow the update to proceed and the foreign-key constraint will be checked against the state after the update. The latter will prevent the update to run even if the state after the update would still satisfy the constraint. This prevents updating a referenced row to a value that is distinct but compares as equal (for example, a character string with a different case variant, if a character string type with a case-insensitive collation is used).

Normally, a referencing row need not satisfy the foreign key constraint if any of its referencing columns are null. If `MATCH FULL` is added to the foreign key declaration, a referencing row escapes satisfying the constraint only if all its referencing columns are null (so a mix of null and non-null values is guaranteed to fail a `MATCH FULL` constraint). If you don't want referencing rows to be able to avoid satisfying the foreign key constraint, declare the referencing column(s) as `NOT NULL`.

A foreign key must reference columns that either are a primary key or form a unique constraint, or are columns from a non-partial unique index. This means that the referenced columns always have an index to allow efficient lookups on whether a referencing row has a match. Since a `DELETE` of a row from the referenced table or an `UPDATE` of a referenced column will require a scan of the referencing table for rows matching the old value, it is often a good idea to index the referencing columns too. Because this is not always needed, and there are many choices available on how to index, the declaration of a foreign key constraint does not automatically create an index on the referencing columns.

More information about updating and deleting data is in Chapter 6. Also see the description of foreign key constraint syntax in the reference documentation for `CREATE TABLE`.

5.5.6. Exclusion Constraints

Exclusion constraints ensure that if any two rows are compared on the specified columns or expressions using the specified operators, at least one of these operator comparisons will return false or null. The syntax is:

```
CREATE TABLE circles (
    c circle,
    EXCLUDE USING gist (c WITH &&)
```

);

See also `CREATE TABLE ... CONSTRAINT ... EXCLUDE` for details.

Adding an exclusion constraint will automatically create an index of the type specified in the constraint declaration.

5.6. System Columns

Every table has several *system columns* that are implicitly defined by the system. Therefore, these names cannot be used as names of user-defined columns. (Note that these restrictions are separate from whether the name is a key word or not; quoting a name will not allow you to escape these restrictions.) You do not really need to be concerned about these columns; just know they exist.

`tableoid`

The OID of the table containing this row. This column is particularly handy for queries that select from partitioned tables (see Section 5.12) or inheritance hierarchies (see Section 5.11), since without it, it's difficult to tell which individual table a row came from. The `tableoid` can be joined against the `oid` column of `pg_class` to obtain the table name.

`xmin`

The identity (transaction ID) of the inserting transaction for this row version. (A row version is an individual state of a row; each update of a row creates a new row version for the same logical row.)

`cmin`

The command identifier (starting at zero) within the inserting transaction.

`xmax`

The identity (transaction ID) of the deleting transaction, or zero for an undeleted row version. It is possible for this column to be nonzero in a visible row version. That usually indicates that the deleting transaction hasn't committed yet, or that an attempted deletion was rolled back.

`cmax`

The command identifier within the deleting transaction, or zero.

`ctid`

The physical location of the row version within its table. Note that although the `ctid` can be used to locate the row version very quickly, a row's `ctid` will change if it is updated or moved by `VACUUM FULL`. Therefore `ctid` should not be used as a row identifier. A primary key should be used to identify logical rows.

Transaction identifiers are also 32-bit quantities. In a long-lived database it is possible for transaction IDs to wrap around. This is not a fatal problem given appropriate maintenance procedures; see Chapter 24 for details. It is unwise, however, to depend on the uniqueness of transaction IDs over the long term (more than one billion transactions).

Command identifiers are also 32-bit quantities. This creates a hard limit of 2^{32} (4 billion) SQL commands within a single transaction. In practice this limit is not a problem — note that the limit is on the number of SQL commands, not the number of rows processed. Also, only commands that actually modify the database contents will consume a command identifier.

5.7. Modifying Tables

When you create a table and you realize that you made a mistake, or the requirements of the application change, you can drop the table and create it again. But this is not a convenient option if the table is already filled with data, or if the table is referenced by other database objects (for instance a foreign key constraint). Therefore PostgreSQL provides a family of commands to make modifications to existing tables. Note that this is conceptually distinct from altering the data contained in the table: here we are interested in altering the definition, or structure, of the table.

You can:

- Add columns
- Remove columns
- Add constraints
- Remove constraints
- Change default values
- Change column data types
- Rename columns
- Rename tables

All these actions are performed using the `ALTER TABLE` command, whose reference page contains details beyond those given here.

5.7.1. Adding a Column

To add a column, use a command like:

```
ALTER TABLE products ADD COLUMN description text;
```

The new column is initially filled with whatever default value is given (null if you don't specify a `DEFAULT` clause).

Tip

Adding a column with a constant default value does not require each row of the table to be updated when the `ALTER TABLE` statement is executed. Instead, the default value will be returned the next time the row is accessed, and applied when the table is rewritten, making the `ALTER TABLE` very fast even on large tables.

If the default value is volatile (e.g., `clock_timestamp()`) each row will need to be updated with the value calculated at the time `ALTER TABLE` is executed. To avoid a potentially lengthy update operation, particularly if you intend to fill the column with mostly nondefault values anyway, it may be preferable to add the column with no default, insert the correct values using `UPDATE`, and then add any desired default as described below.

You can also define constraints on the column at the same time, using the usual syntax:

```
ALTER TABLE products ADD COLUMN description text CHECK (description <> '');
```

In fact all the options that can be applied to a column description in `CREATE TABLE` can be used here. Keep in mind however that the default value must satisfy the given constraints, or the `ADD` will fail. Alternatively, you can add constraints later (see below) after you've filled in the new column correctly.

5.7.2. Removing a Column

To remove a column, use a command like:

```
ALTER TABLE products DROP COLUMN description;
```

Whatever data was in the column disappears. Table constraints involving the column are dropped, too. However, if the column is referenced by a foreign key constraint of another table, PostgreSQL will not silently drop that constraint. You can authorize dropping everything that depends on the column by adding `CASCADE`:

```
ALTER TABLE products DROP COLUMN description CASCADE;
```

See Section 5.15 for a description of the general mechanism behind this.

5.7.3. Adding a Constraint

To add a constraint, the table constraint syntax is used. For example:

```
ALTER TABLE products ADD CHECK (name <> '');  
ALTER TABLE products ADD CONSTRAINT some_name UNIQUE (product_no);  
ALTER TABLE products ADD FOREIGN KEY (product_group_id) REFERENCES  
    product_groups;
```

To add a not-null constraint, which is normally not written as a table constraint, this special syntax is available:

```
ALTER TABLE products ALTER COLUMN product_no SET NOT NULL;
```

This command silently does nothing if the column already has a not-null constraint.

The constraint will be checked immediately, so the table data must satisfy the constraint before it can be added.

5.7.4. Removing a Constraint

To remove a constraint you need to know its name. If you gave it a name then that's easy. Otherwise the system assigned a generated name, which you need to find out. The `psql` command `\d tablename` can be helpful here; other interfaces might also provide a way to inspect table details. Then the command is:

```
ALTER TABLE products DROP CONSTRAINT some_name;
```

As with dropping a column, you need to add `CASCADE` if you want to drop a constraint that something else depends on. An example is that a foreign key constraint depends on a unique or primary key constraint on the referenced column(s).

Simplified syntax is available to drop a not-null constraint:

```
ALTER TABLE products ALTER COLUMN product_no DROP NOT NULL;
```

This mirrors the `SET NOT NULL` syntax for adding a not-null constraint. This command will silently do nothing if the column does not have a not-null constraint. (Recall that a column can have at most one not-null constraint, so it is never ambiguous which constraint this command acts on.)

5.7.5. Changing a Column's Default Value

To set a new default for a column, use a command like:

```
ALTER TABLE products ALTER COLUMN price SET DEFAULT 7.77;
```

Note that this doesn't affect any existing rows in the table, it just changes the default for future `INSERT` commands.

To remove any default value, use:

```
ALTER TABLE products ALTER COLUMN price DROP DEFAULT;
```

This is effectively the same as setting the default to null. As a consequence, it is not an error to drop a default where one hadn't been defined, because the default is implicitly the null value.

5.7.6. Changing a Column's Data Type

To convert a column to a different data type, use a command like:

```
ALTER TABLE products ALTER COLUMN price TYPE numeric(10,2);
```

This will succeed only if each existing entry in the column can be converted to the new type by an implicit cast. If a more complex conversion is needed, you can add a `USING` clause that specifies how to compute the new values from the old.

PostgreSQL will attempt to convert the column's default value (if any) to the new type, as well as any constraints that involve the column. But these conversions might fail, or might produce surprising results. It's often best to drop any constraints on the column before altering its type, and then add back suitably modified constraints afterwards.

5.7.7. Renaming a Column

To rename a column:

```
ALTER TABLE products RENAME COLUMN product_no TO product_number;
```

5.7.8. Renaming a Table

To rename a table:

```
ALTER TABLE products RENAME TO items;
```

5.8. Privileges

When an object is created, it is assigned an owner. The owner is normally the role that executed the creation statement. For most kinds of objects, the initial state is that only the owner (or a superuser) can do anything with the object. To allow other roles to use it, *privileges* must be granted.

There are different kinds of privileges: `SELECT`, `INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `REFERENCES`, `TRIGGER`, `CREATE`, `CONNECT`, `TEMPORARY`, `EXECUTE`, `USAGE`, `SET`, `ALTER SYSTEM`, and `MAINTAIN`. The privileges applicable to a particular object vary depending on the object's type (table, function, etc.). More detail about the meanings of these privileges appears below. The following sections and chapters will also show you how these privileges are used.

The right to modify or destroy an object is inherent in being the object's owner, and cannot be granted or revoked in itself. (However, like all privileges, that right can be inherited by members of the owning role; see Section 21.3.)

An object can be assigned to a new owner with an **ALTER** command of the appropriate kind for the object, for example

```
ALTER TABLE table_name OWNER TO new_owner;
```

Superusers can always do this; ordinary roles can only do it if they are both the current owner of the object (or inherit the privileges of the owning role) and able to **SET ROLE** to the new owning role. All object privileges of the old owner are transferred to the new owner along with the ownership.

To assign privileges, the **GRANT** command is used. For example, if **joe** is an existing role, and **accounts** is an existing table, the privilege to update the table can be granted with:

```
GRANT UPDATE ON accounts TO joe;
```

Writing **ALL** in place of a specific privilege grants all privileges that are relevant for the object type.

The special “role” name **PUBLIC** can be used to grant a privilege to every role on the system. Also, “group” roles can be set up to help manage privileges when there are many users of a database — for details see Chapter 21.

To revoke a previously-granted privilege, use the fittingly named **REVOKE** command:

```
REVOKE ALL ON accounts FROM PUBLIC;
```

Ordinarily, only the object's owner (or a superuser) can grant or revoke privileges on an object. However, it is possible to grant a privilege “with grant option”, which gives the recipient the right to grant it in turn to others. If the grant option is subsequently revoked then all who received the privilege from that recipient (directly or through a chain of grants) will lose the privilege. For details see the **GRANT** and **REVOKE** reference pages.

An object's owner can choose to revoke their own ordinary privileges, for example to make a table read-only for themselves as well as others. But owners are always treated as holding all grant options, so they can always re-grant their own privileges.

The available privileges are:

SELECT

Allows **SELECT** from any column, or specific column(s), of a table, view, materialized view, or other table-like object. Also allows use of **COPY TO**. This privilege is also needed to reference existing column values in **UPDATE**, **DELETE**, or **MERGE**. For sequences, this privilege also allows use of the **currval** function. For large objects, this privilege allows the object to be read.

INSERT

Allows **INSERT** of a new row into a table, view, etc. Can be granted on specific column(s), in which case only those columns may be assigned to in the **INSERT** command (other columns will therefore receive default values). Also allows use of **COPY FROM**.

UPDATE

Allows **UPDATE** of any column, or specific column(s), of a table, view, etc. (In practice, any nontrivial **UPDATE** command will require **SELECT** privilege as well, since it must reference table columns to determine which rows to update, and/or to compute new values for columns.) **SELECT ... FOR UPDATE** and **SELECT ... FOR SHARE** also require this privilege on at least one column, in addition to the **SELECT** privilege. For sequences, this privilege allows use of the **nextval** and **setval** functions. For large objects, this privilege allows writing or truncating the object.

DELETE

Allows DELETE of a row from a table, view, etc. (In practice, any nontrivial DELETE command will require SELECT privilege as well, since it must reference table columns to determine which rows to delete.)

TRUNCATE

Allows TRUNCATE on a table.

REFERENCES

Allows creation of a foreign key constraint referencing a table, or specific column(s) of a table. Great care should be taken when granting this privilege, since a user who creates a foreign key can arrange for enforcement of that foreign key to call an arbitrary function, such as a cast function, and such functions will be called with the privileges of the table owner.

TRIGGER

Allows creation of a trigger on a table, view, etc. Great care should be taken when granting this privilege, since any triggers added to a table or view will be executed with the privileges of users who modify it.

CREATE

For databases, allows new schemas and publications to be created within the database, and allows trusted extensions to be installed within the database.

For schemas, allows new objects to be created within the schema. To rename an existing object, you must own the object *and* have this privilege for the containing schema.

For tablespaces, allows tables, indexes, and temporary files to be created within the tablespace, and allows databases to be created that have the tablespace as their default tablespace.

Note that revoking this privilege will not alter the existence or location of existing objects.

CONNECT

Allows the grantee to connect to the database. This privilege is checked at connection startup (in addition to checking any restrictions imposed by `pg_hba.conf`).

TEMPORARY

Allows temporary tables to be created while using the database.

EXECUTE

Allows calling a function or procedure, including use of any operators that are implemented on top of the function. This is the only type of privilege that is applicable to functions and procedures.

USAGE

For procedural languages, allows use of the language for the creation of functions in that language. This is the only type of privilege that is applicable to procedural languages.

For schemas, allows access to objects contained in the schema (assuming that the objects' own privilege requirements are also met). Essentially this allows the grantee to “look up” objects within the schema. Without this permission, it is still possible to see the object names, e.g., by querying system catalogs. Also, after revoking this permission, existing sessions might have statements that have previously performed this lookup, so this is not a completely secure way to prevent object access.

For sequences, allows use of the `currval` and `nextval` functions.

For types and domains, allows use of the type or domain in the creation of tables, functions, and other schema objects. (Note that this privilege does not control all “usage” of the type, such as values of the type appearing in queries. It only prevents objects from being created that depend on the type. The main purpose of this privilege is controlling which users can create dependencies on a type, which could prevent the owner from changing the type later.)

For foreign-data wrappers, allows creation of new servers using the foreign-data wrapper.

For foreign servers, allows creation of foreign tables using the server. Grantees may also create, alter, or drop their own user mappings associated with that server.

SET

Allows a server configuration parameter to be set to a new value within the current session. (While this privilege can be granted on any parameter, it is meaningless except for parameters that would normally require superuser privilege to set.)

ALTER SYSTEM

Allows a server configuration parameter to be configured to a new value using the `ALTER SYSTEM` command.

MAINTAIN

Allows `VACUUM`, `ANALYZE`, `CLUSTER`, `REFRESH MATERIALIZED VIEW`, `REINDEX`, `LOCK TABLE`, and database object statistics manipulation functions (see Table 9.105) on a relation.

The privileges required by other commands are listed on the reference page of the respective command.

PostgreSQL grants privileges on some types of objects to `PUBLIC` by default when the objects are created. No privileges are granted to `PUBLIC` by default on tables, table columns, sequences, foreign data wrappers, foreign servers, large objects, schemas, tablespaces, or configuration parameters. For other types of objects, the default privileges granted to `PUBLIC` are as follows: `CONNECT` and `TEMPORARY` (create temporary tables) privileges for databases; `EXECUTE` privilege for functions and procedures; and `USAGE` privilege for languages and data types (including domains). The object owner can, of course, `REVOKE` both default and expressly granted privileges. (For maximum security, issue the `REVOKE` in the same transaction that creates the object; then there is no window in which another user can use the object.) Also, these default privilege settings can be overridden using the `ALTER DEFAULT PRIVILEGES` command.

Table 5.1 shows the one-letter abbreviations that are used for these privilege types in `ACL` values. You will see these letters in the output of the `psql` commands listed below, or when looking at `ACL` columns of system catalogs.

Table 5.1. ACL Privilege Abbreviations

Privilege	Abbreviation	Applicable Object Types
SELECT	r (“read”)	LARGE OBJECT, SEQUENCE, TABLE (and table-like objects), table column
INSERT	a (“append”)	TABLE, table column
UPDATE	w (“write”)	LARGE OBJECT, SEQUENCE, TABLE, table column
DELETE	d	TABLE
TRUNCATE	D	TABLE
REFERENCES	x	TABLE, table column
TRIGGER	t	TABLE

Privilege	Abbreviation	Applicable Object Types
CREATE	C	DATABASE, SCHEMA, TABLESPACE
CONNECT	c	DATABASE
TEMPORARY	T	DATABASE
EXECUTE	X	FUNCTION, PROCEDURE
USAGE	U	DOMAIN, FOREIGN DATA WRAPPER, FOREIGN SERVER, LANGUAGE, SCHEMA, SEQUENCE, TYPE
SET	s	PARAMETER
ALTER SYSTEM	A	PARAMETER
MAINTAIN	m	TABLE

Table 5.2 summarizes the privileges available for each type of SQL object, using the abbreviations shown above. It also shows the `psql` command that can be used to examine privilege settings for each object type.

Table 5.2. Summary of Access Privileges

Object Type	All Privileges	Default PUBLIC Privileges	psql Command
DATABASE	CTc	Tc	\l
DOMAIN	U	U	\dD+
FUNCTION or PROCEDURE	X	X	\df+
FOREIGN DATA WRAPPER	U	none	\dew+
FOREIGN SERVER	U	none	\des+
LANGUAGE	U	U	\dL+
LARGE OBJECT	rw	none	\dl+
PARAMETER	SA	none	\dconfig+
SCHEMA	UC	none	\dn+
SEQUENCE	rwU	none	\dp
TABLE (and table-like objects)	arwdDxtm	none	\dp
Table column	arwx	none	\dp
TABLESPACE	C	none	\db+
TYPE	U	U	\dT+

The privileges that have been granted for a particular object are displayed as a list of `aclitem` entries, each having the format:

grantee=privilege-abbreviation[].../grantor*

Each `aclitem` lists all the permissions of one grantee that have been granted by a particular grantor. Specific privileges are represented by one-letter abbreviations from Table 5.1, with `*` appended if the privilege was granted with grant option. For example, `calvin=r*w/hobbes` specifies that the role `calvin` has the privilege `SELECT` (`r`) with grant option (`*`) as well as the non-grantable privilege `UPDATE` (`w`), both granted by the role `hobbes`. If `calvin` also has some privileges on the same object granted by a different grantor, those would appear as a separate `aclitem` entry. An empty grantee field in an `aclitem` stands for `PUBLIC`.

As an example, suppose that user `miriam` creates table `mytable` and does:

```
GRANT SELECT ON mytable TO PUBLIC;
GRANT SELECT, UPDATE, INSERT ON mytable TO admin;
GRANT SELECT (col1), UPDATE (col1) ON mytable TO miriam_rw;
```

Then psql's \dp command would show:

```
=> \dp mytable
```

Schema	Name	Type	Access privileges	Column
privileges		Policies		
public	mytable	table	miriam=arwdDxtm/miriam+	col1:
			=r/miriam	
miriam_rw			rw/miriam	
			admin=arw/miriam	

(1 row)

If the “Access privileges” column is empty for a given object, it means the object has default privileges (that is, its privileges entry in the relevant system catalog is null). Default privileges always include all privileges for the owner, and can include some privileges for **PUBLIC** depending on the object type, as explained above. The first **GRANT** or **REVOKE** on an object will instantiate the default privileges (producing, for example, `miriam=arwdDxt/miriam`) and then modify them per the specified request. Similarly, entries are shown in “Column privileges” only for columns with nondefault privileges. (Note: for this purpose, “default privileges” always means the built-in default privileges for the object's type. An object whose privileges have been affected by an **ALTER DEFAULT PRIVILEGES** command will always be shown with an explicit privilege entry that includes the effects of the **ALTER**.)

Notice that the owner's implicit grant options are not marked in the access privileges display. A ***** will appear only when grant options have been explicitly granted to someone.

The “Access privileges” column shows **(none)** when the object's privileges entry is non-null but empty. This means that no privileges are granted at all, even to the object's owner — a rare situation. (The owner still has implicit grant options in this case, and so could re-grant her own privileges; but she has none at the moment.)

5.9. Row Security Policies

In addition to the SQL-standard privilege system available through **GRANT**, tables can have *row security policies* that restrict, on a per-user basis, which rows can be returned by normal queries or inserted, updated, or deleted by data modification commands. This feature is also known as *Row-Level Security*. By default, tables do not have any policies, so that if a user has access privileges to a table according to the SQL privilege system, all rows within it are equally available for querying or updating.

When row security is enabled on a table (with **ALTER TABLE ... ENABLE ROW LEVEL SECURITY**), all normal access to the table for selecting rows or modifying rows must be allowed by a row security policy. (However, the table's owner is typically not subject to row security policies.) If no policy exists for the table, a default-deny policy is used, meaning that no rows are visible or can be modified. Operations that apply to the whole table, such as **TRUNCATE** and **REFERENCES**, are not subject to row security.

Row security policies can be specific to commands, or to roles, or to both. A policy can be specified to apply to **ALL** commands, or to **SELECT**, **INSERT**, **UPDATE**, or **DELETE**. Multiple roles can be assigned to a given policy, and normal role membership and inheritance rules apply.

To specify which rows are visible or modifiable according to a policy, an expression is required that returns a Boolean result. This expression will be evaluated for each row prior to any conditions or functions coming from the user's query. (The only exceptions to this rule are `leakproof` functions, which are guaranteed to not leak information; the optimizer may choose to apply such functions ahead of the row-security check.) Rows for which the expression does not return `true` will not be processed. Separate expressions may be specified to provide independent control over the rows which are visible and the rows which are allowed to be modified. Policy expressions are run as part of the query and with the privileges of the user running the query, although security-definer functions can be used to access data not available to the calling user.

Superusers and roles with the `BYPASSRLS` attribute always bypass the row security system when accessing a table. Table owners normally bypass row security as well, though a table owner can choose to be subject to row security with `ALTER TABLE ... FORCE ROW LEVEL SECURITY`.

Enabling and disabling row security, as well as adding policies to a table, is always the privilege of the table owner only.

Policies are created using the `CREATE POLICY` command, altered using the `ALTER POLICY` command, and dropped using the `DROP POLICY` command. To enable and disable row security for a given table, use the `ALTER TABLE` command.

Each policy has a name and multiple policies can be defined for a table. As policies are table-specific, each policy for a table must have a unique name. Different tables may have policies with the same name.

When multiple policies apply to a given query, they are combined using either `OR` (for permissive policies, which are the default) or using `AND` (for restrictive policies). The `OR` behavior is similar to the rule that a given role has the privileges of all roles that they are a member of. Permissive vs. restrictive policies are discussed further below.

As a simple example, here is how to create a policy on the `account` relation to allow only members of the `managers` role to access rows, and only rows of their accounts:

```
CREATE TABLE accounts (manager text, company text, contact_email
text);
```

```
ALTER TABLE accounts ENABLE ROW LEVEL SECURITY;
```

```
CREATE POLICY account_managers ON accounts TO managers
USING (manager = current_user);
```

The policy above implicitly provides a `WITH CHECK` clause identical to its `USING` clause, so that the constraint applies both to rows selected by a command (so a manager cannot `SELECT`, `UPDATE`, or `DELETE` existing rows belonging to a different manager) and to rows modified by a command (so rows belonging to a different manager cannot be created via `INSERT` or `UPDATE`).

If no role is specified, or the special user name `PUBLIC` is used, then the policy applies to all users on the system. To allow all users to access only their own row in a `users` table, a simple policy can be used:

```
CREATE POLICY user_policy ON users
USING (user_name = current_user);
```

This works similarly to the previous example.

To use a different policy for rows that are being added to the table compared to those rows that are visible, multiple policies can be combined. This pair of policies would allow all users to view all rows in the `users` table, but only modify their own:

```
CREATE POLICY user_sel_policy ON users
  FOR SELECT
  USING (true);
CREATE POLICY user_mod_policy ON users
  USING (user_name = current_user);
```

In a `SELECT` command, these two policies are combined using `OR`, with the net effect being that all rows can be selected. In other command types, only the second policy applies, so that the effects are the same as before.

Row security can also be disabled with the `ALTER TABLE` command. Disabling row security does not remove any policies that are defined on the table; they are simply ignored. Then all rows in the table are visible and modifiable, subject to the standard SQL privileges system.

Below is a larger example of how this feature can be used in production environments. The table `passwd` emulates a Unix password file:

```
-- Simple passwd-file based example
CREATE TABLE passwd (
  user_name      text UNIQUE NOT NULL,
  pwhash         text,
  uid            int  PRIMARY KEY,
  gid            int  NOT NULL,
  real_name      text NOT NULL,
  home_phone     text,
  extra_info     text,
  home_dir       text NOT NULL,
  shell          text NOT NULL
);

CREATE ROLE admin; -- Administrator
CREATE ROLE bob;   -- Normal user
CREATE ROLE alice; -- Normal user

-- Populate the table
INSERT INTO passwd VALUES
  ('admin', 'xxx', 0, 0, 'Admin', '111-222-3333', null, '/root', '/bin/
dash');
INSERT INTO passwd VALUES
  ('bob', 'xxx', 1, 1, 'Bob', '123-456-7890', null, '/home/bob', '/bin/
zsh');
INSERT INTO passwd VALUES
  ('alice', 'xxx', 2, 1, 'Alice', '098-765-4321', null, '/home/alice', '/
bin/zsh');

-- Be sure to enable row-level security on the table
ALTER TABLE passwd ENABLE ROW LEVEL SECURITY;

-- Create policies
-- Administrator can see all rows and add any rows
CREATE POLICY admin_all ON passwd TO admin USING (true) WITH CHECK
(true);
-- Normal users can view all rows
CREATE POLICY all_view ON passwd FOR SELECT USING (true);
-- Normal users can update their own records, but
-- limit which shells a normal user is allowed to set
CREATE POLICY user_mod ON passwd FOR UPDATE
```

```

    USING (current_user = user_name)
    WITH CHECK (
        current_user = user_name AND
        shell IN ('/bin/bash', '/bin/sh', '/bin/dash', '/bin/zsh', '/bin/
tcsh')
    );

-- Allow admin all normal rights
GRANT SELECT, INSERT, UPDATE, DELETE ON passwd TO admin;
-- Users only get select access on public columns
GRANT SELECT
    (user_name, uid, gid, real_name, home_phone, extra_info,
    home_dir, shell)
    ON passwd TO public;
-- Allow users to update certain columns
GRANT UPDATE
    (pwhash, real_name, home_phone, extra_info, shell)
    ON passwd TO public;

```

As with any security settings, it's important to test and ensure that the system is behaving as expected. Using the example above, this demonstrates that the permission system is working properly.

```

-- admin can view all rows and fields
postgres=> set role admin;
SET
postgres=> table passwd;
user_name | pwhash | uid | gid | real_name | home_phone |
extra_info | home_dir | shell
-----+-----+-----+-----+-----+-----+-----
+-----+-----+-----+-----+-----+-----+-----
admin      | xxx    | 0   | 0   | Admin     | 111-222-3333 |
| /root    | /bin/dash
bob        | xxx    | 1   | 1   | Bob       | 123-456-7890 |
| /home/bob | /bin/zsh
alice      | xxx    | 2   | 1   | Alice     | 098-765-4321 |
| /home/alice | /bin/zsh
(3 rows)

-- Test what Alice is able to do
postgres=> set role alice;
SET
postgres=> table passwd;
ERROR: permission denied for table passwd
postgres=> select
    user_name, real_name, home_phone, extra_info, home_dir, shell from
passwd;
user_name | real_name | home_phone | extra_info | home_dir |
shell
-----+-----+-----+-----+-----+-----
+-----+-----+-----+-----+-----+-----
admin      | Admin     | 111-222-3333 |          | /root    |
| /bin/dash
bob        | Bob       | 123-456-7890 |          | /home/bob |
| /bin/zsh
alice      | Alice     | 098-765-4321 |          | /home/alice |
| /bin/zsh
(3 rows)

```

```
postgres=> update passwd set user_name = 'joe';
ERROR: permission denied for table passwd
-- Alice is allowed to change her own real_name, but no others
postgres=> update passwd set real_name = 'Alice Doe';
UPDATE 1
postgres=> update passwd set real_name = 'John Doe' where user_name
= 'admin';
UPDATE 0
postgres=> update passwd set shell = '/bin/xx';
ERROR: new row violates WITH CHECK OPTION for "passwd"
postgres=> delete from passwd;
ERROR: permission denied for table passwd
postgres=> insert into passwd (user_name) values ('xxx');
ERROR: permission denied for table passwd
-- Alice can change her own password; RLS silently prevents
  updating other rows
postgres=> update passwd set pwhash = 'abc';
UPDATE 1
```

All of the policies constructed thus far have been permissive policies, meaning that when multiple policies are applied they are combined using the “OR” Boolean operator. While permissive policies can be constructed to only allow access to rows in the intended cases, it can be simpler to combine permissive policies with restrictive policies (which the records must pass and which are combined using the “AND” Boolean operator). Building on the example above, we add a restrictive policy to require the administrator to be connected over a local Unix socket to access the records of the `passwd` table:

```
CREATE POLICY admin_local_only ON passwd AS RESTRICTIVE TO admin
  USING (pg_catalog.inet_client_addr() IS NULL);
```

We can then see that an administrator connecting over a network will not see any records, due to the restrictive policy:

```
=> SELECT current_user;
  current_user
-----
admin
(1 row)

=> select inet_client_addr();
  inet_client_addr
-----
127.0.0.1
(1 row)

=> TABLE passwd;
 user_name | pwhash | uid | gid | real_name | home_phone |
extra_info | home_dir | shell
-----+-----+-----+-----+-----+-----+-----
(0 rows)

=> UPDATE passwd set pwhash = NULL;
UPDATE 0
```

Referential integrity checks, such as unique or primary key constraints and foreign key references, always bypass row security to ensure that data integrity is maintained. Care must be taken when developing schemas and row level policies to avoid “covert channel” leaks of information through such referential integrity checks.

In some contexts it is important to be sure that row security is not being applied. For example, when taking a backup, it could be disastrous if row security silently caused some rows to be omitted from the backup. In such a situation, you can set the `row_security` configuration parameter to `off`. This does not in itself bypass row security; what it does is throw an error if any query's results would get filtered by a policy. The reason for the error can then be investigated and fixed.

In the examples above, the policy expressions consider only the current values in the row to be accessed or updated. This is the simplest and best-performing case; when possible, it's best to design row security applications to work this way. If it is necessary to consult other rows or other tables to make a policy decision, that can be accomplished using sub-`SELECT`s, or functions that contain `SELECT`s, in the policy expressions. Be aware however that such accesses can create race conditions that could allow information leakage if care is not taken. As an example, consider the following table design:

```
-- definition of privilege groups
CREATE TABLE groups (group_id int PRIMARY KEY,
                     group_name text NOT NULL);

INSERT INTO groups VALUES
  (1, 'low'),
  (2, 'medium'),
  (5, 'high');

GRANT ALL ON groups TO alice; -- alice is the administrator
GRANT SELECT ON groups TO public;

-- definition of users' privilege levels
CREATE TABLE users (user_name text PRIMARY KEY,
                    group_id int NOT NULL REFERENCES groups);

INSERT INTO users VALUES
  ('alice', 5),
  ('bob', 2),
  ('mallory', 2);

GRANT ALL ON users TO alice;
GRANT SELECT ON users TO public;

-- table holding the information to be protected
CREATE TABLE information (info text,
                          group_id int NOT NULL REFERENCES groups);

INSERT INTO information VALUES
  ('barely secret', 1),
  ('slightly secret', 2),
  ('very secret', 5);

ALTER TABLE information ENABLE ROW LEVEL SECURITY;

-- a row should be visible to/updatable by users whose security
  group_id is
-- greater than or equal to the row's group_id
CREATE POLICY fp_s ON information FOR SELECT
```

```
    USING (group_id <= (SELECT group_id FROM users WHERE user_name =
current_user));
CREATE POLICY fp_u ON information FOR UPDATE
    USING (group_id <= (SELECT group_id FROM users WHERE user_name =
current_user));
```

```
-- we rely only on RLS to protect the information table
GRANT ALL ON information TO public;
```

Now suppose that `alice` wishes to change the “slightly secret” information, but decides that `mallory` should not be trusted with the new content of that row, so she does:

```
BEGIN;
UPDATE users SET group_id = 1 WHERE user_name = 'mallory';
UPDATE information SET info = 'secret from mallory' WHERE group_id
= 2;
COMMIT;
```

That looks safe; there is no window wherein `mallory` should be able to see the “secret from mallory” string. However, there is a race condition here. If `mallory` is concurrently doing, say,

```
SELECT * FROM information WHERE group_id = 2 FOR UPDATE;
```

and her transaction is in `READ COMMITTED` mode, it is possible for her to see “secret from mallory”. That happens if her transaction reaches the `information` row just after `alice`’s does. It blocks waiting for `alice`’s transaction to commit, then fetches the updated row contents thanks to the `FOR UPDATE` clause. However, it does *not* fetch an updated row for the implicit `SELECT` from `users`, because that sub-`SELECT` did not have `FOR UPDATE`; instead the `users` row is read with the snapshot taken at the start of the query. Therefore, the policy expression tests the old value of `mallory`’s privilege level and allows her to see the updated row.

There are several ways around this problem. One simple answer is to use `SELECT . . . FOR SHARE` in sub-`SELECT`s in row security policies. However, that requires granting `UPDATE` privilege on the referenced table (here `users`) to the affected users, which might be undesirable. (But another row security policy could be applied to prevent them from actually exercising that privilege; or the sub-`SELECT` could be embedded into a security definer function.) Also, heavy concurrent use of row share locks on the referenced table could pose a performance problem, especially if updates of it are frequent. Another solution, practical if updates of the referenced table are infrequent, is to take an `ACCESS EXCLUSIVE` lock on the referenced table when updating it, so that no concurrent transactions could be examining old row values. Or one could just wait for all concurrent transactions to end after committing an update of the referenced table and before making changes that rely on the new security situation.

For additional details see `CREATE POLICY` and `ALTER TABLE`.

5.10. Schemas

A PostgreSQL database cluster contains one or more named databases. Roles and a few other object types are shared across the entire cluster. A client connection to the server can only access data in a single database, the one specified in the connection request.

Note

Users of a cluster do not necessarily have the privilege to access every database in the cluster. Sharing of role names means that there cannot be different roles named, say, `joe` in two databases in the same cluster; but the system can be configured to allow `joe` access to only some of the databases.

A database contains one or more named *schemas*, which in turn contain tables. Schemas also contain other kinds of named objects, including data types, functions, and operators. Within one schema, two objects of the same type cannot have the same name. Furthermore, tables, sequences, indexes, views, materialized views, and foreign tables share the same namespace, so that, for example, an index and a table must have different names if they are in the same schema. The same object name can be used in different schemas without conflict; for example, both `schema1` and `myschema` can contain tables named `mytable`. Unlike databases, schemas are not rigidly separated: a user can access objects in any of the schemas in the database they are connected to, if they have privileges to do so.

There are several reasons why one might want to use schemas:

- To allow many users to use one database without interfering with each other.
- To organize database objects into logical groups to make them more manageable.
- Third-party applications can be put into separate schemas so they do not collide with the names of other objects.

Schemas are analogous to directories at the operating system level, except that schemas cannot be nested.

5.10.1. Creating a Schema

To create a schema, use the `CREATE SCHEMA` command. Give the schema a name of your choice. For example:

```
CREATE SCHEMA myschema;
```

To create or access objects in a schema, write a *qualified name* consisting of the schema name and table name separated by a dot:

schema.table

This works anywhere a table name is expected, including the table modification commands and the data access commands discussed in the following chapters. (For brevity we will speak of tables only, but the same ideas apply to other kinds of named objects, such as types and functions.)

Actually, the even more general syntax

database.schema.table

can be used too, but at present this is just for pro forma compliance with the SQL standard. If you write a database name, it must be the same as the database you are connected to.

So to create a table in the new schema, use:

```
CREATE TABLE myschema.mytable (  
    ...  
);
```

To drop a schema if it's empty (all objects in it have been dropped), use:

```
DROP SCHEMA myschema;
```

To drop a schema including all contained objects, use:

```
DROP SCHEMA myschema CASCADE;
```

See Section 5.15 for a description of the general mechanism behind this.

Often you will want to create a schema owned by someone else (since this is one of the ways to restrict the activities of your users to well-defined namespaces). The syntax for that is:

```
CREATE SCHEMA schema_name AUTHORIZATION user_name;
```

You can even omit the schema name, in which case the schema name will be the same as the user name. See Section 5.10.6 for how this can be useful.

Schema names beginning with `pg_` are reserved for system purposes and cannot be created by users.

5.10.2. The Public Schema

In the previous sections we created tables without specifying any schema names. By default such tables (and other objects) are automatically put into a schema named “public”. Every new database contains such a schema. Thus, the following are equivalent:

```
CREATE TABLE products ( ... );
```

and:

```
CREATE TABLE public.products ( ... );
```

5.10.3. The Schema Search Path

Qualified names are tedious to write, and it's often best not to wire a particular schema name into applications anyway. Therefore tables are often referred to by *unqualified names*, which consist of just the table name. The system determines which table is meant by following a *search path*, which is a list of schemas to look in. The first matching table in the search path is taken to be the one wanted. If there is no match in the search path, an error is reported, even if matching table names exist in other schemas in the database.

The ability to create like-named objects in different schemas complicates writing a query that references precisely the same objects every time. It also opens up the potential for users to change the behavior of other users' queries, maliciously or accidentally. Due to the prevalence of unqualified names in queries and their use in PostgreSQL internals, adding a schema to `search_path` effectively trusts all users having `CREATE` privilege on that schema. When you run an ordinary query, a malicious user able to create objects in a schema of your search path can take control and execute arbitrary SQL functions as though you executed them.

The first schema named in the search path is called the current schema. Aside from being the first schema searched, it is also the schema in which new tables will be created if the `CREATE TABLE` command does not specify a schema name.

To show the current search path, use the following command:

```
SHOW search_path;
```

In the default setup this returns:

```
search_path
-----
"$user", public
```

The first element specifies that a schema with the same name as the current user is to be searched. If no such schema exists, the entry is ignored. The second element refers to the public schema that we have seen already.

The first schema in the search path that exists is the default location for creating new objects. That is the reason that by default objects are created in the public schema. When objects are referenced in any other context without schema qualification (table modification, data modification, or query commands) the search path is traversed until a matching object is found. Therefore, in the default configuration, any unqualified access again can only refer to the public schema.

To put our new schema in the path, we use:

```
SET search_path TO myschema,public;
```

(We omit the `$user` here because we have no immediate need for it.) And then we can access the table without schema qualification:

```
DROP TABLE mytable;
```

Also, since `myschema` is the first element in the path, new objects would by default be created in it.

We could also have written:

```
SET search_path TO myschema;
```

Then we no longer have access to the public schema without explicit qualification. There is nothing special about the public schema except that it exists by default. It can be dropped, too.

See also Section 9.27 for other ways to manipulate the schema search path.

The search path works in the same way for data type names, function names, and operator names as it does for table names. Data type and function names can be qualified in exactly the same way as table names. If you need to write a qualified operator name in an expression, there is a special provision: you must write

```
OPERATOR(schema.operator)
```

This is needed to avoid syntactic ambiguity. An example is:

```
SELECT 3 OPERATOR(pg_catalog.+) 4;
```

In practice one usually relies on the search path for operators, so as not to have to write anything so ugly as that.

5.10.4. Schemas and Privileges

By default, users cannot access any objects in schemas they do not own. To allow that, the owner of the schema must grant the `USAGE` privilege on the schema. By default, everyone has that privilege

on the schema `public`. To allow users to make use of the objects in a schema, additional privileges might need to be granted, as appropriate for the object.

A user can also be allowed to create objects in someone else's schema. To allow that, the `CREATE` privilege on the schema needs to be granted. In databases upgraded from PostgreSQL 14 or earlier, everyone has that privilege on the schema `public`. Some usage patterns call for revoking that privilege:

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
```

(The first “public” is the schema, the second “public” means “every user”. In the first sense it is an identifier, in the second sense it is a key word, hence the different capitalization; recall the guidelines from Section 4.1.1.)

5.10.5. The System Catalog Schema

In addition to `public` and user-created schemas, each database contains a `pg_catalog` schema, which contains the system tables and all the built-in data types, functions, and operators. `pg_catalog` is always effectively part of the search path. If it is not named explicitly in the path then it is implicitly searched *before* searching the path's schemas. This ensures that built-in names will always be findable. However, you can explicitly place `pg_catalog` at the end of your search path if you prefer to have user-defined names override built-in names.

Since system table names begin with `pg_`, it is best to avoid such names to ensure that you won't suffer a conflict if some future version defines a system table named the same as your table. (With the default search path, an unqualified reference to your table name would then be resolved as the system table instead.) System tables will continue to follow the convention of having names beginning with `pg_`, so that they will not conflict with unqualified user-table names so long as users avoid the `pg_` prefix.

5.10.6. Usage Patterns

Schemas can be used to organize your data in many ways. A *secure schema usage pattern* prevents untrusted users from changing the behavior of other users' queries. When a database does not use a secure schema usage pattern, users wishing to securely query that database would take protective action at the beginning of each session. Specifically, they would begin each session by setting `search_path` to the empty string or otherwise removing schemas that are writable by non-superusers from `search_path`. There are a few usage patterns easily supported by the default configuration:

- Constrain ordinary users to user-private schemas. To implement this pattern, first ensure that no schemas have public `CREATE` privileges. Then, for every user needing to create non-temporary objects, create a schema with the same name as that user, for example `CREATE SCHEMA alice AUTHORIZATION alice`. (Recall that the default search path starts with `$user`, which resolves to the user name. Therefore, if each user has a separate schema, they access their own schemas by default.) This pattern is a secure schema usage pattern unless an untrusted user is the database owner or has been granted `ADMIN OPTION` on a relevant role, in which case no secure schema usage pattern exists.

In PostgreSQL 15 and later, the default configuration supports this usage pattern. In prior versions, or when using a database that has been upgraded from a prior version, you will need to remove the public `CREATE` privilege from the `public` schema (issue `REVOKE CREATE ON SCHEMA public FROM PUBLIC`). Then consider auditing the `public` schema for objects named like objects in schema `pg_catalog`.

- Remove the public schema from the default search path, by modifying `postgresql.conf` or by issuing `ALTER ROLE ALL SET search_path = "$user"`. Then, grant privileges to create in the public schema. Only qualified names will choose public schema objects. While qualified table references are fine, calls to functions in the public schema will be unsafe or unreliable. If you create

functions or extensions in the public schema, use the first pattern instead. Otherwise, like the first pattern, this is secure unless an untrusted user is the database owner or has been granted `ADMIN OPTION` on a relevant role.

- Keep the default search path, and grant privileges to create in the public schema. All users access the public schema implicitly. This simulates the situation where schemas are not available at all, giving a smooth transition from the non-schema-aware world. However, this is never a secure pattern. It is acceptable only when the database has a single user or a few mutually-trusting users. In databases upgraded from PostgreSQL 14 or earlier, this is the default.

For any pattern, to install shared applications (tables to be used by everyone, additional functions provided by third parties, etc.), put them into separate schemas. Remember to grant appropriate privileges to allow the other users to access them. Users can then refer to these additional objects by qualifying the names with a schema name, or they can put the additional schemas into their search path, as they choose.

5.10.7. Portability

In the SQL standard, the notion of objects in the same schema being owned by different users does not exist. Moreover, some implementations do not allow you to create schemas that have a different name than their owner. In fact, the concepts of schema and user are nearly equivalent in a database system that implements only the basic schema support specified in the standard. Therefore, many users consider qualified names to really consist of *user_name.table_name*. This is how PostgreSQL will effectively behave if you create a per-user schema for every user.

Also, there is no concept of a `public` schema in the SQL standard. For maximum conformance to the standard, you should not use the `public` schema.

Of course, some SQL database systems might not implement schemas at all, or provide namespace support by allowing (possibly limited) cross-database access. If you need to work with those systems, then maximum portability would be achieved by not using schemas at all.

5.11. Inheritance

PostgreSQL implements table inheritance, which can be a useful tool for database designers. (SQL:1999 and later define a type inheritance feature, which differs in many respects from the features described here.)

Let's start with an example: suppose we are trying to build a data model for cities. Each state has many cities, but only one capital. We want to be able to quickly retrieve the capital city for any particular state. This can be done by creating two tables, one for state capitals and one for cities that are not capitals. However, what happens when we want to ask for data about a city, regardless of whether it is a capital or not? The inheritance feature can help to resolve this problem. We define the `capitals` table so that it inherits from `cities`:

```
CREATE TABLE cities (  
    name          text,  
    population    float,  
    elevation     int    -- in feet  
);  
  
CREATE TABLE capitals (  
    state         char(2)  
) INHERITS (cities);
```

In this case, the `capitals` table *inherits* all the columns of its parent table, `cities`. State capitals also have an extra column, `state`, that shows their state.

In PostgreSQL, a table can inherit from zero or more other tables, and a query can reference either all rows of a table or all rows of a table plus all of its descendant tables. The latter behavior is the default. For example, the following query finds the names of all cities, including state capitals, that are located at an elevation over 500 feet:

```
SELECT name, elevation
FROM cities
WHERE elevation > 500;
```

Given the sample data from the PostgreSQL tutorial (see Section 2.1), this returns:

name	elevation
Las Vegas	2174
Mariposa	1953
Madison	845

On the other hand, the following query finds all the cities that are not state capitals and are situated at an elevation over 500 feet:

```
SELECT name, elevation
FROM ONLY cities
WHERE elevation > 500;
```

name	elevation
Las Vegas	2174
Mariposa	1953

Here the `ONLY` keyword indicates that the query should apply only to `cities`, and not any tables below `cities` in the inheritance hierarchy. Many of the commands that we have already discussed — `SELECT`, `UPDATE` and `DELETE` — support the `ONLY` keyword.

You can also write the table name with a trailing `*` to explicitly specify that descendant tables are included:

```
SELECT name, elevation
FROM cities*
WHERE elevation > 500;
```

Writing `*` is not necessary, since this behavior is always the default. However, this syntax is still supported for compatibility with older releases where the default could be changed.

In some cases you might wish to know which table a particular row originated from. There is a system column called `tableoid` in each table which can tell you the originating table:

```
SELECT c.tableoid, c.name, c.elevation
FROM cities c
WHERE c.elevation > 500;
```

which returns:

tableoid	name	elevation
139793	Las Vegas	2174

139793		Mariposa		1953
139798		Madison		845

(If you try to reproduce this example, you will probably get different numeric OIDs.) By doing a join with `pg_class` you can see the actual table names:

```
SELECT p.relname, c.name, c.elevation
FROM cities c, pg_class p
WHERE c.elevation > 500 AND c.tableoid = p.oid;
```

which returns:

relname		name		elevation
-----+-----+-----				
cities		Las Vegas		2174
cities		Mariposa		1953
capitals		Madison		845

Another way to get the same effect is to use the `regclass` alias type, which will print the table OID symbolically:

```
SELECT c.tableoid::regclass, c.name, c.elevation
FROM cities c
WHERE c.elevation > 500;
```

Inheritance does not automatically propagate data from `INSERT` or `COPY` commands to other tables in the inheritance hierarchy. In our example, the following `INSERT` statement will fail:

```
INSERT INTO cities (name, population, elevation, state)
VALUES ('Albany', NULL, NULL, 'NY');
```

We might hope that the data would somehow be routed to the `capitals` table, but this does not happen: `INSERT` always inserts into exactly the table specified. In some cases it is possible to redirect the insertion using a rule (see Chapter 39). However that does not help for the above case because the `cities` table does not contain the column `state`, and so the command will be rejected before the rule can be applied.

All check constraints and not-null constraints on a parent table are automatically inherited by its children, unless explicitly specified otherwise with `NO INHERIT` clauses. Other types of constraints (unique, primary key, and foreign key constraints) are not inherited.

A table can inherit from more than one parent table, in which case it has the union of the columns defined by the parent tables. Any columns declared in the child table's definition are added to these. If the same column name appears in multiple parent tables, or in both a parent table and the child's definition, then these columns are “merged” so that there is only one such column in the child table. To be merged, columns must have the same data types, else an error is raised. Inheritable check constraints and not-null constraints are merged in a similar fashion. Thus, for example, a merged column will be marked not-null if any one of the column definitions it came from is marked not-null. Check constraints are merged if they have the same name, and the merge will fail if their conditions are different.

Table inheritance is typically established when the child table is created, using the `INHERITS` clause of the `CREATE TABLE` statement. Alternatively, a table which is already defined in a compatible way can have a new parent relationship added, using the `INHERIT` variant of `ALTER TABLE`. To do this the new child table must already include columns with the same names and types as the columns of the parent. It must also include check constraints with the same names and check expressions as those of the parent. Similarly an inheritance link can be removed from a child using the `NO INHERIT` variant

of `ALTER TABLE`. Dynamically adding and removing inheritance links like this can be useful when the inheritance relationship is being used for table partitioning (see Section 5.12).

One convenient way to create a compatible table that will later be made a new child is to use the `LIKE` clause in `CREATE TABLE`. This creates a new table with the same columns as the source table. If there are any `CHECK` constraints defined on the source table, the `INCLUDING CONSTRAINTS` option to `LIKE` should be specified, as the new child must have constraints matching the parent to be considered compatible.

A parent table cannot be dropped while any of its children remain. Neither can columns or check constraints of child tables be dropped or altered if they are inherited from any parent tables. If you wish to remove a table and all of its descendants, one easy way is to drop the parent table with the `CASCADE` option (see Section 5.15).

`ALTER TABLE` will propagate any changes in column data definitions and check constraints down the inheritance hierarchy. Again, dropping columns that are depended on by other tables is only possible when using the `CASCADE` option. `ALTER TABLE` follows the same rules for duplicate column merging and rejection that apply during `CREATE TABLE`.

Inherited queries perform access permission checks on the parent table only. Thus, for example, granting `UPDATE` permission on the `cities` table implies permission to update rows in the `capitals` table as well, when they are accessed through `cities`. This preserves the appearance that the data is (also) in the parent table. But the `capitals` table could not be updated directly without an additional grant. In a similar way, the parent table's row security policies (see Section 5.9) are applied to rows coming from child tables during an inherited query. A child table's policies, if any, are applied only when it is the table explicitly named in the query; and in that case, any policies attached to its parent(s) are ignored.

Foreign tables (see Section 5.13) can also be part of inheritance hierarchies, either as parent or child tables, just as regular tables can be. If a foreign table is part of an inheritance hierarchy then any operations not supported by the foreign table are not supported on the whole hierarchy either.

5.11.1. Caveats

Note that not all SQL commands are able to work on inheritance hierarchies. Commands that are used for data querying, data modification, or schema modification (e.g., `SELECT`, `UPDATE`, `DELETE`, most variants of `ALTER TABLE`, but not `INSERT` or `ALTER TABLE . . . RENAME`) typically default to including child tables and support the `ONLY` notation to exclude them. The majority of commands that do database maintenance and tuning (e.g., `REINDEX`) only work on individual, physical tables and do not support recursing over inheritance hierarchies. However, both `VACUUM` and `ANALYZE` commands default to including child tables and the `ONLY` notation is supported to allow them to be excluded. The respective behavior of each individual command is documented in its reference page (SQL Commands).

A serious limitation of the inheritance feature is that indexes (including unique constraints) and foreign key constraints only apply to single tables, not to their inheritance children. This is true on both the referencing and referenced sides of a foreign key constraint. Thus, in the terms of the above example:

- If we declared `cities.name` to be `UNIQUE` or a `PRIMARY KEY`, this would not stop the `capitals` table from having rows with names duplicating rows in `cities`. And those duplicate rows would by default show up in queries from `cities`. In fact, by default `capitals` would have no unique constraint at all, and so could contain multiple rows with the same name. You could add a unique constraint to `capitals`, but this would not prevent duplication compared to `cities`.
- Similarly, if we were to specify that `cities.name` `REFERENCES` some other table, this constraint would not automatically propagate to `capitals`. In this case you could work around it by manually adding the same `REFERENCES` constraint to `capitals`.
- Specifying that another table's column `REFERENCES cities(name)` would allow the other table to contain city names, but not capital names. There is no good workaround for this case.

Some functionality not implemented for inheritance hierarchies is implemented for declarative partitioning. Considerable care is needed in deciding whether partitioning with legacy inheritance is useful for your application.

5.12. Table Partitioning

PostgreSQL supports basic table partitioning. This section describes why and how to implement partitioning as part of your database design.

5.12.1. Overview

Partitioning refers to splitting what is logically one large table into smaller physical pieces. Partitioning can provide several benefits:

- Query performance can be improved dramatically in certain situations, particularly when most of the heavily accessed rows of the table are in a single partition or a small number of partitions. Partitioning effectively substitutes for the upper tree levels of indexes, making it more likely that the heavily-used parts of the indexes fit in memory.
- When queries or updates access a large percentage of a single partition, performance can be improved by using a sequential scan of that partition instead of using an index, which would require random-access reads scattered across the whole table.
- Bulk loads and deletes can be accomplished by adding or removing partitions, if the usage pattern is accounted for in the partitioning design. Dropping an individual partition using `DROP TABLE`, or doing `ALTER TABLE DETACH PARTITION`, is far faster than a bulk operation. These commands also entirely avoid the `VACUUM` overhead caused by a bulk `DELETE`.
- Seldom-used data can be migrated to cheaper and slower storage media.

These benefits will normally be worthwhile only when a table would otherwise be very large. The exact point at which a table will benefit from partitioning depends on the application, although a rule of thumb is that the size of the table should exceed the physical memory of the database server.

PostgreSQL offers built-in support for the following forms of partitioning:

Range Partitioning

The table is partitioned into “ranges” defined by a key column or set of columns, with no overlap between the ranges of values assigned to different partitions. For example, one might partition by date ranges, or by ranges of identifiers for particular business objects. Each range's bounds are understood as being inclusive at the lower end and exclusive at the upper end. For example, if one partition's range is from 1 to 10, and the next one's range is from 10 to 20, then value 10 belongs to the second partition not the first.

List Partitioning

The table is partitioned by explicitly listing which key value(s) appear in each partition.

Hash Partitioning

The table is partitioned by specifying a modulus and a remainder for each partition. Each partition will hold the rows for which the hash value of the partition key divided by the specified modulus will produce the specified remainder.

If your application needs to use other forms of partitioning not listed above, alternative methods such as inheritance and `UNION ALL` views can be used instead. Such methods offer flexibility but do not have some of the performance benefits of built-in declarative partitioning.

5.12.2. Declarative Partitioning

PostgreSQL allows you to declare that a table is divided into partitions. The table that is divided is referred to as a *partitioned table*. The declaration includes the *partitioning method* as described above, plus a list of columns or expressions to be used as the *partition key*.

The partitioned table itself is a “virtual” table having no storage of its own. Instead, the storage belongs to *partitions*, which are otherwise-ordinary tables associated with the partitioned table. Each partition stores a subset of the data as defined by its *partition bounds*. All rows inserted into a partitioned table will be routed to the appropriate one of the partitions based on the values of the partition key column(s). Updating the partition key of a row will cause it to be moved into a different partition if it no longer satisfies the partition bounds of its original partition.

Partitions may themselves be defined as partitioned tables, resulting in *sub-partitioning*. Although all partitions must have the same columns as their partitioned parent, partitions may have their own indexes, constraints and default values, distinct from those of other partitions. See `CREATE TABLE` for more details on creating partitioned tables and partitions.

It is not possible to turn a regular table into a partitioned table or vice versa. However, it is possible to add an existing regular or partitioned table as a partition of a partitioned table, or remove a partition from a partitioned table turning it into a standalone table; this can simplify and speed up many maintenance processes. See `ALTER TABLE` to learn more about the `ATTACH PARTITION` and `DETACH PARTITION` sub-commands.

Partitions can also be foreign tables, although considerable care is needed because it is then the user's responsibility that the contents of the foreign table satisfy the partitioning rule. There are some other restrictions as well. See `CREATE FOREIGN TABLE` for more information.

5.12.2.1. Example

Suppose we are constructing a database for a large ice cream company. The company measures peak temperatures every day as well as ice cream sales in each region. Conceptually, we want a table like:

```
CREATE TABLE measurement (  
    city_id      int not null,  
    logdate      date not null,  
    peaktemp     int,  
    unitsales    int  
);
```

We know that most queries will access just the last week's, month's or quarter's data, since the main use of this table will be to prepare online reports for management. To reduce the amount of old data that needs to be stored, we decide to keep only the most recent 3 years worth of data. At the beginning of each month we will remove the oldest month's data. In this situation we can use partitioning to help us meet all of our different requirements for the measurements table.

To use declarative partitioning in this case, use the following steps:

1. Create the `measurement` table as a partitioned table by specifying the `PARTITION BY` clause, which includes the partitioning method (`RANGE` in this case) and the list of column(s) to use as the partition key.

```
CREATE TABLE measurement (  
    city_id      int not null,  
    logdate      date not null,  
    peaktemp     int,  
    unitsales    int
```

```
) PARTITION BY RANGE (logdate);
```

2. Create partitions. Each partition's definition must specify bounds that correspond to the partitioning method and partition key of the parent. Note that specifying bounds such that the new partition's values would overlap with those in one or more existing partitions will cause an error.

Partitions thus created are in every way normal PostgreSQL tables (or, possibly, foreign tables). It is possible to specify a tablespace and storage parameters for each partition separately.

For our example, each partition should hold one month's worth of data, to match the requirement of deleting one month's data at a time. So the commands might look like:

```
CREATE TABLE measurement_y2006m02 PARTITION OF measurement
    FOR VALUES FROM ('2006-02-01') TO ('2006-03-01');
```

```
CREATE TABLE measurement_y2006m03 PARTITION OF measurement
    FOR VALUES FROM ('2006-03-01') TO ('2006-04-01');
```

```
...
```

```
CREATE TABLE measurement_y2007m11 PARTITION OF measurement
    FOR VALUES FROM ('2007-11-01') TO ('2007-12-01');
```

```
CREATE TABLE measurement_y2007m12 PARTITION OF measurement
    FOR VALUES FROM ('2007-12-01') TO ('2008-01-01')
    TABLESPACE fasttablespace;
```

```
CREATE TABLE measurement_y2008m01 PARTITION OF measurement
    FOR VALUES FROM ('2008-01-01') TO ('2008-02-01')
    WITH (parallel_workers = 4)
    TABLESPACE fasttablespace;
```

(Recall that adjacent partitions can share a bound value, since range upper bounds are treated as exclusive bounds.)

If you wish to implement sub-partitioning, again specify the `PARTITION BY` clause in the commands used to create individual partitions, for example:

```
CREATE TABLE measurement_y2006m02 PARTITION OF measurement
    FOR VALUES FROM ('2006-02-01') TO ('2006-03-01')
    PARTITION BY RANGE (peaktemp);
```

After creating partitions of `measurement_y2006m02`, any data inserted into `measurement` that is mapped to `measurement_y2006m02` (or data that is directly inserted into `measurement_y2006m02`, which is allowed provided its partition constraint is satisfied) will be further redirected to one of its partitions based on the `peaktemp` column. The partition key specified may overlap with the parent's partition key, although care should be taken when specifying the bounds of a sub-partition such that the set of data it accepts constitutes a subset of what the partition's own bounds allow; the system does not try to check whether that's really the case.

Inserting data into the parent table that does not map to one of the existing partitions will cause an error; an appropriate partition must be added manually.

It is not necessary to manually create table constraints describing the partition boundary conditions for partitions. Such constraints will be created automatically.

3. Create an index on the key column(s), as well as any other indexes you might want, on the partitioned table. (The key index is not strictly necessary, but in most scenarios it is helpful.) This automatically creates a matching index on each partition, and any partitions you create or attach

later will also have such an index. An index or unique constraint declared on a partitioned table is “virtual” in the same way that the partitioned table is: the actual data is in child indexes on the individual partition tables.

```
CREATE INDEX ON measurement (logdate);
```

4. Ensure that the `enable_partition_pruning` configuration parameter is not disabled in `postgresql.conf`. If it is, queries will not be optimized as desired.

In the above example we would be creating a new partition each month, so it might be wise to write a script that generates the required DDL automatically.

5.12.2.2. Partition Maintenance

Normally the set of partitions established when initially defining the table is not intended to remain static. It is common to want to remove partitions holding old data and periodically add new partitions for new data. One of the most important advantages of partitioning is precisely that it allows this otherwise painful task to be executed nearly instantaneously by manipulating the partition structure, rather than physically moving large amounts of data around.

The simplest option for removing old data is to drop the partition that is no longer necessary:

```
DROP TABLE measurement_y2006m02;
```

This can very quickly delete millions of records because it doesn't have to individually delete every record. Note however that the above command requires taking an `ACCESS EXCLUSIVE` lock on the parent table.

Another option that is often preferable is to remove the partition from the partitioned table but retain access to it as a table in its own right. This has two forms:

```
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02;  
ALTER TABLE measurement DETACH PARTITION measurement_y2006m02  
CONCURRENTLY;
```

These allow further operations to be performed on the data before it is dropped. For example, this is often a useful time to back up the data using `COPY`, `pg_dump`, or similar tools. It might also be a useful time to aggregate data into smaller formats, perform other data manipulations, or run reports. The first form of the command requires an `ACCESS EXCLUSIVE` lock on the parent table. Adding the `CONCURRENTLY` qualifier as in the second form allows the detach operation to require only `SHARE UPDATE EXCLUSIVE` lock on the parent table, but see `ALTER TABLE . . . DETACH PARTITION` for details on the restrictions.

Similarly we can add a new partition to handle new data. We can create an empty partition in the partitioned table just as the original partitions were created above:

```
CREATE TABLE measurement_y2008m02 PARTITION OF measurement  
FOR VALUES FROM ('2008-02-01') TO ('2008-03-01')  
TABLESPACE fasttablespace;
```

As an alternative to creating a new partition, it is sometimes more convenient to create a new table separate from the partition structure and attach it as a partition later. This allows new data to be loaded, checked, and transformed prior to it appearing in the partitioned table. Moreover, the `ATTACH PARTITION` operation requires only a `SHARE UPDATE EXCLUSIVE` lock on the partitioned table rather than the `ACCESS EXCLUSIVE` lock required by `CREATE TABLE . . . PARTITION OF`, so it is more friendly to concurrent operations on the partitioned table; see `ALTER TABLE . . . ATTACH PARTITION` for additional details. The `CREATE TABLE . . . LIKE` option can be helpful to avoid tediously repeating the parent table's definition; for example:

```
CREATE TABLE measurement_y2008m02
  (LIKE measurement INCLUDING DEFAULTS INCLUDING CONSTRAINTS)
  TABLESPACE fasttablespace;

ALTER TABLE measurement_y2008m02 ADD CONSTRAINT y2008m02
  CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE
    '2008-03-01' );

\copy measurement_y2008m02 from 'measurement_y2008m02'
-- possibly some other data preparation work

ALTER TABLE measurement ATTACH PARTITION measurement_y2008m02
  FOR VALUES FROM ( '2008-02-01' ) TO ( '2008-03-01' );
```

Note that when running the `ATTACH PARTITION` command, the table will be scanned to validate the partition constraint while holding an `ACCESS EXCLUSIVE` lock on that partition. As shown above, it is recommended to avoid this scan by creating a `CHECK` constraint matching the expected partition constraint on the table prior to attaching it. Once the `ATTACH PARTITION` is complete, it is recommended to drop the now-redundant `CHECK` constraint. If the table being attached is itself a partitioned table, then each of its sub-partitions will be recursively locked and scanned until either a suitable `CHECK` constraint is encountered or the leaf partitions are reached.

Similarly, if the partitioned table has a `DEFAULT` partition, it is recommended to create a `CHECK` constraint which excludes the to-be-attached partition's constraint. If this is not done, the `DEFAULT` partition will be scanned to verify that it contains no records which should be located in the partition being attached. This operation will be performed whilst holding an `ACCESS EXCLUSIVE` lock on the `DEFAULT` partition. If the `DEFAULT` partition is itself a partitioned table, then each of its partitions will be recursively checked in the same way as the table being attached, as mentioned above.

As mentioned earlier, it is possible to create indexes on partitioned tables so that they are applied automatically to the entire hierarchy. This can be very convenient as not only will all existing partitions be indexed, but any future partitions will be as well. However, one limitation when creating new indexes on partitioned tables is that it is not possible to use the `CONCURRENTLY` qualifier, which could lead to long lock times. To avoid this, you can use `CREATE INDEX ON ONLY` the partitioned table, which creates the new index marked as invalid, preventing automatic application to existing partitions. Instead, indexes can then be created individually on each partition using `CONCURRENTLY` and *attached* to the partitioned index on the parent using `ALTER INDEX ... ATTACH PARTITION`. Once indexes for all the partitions are attached to the parent index, the parent index will be marked valid automatically. Example:

```
CREATE INDEX measurement_usls_idx ON ONLY measurement (unitsales);

CREATE INDEX CONCURRENTLY measurement_usls_2006m02_idx
  ON measurement_y2006m02 (unitsales);
ALTER INDEX measurement_usls_idx
  ATTACH PARTITION measurement_usls_2006m02_idx;
...
```

This technique can be used with `UNIQUE` and `PRIMARY KEY` constraints too; the indexes are created implicitly when the constraint is created. Example:

```
ALTER TABLE ONLY measurement ADD UNIQUE (city_id, logdate);

ALTER TABLE measurement_y2006m02 ADD UNIQUE (city_id, logdate);
ALTER INDEX measurement_city_id_logdate_key
  ATTACH PARTITION measurement_y2006m02_city_id_logdate_key;
...
```

5.12.2.3. Limitations

The following limitations apply to partitioned tables:

- To create a unique or primary key constraint on a partitioned table, the partition keys must not include any expressions or function calls and the constraint's columns must include all of the partition key columns. This limitation exists because the individual indexes making up the constraint can only directly enforce uniqueness within their own partitions; therefore, the partition structure itself must guarantee that there are not duplicates in different partitions.
- Similarly an exclusion constraint must include all the partition key columns. Furthermore the constraint must compare those columns for equality (not e.g. &&). Again, this limitation stems from not being able to enforce cross-partition restrictions. The constraint may include additional columns that aren't part of the partition key, and it may compare those with any operators you like.
- `BEFORE ROW` triggers on `INSERT` cannot change which partition is the final destination for a new row.
- Mixing temporary and permanent relations in the same partition tree is not allowed. Hence, if the partitioned table is permanent, so must be its partitions and likewise if the partitioned table is temporary. When using temporary relations, all members of the partition tree have to be from the same session.

Individual partitions are linked to their partitioned table using inheritance behind-the-scenes. However, it is not possible to use all of the generic features of inheritance with declaratively partitioned tables or their partitions, as discussed below. Notably, a partition cannot have any parents other than the partitioned table it is a partition of, nor can a table inherit from both a partitioned table and a regular table. That means partitioned tables and their partitions never share an inheritance hierarchy with regular tables.

Since a partition hierarchy consisting of the partitioned table and its partitions is still an inheritance hierarchy, `tableoid` and all the normal rules of inheritance apply as described in Section 5.11, with a few exceptions:

- Partitions cannot have columns that are not present in the parent. It is not possible to specify columns when creating partitions with `CREATE TABLE`, nor is it possible to add columns to partitions after-the-fact using `ALTER TABLE`. Tables may be added as a partition with `ALTER TABLE ... ATTACH PARTITION` only if their columns exactly match the parent.
- Both `CHECK` and `NOT NULL` constraints of a partitioned table are always inherited by all its partitions; it is not allowed to create `NO INHERIT` constraints of those types. You cannot drop a constraint of those types if the same constraint is present in the parent table.
- Using `ONLY` to add or drop a constraint on only the partitioned table is supported as long as there are no partitions. Once partitions exist, using `ONLY` will result in an error for any constraints other than `UNIQUE` and `PRIMARY KEY`. Instead, constraints on the partitions themselves can be added and (if they are not present in the parent table) dropped.
- As a partitioned table does not have any data itself, attempts to use `TRUNCATE ONLY` on a partitioned table will always return an error.

5.12.3. Partitioning Using Inheritance

While the built-in declarative partitioning is suitable for most common use cases, there are some circumstances where a more flexible approach may be useful. Partitioning can be implemented using table inheritance, which allows for several features not supported by declarative partitioning, such as:

- For declarative partitioning, partitions must have exactly the same set of columns as the partitioned table, whereas with table inheritance, child tables may have extra columns not present in the parent.
- Table inheritance allows for multiple inheritance.

- Declarative partitioning only supports range, list and hash partitioning, whereas table inheritance allows data to be divided in a manner of the user's choosing. (Note, however, that if constraint exclusion is unable to prune child tables effectively, query performance might be poor.)

5.12.3.1. Example

This example builds a partitioning structure equivalent to the declarative partitioning example above. Use the following steps:

1. Create the “root” table, from which all of the “child” tables will inherit. This table will contain no data. Do not define any check constraints on this table, unless you intend them to be applied equally to all child tables. There is no point in defining any indexes or unique constraints on it, either. For our example, the root table is the `measurement` table as originally defined:

```
CREATE TABLE measurement (  
    city_id      int not null,  
    logdate      date not null,  
    peaktemp     int,  
    unitsales    int  
);
```

2. Create several “child” tables that each inherit from the root table. Normally, these tables will not add any columns to the set inherited from the root. Just as with declarative partitioning, these tables are in every way normal PostgreSQL tables (or foreign tables).

```
CREATE TABLE measurement_y2006m02 () INHERITS (measurement);  
CREATE TABLE measurement_y2006m03 () INHERITS (measurement);  
...  
CREATE TABLE measurement_y2007m11 () INHERITS (measurement);  
CREATE TABLE measurement_y2007m12 () INHERITS (measurement);  
CREATE TABLE measurement_y2008m01 () INHERITS (measurement);
```

3. Add non-overlapping table constraints to the child tables to define the allowed key values in each.

Typical examples would be:

```
CHECK ( x = 1 )  
CHECK ( county IN ( 'Oxfordshire', 'Buckinghamshire',  
    'Warwickshire' ) )  
CHECK ( outletID >= 100 AND outletID < 200 )
```

Ensure that the constraints guarantee that there is no overlap between the key values permitted in different child tables. A common mistake is to set up range constraints like:

```
CHECK ( outletID BETWEEN 100 AND 200 )  
CHECK ( outletID BETWEEN 200 AND 300 )
```

This is wrong since it is not clear which child table the key value 200 belongs in. Instead, ranges should be defined in this style:

```
CREATE TABLE measurement_y2006m02 (  
    CHECK ( logdate >= DATE '2006-02-01' AND logdate < DATE  
    '2006-03-01' )  
) INHERITS (measurement);  
  
CREATE TABLE measurement_y2006m03 (  
    CHECK ( logdate >= DATE '2006-03-01' AND logdate < DATE  
    '2006-04-01' )  
) INHERITS (measurement);
```

```
        CHECK ( logdate >= DATE '2006-03-01' AND logdate < DATE
'2006-04-01' )
    ) INHERITS (measurement);
```

```
...
CREATE TABLE measurement_y2007m11 (
    CHECK ( logdate >= DATE '2007-11-01' AND logdate < DATE
'2007-12-01' )
    ) INHERITS (measurement);
```

```
CREATE TABLE measurement_y2007m12 (
    CHECK ( logdate >= DATE '2007-12-01' AND logdate < DATE
'2008-01-01' )
    ) INHERITS (measurement);
```

```
CREATE TABLE measurement_y2008m01 (
    CHECK ( logdate >= DATE '2008-01-01' AND logdate < DATE
'2008-02-01' )
    ) INHERITS (measurement);
```

4. For each child table, create an index on the key column(s), as well as any other indexes you might want.

```
CREATE INDEX measurement_y2006m02_logdate ON
measurement_y2006m02 (logdate);
CREATE INDEX measurement_y2006m03_logdate ON
measurement_y2006m03 (logdate);
CREATE INDEX measurement_y2007m11_logdate ON
measurement_y2007m11 (logdate);
CREATE INDEX measurement_y2007m12_logdate ON
measurement_y2007m12 (logdate);
CREATE INDEX measurement_y2008m01_logdate ON
measurement_y2008m01 (logdate);
```

5. We want our application to be able to say `INSERT INTO measurement ...` and have the data be redirected into the appropriate child table. We can arrange that by attaching a suitable trigger function to the root table. If data will be added only to the latest child, we can use a very simple trigger function:

```
CREATE OR REPLACE FUNCTION measurement_insert_trigger()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO measurement_y2008m01 VALUES (NEW.*);
    RETURN NULL;
END;
$$
LANGUAGE plpgsql;
```

After creating the function, we create a trigger which calls the trigger function:

```
CREATE TRIGGER insert_measurement_trigger
    BEFORE INSERT ON measurement
    FOR EACH ROW EXECUTE FUNCTION measurement_insert_trigger();
```

We must redefine the trigger function each month so that it always inserts into the current child table. The trigger definition does not need to be updated, however.

We might want to insert data and have the server automatically locate the child table into which the row should be added. We could do this with a more complex trigger function, for example:

```
CREATE OR REPLACE FUNCTION measurement_insert_trigger()
RETURNS TRIGGER AS $$
BEGIN
    IF ( NEW.logdate >= DATE '2006-02-01' AND
        NEW.logdate < DATE '2006-03-01' ) THEN
        INSERT INTO measurement_y2006m02 VALUES (NEW.*);
    ELSIF ( NEW.logdate >= DATE '2006-03-01' AND
        NEW.logdate < DATE '2006-04-01' ) THEN
        INSERT INTO measurement_y2006m03 VALUES (NEW.*);
    ...
    ELSIF ( NEW.logdate >= DATE '2008-01-01' AND
        NEW.logdate < DATE '2008-02-01' ) THEN
        INSERT INTO measurement_y2008m01 VALUES (NEW.*);
    ELSE
        RAISE EXCEPTION 'Date out of range. Fix the
measurement_insert_trigger() function!';
    END IF;
    RETURN NULL;
END;
$$
LANGUAGE plpgsql;
```

The trigger definition is the same as before. Note that each IF test must exactly match the CHECK constraint for its child table.

While this function is more complex than the single-month case, it doesn't need to be updated as often, since branches can be added in advance of being needed.

Note

In practice, it might be best to check the newest child first, if most inserts go into that child. For simplicity, we have shown the trigger's tests in the same order as in other parts of this example.

A different approach to redirecting inserts into the appropriate child table is to set up rules, instead of a trigger, on the root table. For example:

```
CREATE RULE measurement_insert_y2006m02 AS
ON INSERT TO measurement WHERE
    ( logdate >= DATE '2006-02-01' AND logdate < DATE
    '2006-03-01' )
DO INSTEAD
    INSERT INTO measurement_y2006m02 VALUES (NEW.*);
...
CREATE RULE measurement_insert_y2008m01 AS
ON INSERT TO measurement WHERE
    ( logdate >= DATE '2008-01-01' AND logdate < DATE
    '2008-02-01' )
DO INSTEAD
    INSERT INTO measurement_y2008m01 VALUES (NEW.*);
```

A rule has significantly more overhead than a trigger, but the overhead is paid once per query rather than once per row, so this method might be advantageous for bulk-insert situations. In most cases, however, the trigger method will offer better performance.

Be aware that `COPY` ignores rules. If you want to use `COPY` to insert data, you'll need to copy into the correct child table rather than directly into the root. `COPY` does fire triggers, so you can use it normally if you use the trigger approach.

Another disadvantage of the rule approach is that there is no simple way to force an error if the set of rules doesn't cover the insertion date; the data will silently go into the root table instead.

6. Ensure that the `constraint_exclusion` configuration parameter is not disabled in `postgresql.conf`; otherwise child tables may be accessed unnecessarily.

As we can see, a complex table hierarchy could require a substantial amount of DDL. In the above example we would be creating a new child table each month, so it might be wise to write a script that generates the required DDL automatically.

5.12.3.2. Maintenance for Inheritance Partitioning

To remove old data quickly, simply drop the child table that is no longer necessary:

```
DROP TABLE measurement_y2006m02;
```

To remove the child table from the inheritance hierarchy table but retain access to it as a table in its own right:

```
ALTER TABLE measurement_y2006m02 NO INHERIT measurement;
```

To add a new child table to handle new data, create an empty child table just as the original children were created above:

```
CREATE TABLE measurement_y2008m02 (  
    CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE  
        '2008-03-01' )  
    ) INHERITS (measurement);
```

Alternatively, one may want to create and populate the new child table before adding it to the table hierarchy. This could allow data to be loaded, checked, and transformed before being made visible to queries on the parent table.

```
CREATE TABLE measurement_y2008m02  
    (LIKE measurement INCLUDING DEFAULTS INCLUDING CONSTRAINTS);  
ALTER TABLE measurement_y2008m02 ADD CONSTRAINT y2008m02  
    CHECK ( logdate >= DATE '2008-02-01' AND logdate < DATE  
        '2008-03-01' );  
\copy measurement_y2008m02 from 'measurement_y2008m02'  
-- possibly some other data preparation work  
ALTER TABLE measurement_y2008m02 INHERIT measurement;
```

5.12.3.3. Caveats

The following caveats apply to partitioning implemented using inheritance:

- There is no automatic way to verify that all of the `CHECK` constraints are mutually exclusive. It is safer to create code that generates child tables and creates and/or modifies associated objects than to write each by hand.

- Indexes and foreign key constraints apply to single tables and not to their inheritance children, hence they have some caveats to be aware of.
- The schemes shown here assume that the values of a row's key column(s) never change, or at least do not change enough to require it to move to another partition. An `UPDATE` that attempts to do that will fail because of the `CHECK` constraints. If you need to handle such cases, you can put suitable update triggers on the child tables, but it makes management of the structure much more complicated.
- Manual `VACUUM` and `ANALYZE` commands will automatically process all inheritance child tables. If this is undesirable, you can use the `ONLY` keyword. A command like:

```
ANALYZE ONLY measurement;
```

will only process the root table.

- `INSERT` statements with `ON CONFLICT` clauses are unlikely to work as expected, as the `ON CONFLICT` action is only taken in case of unique violations on the specified target relation, not its child relations.
- Triggers or rules will be needed to route rows to the desired child table, unless the application is explicitly aware of the partitioning scheme. Triggers may be complicated to write, and will be much slower than the tuple routing performed internally by declarative partitioning.

5.12.4. Partition Pruning

Partition pruning is a query optimization technique that improves performance for declaratively partitioned tables. As an example:

```
SET enable_partition_pruning = on;           -- the default
SELECT count(*) FROM measurement WHERE logdate >= DATE
'2008-01-01';
```

Without partition pruning, the above query would scan each of the partitions of the `measurement` table. With partition pruning enabled, the planner will examine the definition of each partition and prove that the partition need not be scanned because it could not contain any rows meeting the query's `WHERE` clause. When the planner can prove this, it excludes (*prunes*) the partition from the query plan.

By using the `EXPLAIN` command and the `enable_partition_pruning` configuration parameter, it's possible to show the difference between a plan for which partitions have been pruned and one for which they have not. A typical unoptimized plan for this type of table setup is:

```
SET enable_partition_pruning = off;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE
'2008-01-01';
```

QUERY PLAN

```
-----
Aggregate  (cost=188.76..188.77 rows=1 width=8)
-> Append  (cost=0.00..181.05 rows=3085 width=0)
      -> Seq Scan on measurement_y2006m02 (cost=0.00..33.12
rows=617 width=0)
          Filter: (logdate >= '2008-01-01'::date)
      -> Seq Scan on measurement_y2006m03 (cost=0.00..33.12
rows=617 width=0)
          Filter: (logdate >= '2008-01-01'::date)
...
```

```
-> Seq Scan on measurement_y2007m11 (cost=0.00..33.12
rows=617 width=0)
    Filter: (logdate >= '2008-01-01'::date)
-> Seq Scan on measurement_y2007m12 (cost=0.00..33.12
rows=617 width=0)
    Filter: (logdate >= '2008-01-01'::date)
-> Seq Scan on measurement_y2008m01 (cost=0.00..33.12
rows=617 width=0)
    Filter: (logdate >= '2008-01-01'::date)
```

Some or all of the partitions might use index scans instead of full-table sequential scans, but the point here is that there is no need to scan the older partitions at all to answer this query. When we enable partition pruning, we get a significantly cheaper plan that will deliver the same answer:

```
SET enable_partition_pruning = on;
EXPLAIN SELECT count(*) FROM measurement WHERE logdate >= DATE
'2008-01-01';

                                QUERY PLAN
-----
Aggregate (cost=37.75..37.76 rows=1 width=8)
-> Seq Scan on measurement_y2008m01 (cost=0.00..33.12 rows=617
width=0)
    Filter: (logdate >= '2008-01-01'::date)
```

Note that partition pruning is driven only by the constraints defined implicitly by the partition keys, not by the presence of indexes. Therefore it isn't necessary to define indexes on the key columns. Whether an index needs to be created for a given partition depends on whether you expect that queries that scan the partition will generally scan a large part of the partition or just a small part. An index will be helpful in the latter case but not the former.

Partition pruning can be performed not only during the planning of a given query, but also during its execution. This is useful as it can allow more partitions to be pruned when clauses contain expressions whose values are not known at query planning time, for example, parameters defined in a **PREPARE** statement, using a value obtained from a subquery, or using a parameterized value on the inner side of a nested loop join. Partition pruning during execution can be performed at any of the following times:

- During initialization of the query plan. Partition pruning can be performed here for parameter values which are known during the initialization phase of execution. Partitions which are pruned during this stage will not show up in the query's **EXPLAIN** or **EXPLAIN ANALYZE**. It is possible to determine the number of partitions which were removed during this phase by observing the “Subplans Removed” property in the **EXPLAIN** output. It's important to note that any partitions removed by the partition pruning done at this stage are still locked at the beginning of execution.
- During actual execution of the query plan. Partition pruning may also be performed here to remove partitions using values which are only known during actual query execution. This includes values from subqueries and values from execution-time parameters such as those from parameterized nested loop joins. Since the value of these parameters may change many times during the execution of the query, partition pruning is performed whenever one of the execution parameters being used by partition pruning changes. Determining if partitions were pruned during this phase requires careful inspection of the **loops** property in the **EXPLAIN ANALYZE** output. Subplans corresponding to different partitions may have different values for it depending on how many times each of them was pruned during execution. Some may be shown as (**never executed**) if they were pruned every time.

Partition pruning can be disabled using the `enable_partition_pruning` setting.

5.12.5. Partitioning and Constraint Exclusion

Constraint exclusion is a query optimization technique similar to partition pruning. While it is primarily used for partitioning implemented using the legacy inheritance method, it can be used for other purposes, including with declarative partitioning.

Constraint exclusion works in a very similar way to partition pruning, except that it uses each table's **CHECK** constraints — which gives it its name — whereas partition pruning uses the table's partition bounds, which exist only in the case of declarative partitioning. Another difference is that constraint exclusion is only applied at plan time; there is no attempt to remove partitions at execution time.

The fact that constraint exclusion uses **CHECK** constraints, which makes it slow compared to partition pruning, can sometimes be used as an advantage: because constraints can be defined even on declaratively-partitioned tables, in addition to their internal partition bounds, constraint exclusion may be able to elide additional partitions from the query plan.

The default (and recommended) setting of `constraint_exclusion` is neither `on` nor `off`, but an intermediate setting called `partition`, which causes the technique to be applied only to queries that are likely to be working on inheritance partitioned tables. The `on` setting causes the planner to examine **CHECK** constraints in all queries, even simple ones that are unlikely to benefit.

The following caveats apply to constraint exclusion:

- Constraint exclusion is only applied during query planning, unlike partition pruning, which can also be applied during query execution.
- Constraint exclusion only works when the query's **WHERE** clause contains constants (or externally supplied parameters). For example, a comparison against a non-immutable function such as `CURRENT_TIMESTAMP` cannot be optimized, since the planner cannot know which child table the function's value might fall into at run time.
- Keep the partitioning constraints simple, else the planner may not be able to prove that child tables might not need to be visited. Use simple equality conditions for list partitioning, or simple range tests for range partitioning, as illustrated in the preceding examples. A good rule of thumb is that partitioning constraints should contain only comparisons of the partitioning column(s) to constants using B-tree-indexable operators, because only B-tree-indexable column(s) are allowed in the partition key.
- All constraints on all children of the parent table are examined during constraint exclusion, so large numbers of children are likely to increase query planning time considerably. So the legacy inheritance based partitioning will work well with up to perhaps a hundred child tables; don't try to use many thousands of children.

5.12.6. Best Practices for Declarative Partitioning

The choice of how to partition a table should be made carefully, as the performance of query planning and execution can be negatively affected by poor design.

One of the most critical design decisions will be the column or columns by which you partition your data. Often the best choice will be to partition by the column or set of columns which most commonly appear in **WHERE** clauses of queries being executed on the partitioned table. **WHERE** clauses that are compatible with the partition bound constraints can be used to prune unneeded partitions. However, you may be forced into making other decisions by requirements for the **PRIMARY KEY** or a **UNIQUE** constraint. Removal of unwanted data is also a factor to consider when planning your partitioning strategy. An entire partition can be detached fairly quickly, so it may be beneficial to design the partition strategy in such a way that all data to be removed at once is located in a single partition.

Choosing the target number of partitions that the table should be divided into is also a critical decision to make. Not having enough partitions may mean that indexes remain too large and that data locality remains poor which could result in low cache hit ratios. However, dividing the table into too many partitions can also cause issues. Too many partitions can mean longer query planning times and higher memory consumption during both query planning and execution, as further described below. When

choosing how to partition your table, it's also important to consider what changes may occur in the future. For example, if you choose to have one partition per customer and you currently have a small number of large customers, consider the implications if in several years you instead find yourself with a large number of small customers. In this case, it may be better to choose to partition by **HASH** and choose a reasonable number of partitions rather than trying to partition by **LIST** and hoping that the number of customers does not increase beyond what it is practical to partition the data by.

Sub-partitioning can be useful to further divide partitions that are expected to become larger than other partitions. Another option is to use range partitioning with multiple columns in the partition key. Either of these can easily lead to excessive numbers of partitions, so restraint is advisable.

It is important to consider the overhead of partitioning during query planning and execution. The query planner is generally able to handle partition hierarchies with up to a few thousand partitions fairly well, provided that typical queries allow the query planner to prune all but a small number of partitions. Planning times become longer and memory consumption becomes higher when more partitions remain after the planner performs partition pruning. Another reason to be concerned about having a large number of partitions is that the server's memory consumption may grow significantly over time, especially if many sessions touch large numbers of partitions. That's because each partition requires its metadata to be loaded into the local memory of each session that touches it.

With data warehouse type workloads, it can make sense to use a larger number of partitions than with an OLTP type workload. Generally, in data warehouses, query planning time is less of a concern as the majority of processing time is spent during query execution. With either of these two types of workload, it is important to make the right decisions early, as re-partitioning large quantities of data can be painfully slow. Simulations of the intended workload are often beneficial for optimizing the partitioning strategy. Never just assume that more partitions are better than fewer partitions, nor vice-versa.

5.13. Foreign Data

PostgreSQL implements portions of the SQL/MED specification, allowing you to access data that resides outside PostgreSQL using regular SQL queries. Such data is referred to as *foreign data*. (Note that this usage is not to be confused with foreign keys, which are a type of constraint within the database.)

Foreign data is accessed with help from a *foreign data wrapper*. A foreign data wrapper is a library that can communicate with an external data source, hiding the details of connecting to the data source and obtaining data from it. There are some foreign data wrappers available as `contrib` modules; see Appendix F. Other kinds of foreign data wrappers might be found as third party products. If none of the existing foreign data wrappers suit your needs, you can write your own; see Chapter 58.

To access foreign data, you need to create a *foreign server* object, which defines how to connect to a particular external data source according to the set of options used by its supporting foreign data wrapper. Then you need to create one or more *foreign tables*, which define the structure of the remote data. A foreign table can be used in queries just like a normal table, but a foreign table has no storage in the PostgreSQL server. Whenever it is used, PostgreSQL asks the foreign data wrapper to fetch data from the external source, or transmit data to the external source in the case of update commands.

Accessing remote data may require authenticating to the external data source. This information can be provided by a *user mapping*, which can provide additional data such as user names and passwords based on the current PostgreSQL role.

For additional information, see `CREATE FOREIGN DATA WRAPPER`, `CREATE SERVER`, `CREATE USER MAPPING`, `CREATE FOREIGN TABLE`, and `IMPORT FOREIGN SCHEMA`.

5.14. Other Database Objects

Tables are the central objects in a relational database structure, because they hold your data. But they are not the only objects that exist in a database. Many other kinds of objects can be created to make the

use and management of the data more efficient or convenient. They are not discussed in this chapter, but we give you a list here so that you are aware of what is possible:

- Views
- Functions, procedures, and operators
- Data types and domains
- Triggers and rewrite rules

Detailed information on these topics appears in Part V.

5.15. Dependency Tracking

When you create complex database structures involving many tables with foreign key constraints, views, triggers, functions, etc. you implicitly create a net of dependencies between the objects. For instance, a table with a foreign key constraint depends on the table it references.

To ensure the integrity of the entire database structure, PostgreSQL makes sure that you cannot drop objects that other objects still depend on. For example, attempting to drop the `products` table we considered in Section 5.5.5, with the `orders` table depending on it, would result in an error message like this:

```
DROP TABLE products;
```

```
ERROR:  cannot drop table products because other objects depend on
it
DETAIL:  constraint orders_product_no_fkey on table orders depends
on table products
HINT:   Use DROP ... CASCADE to drop the dependent objects too.
```

The error message contains a useful hint: if you do not want to bother deleting all the dependent objects individually, you can run:

```
DROP TABLE products CASCADE;
```

and all the dependent objects will be removed, as will any objects that depend on them, recursively. In this case, it doesn't remove the `orders` table, it only removes the foreign key constraint. It stops there because nothing depends on the foreign key constraint. (If you want to check what `DROP ... CASCADE` will do, run `DROP` without `CASCADE` and read the `DETAIL` output.)

Almost all `DROP` commands in PostgreSQL support specifying `CASCADE`. Of course, the nature of the possible dependencies varies with the type of the object. You can also write `RESTRICT` instead of `CASCADE` to get the default behavior, which is to prevent dropping objects that any other objects depend on.

Note

According to the SQL standard, specifying either `RESTRICT` or `CASCADE` is required in a `DROP` command. No database system actually enforces that rule, but whether the default behavior is `RESTRICT` or `CASCADE` varies across systems.

If a `DROP` command lists multiple objects, `CASCADE` is only required when there are dependencies outside the specified group. For example, when saying `DROP TABLE tab1, tab2` the existence of a foreign key referencing `tab1` from `tab2` would not mean that `CASCADE` is needed to succeed.

For a user-defined function or procedure whose body is defined as a string literal, PostgreSQL tracks dependencies associated with the function's externally-visible properties, such as its argument and result types, but *not* dependencies that could only be known by examining the function body. As an example, consider this situation:

```
CREATE TYPE rainbow AS ENUM ('red', 'orange', 'yellow',
                             'green', 'blue', 'purple');

CREATE TABLE my_colors (color rainbow, note text);

CREATE FUNCTION get_color_note (rainbow) RETURNS text AS
    'SELECT note FROM my_colors WHERE color = $1'
    LANGUAGE SQL;
```

(See Section 36.5 for an explanation of SQL-language functions.) PostgreSQL will be aware that the `get_color_note` function depends on the `rainbow` type: dropping the type would force dropping the function, because its argument type would no longer be defined. But PostgreSQL will not consider `get_color_note` to depend on the `my_colors` table, and so will not drop the function if the table is dropped. While there are disadvantages to this approach, there are also benefits. The function is still valid in some sense if the table is missing, though executing it would cause an error; creating a new table of the same name would allow the function to work again.

On the other hand, for an SQL-language function or procedure whose body is written in SQL-standard style, the body is parsed at function definition time and all dependencies recognized by the parser are stored. Thus, if we write the function above as

```
CREATE FUNCTION get_color_note (rainbow) RETURNS text
BEGIN ATOMIC
    SELECT note FROM my_colors WHERE color = $1;
END;
```

then the function's dependency on the `my_colors` table will be known and enforced by `DROP`.

Chapter 6. Data Manipulation

The previous chapter discussed how to create tables and other structures to hold your data. Now it is time to fill the tables with data. This chapter covers how to insert, update, and delete table data. The chapter after this will finally explain how to extract your long-lost data from the database.

6.1. Inserting Data

When a table is created, it contains no data. The first thing to do before a database can be of much use is to insert data. Data is inserted one row at a time. You can also insert more than one row in a single command, but it is not possible to insert something that is not a complete row. Even if you know only some column values, a complete row must be created.

To create a new row, use the INSERT command. The command requires the table name and column values. For example, consider the products table from Chapter 5:

```
CREATE TABLE products (  
    product_no integer,  
    name text,  
    price numeric  
);
```

An example command to insert a row would be:

```
INSERT INTO products VALUES (1, 'Cheese', 9.99);
```

The data values are listed in the order in which the columns appear in the table, separated by commas. Usually, the data values will be literals (constants), but scalar expressions are also allowed.

The above syntax has the drawback that you need to know the order of the columns in the table. To avoid this you can also list the columns explicitly. For example, both of the following commands have the same effect as the one above:

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese',  
9.99);  
INSERT INTO products (name, price, product_no) VALUES ('Cheese',  
9.99, 1);
```

Many users consider it good practice to always list the column names.

If you don't have values for all the columns, you can omit some of them. In that case, the columns will be filled with their default values. For example:

```
INSERT INTO products (product_no, name) VALUES (1, 'Cheese');  
INSERT INTO products VALUES (1, 'Cheese');
```

The second form is a PostgreSQL extension. It fills the columns from the left with as many values as are given, and the rest will be defaulted.

For clarity, you can also request default values explicitly, for individual columns or for the entire row:

```
INSERT INTO products (product_no, name, price) VALUES (1, 'Cheese',  
DEFAULT);
```

```
INSERT INTO products DEFAULT VALUES;
```

You can insert multiple rows in a single command:

```
INSERT INTO products (product_no, name, price) VALUES
  (1, 'Cheese', 9.99),
  (2, 'Bread', 1.99),
  (3, 'Milk', 2.99);
```

It is also possible to insert the result of a query (which might be no rows, one row, or many rows):

```
INSERT INTO products (product_no, name, price)
  SELECT product_no, name, price FROM new_products
  WHERE release_date = 'today';
```

This provides the full power of the SQL query mechanism (Chapter 7) for computing the rows to be inserted.

Tip

When inserting a lot of data at the same time, consider using the COPY command. It is not as flexible as the INSERT command, but is more efficient. Refer to Section 14.4 for more information on improving bulk loading performance.

6.2. Updating Data

The modification of data that is already in the database is referred to as updating. You can update individual rows, all the rows in a table, or a subset of all rows. Each column can be updated separately; the other columns are not affected.

To update existing rows, use the UPDATE command. This requires three pieces of information:

1. The name of the table and column to update
2. The new value of the column
3. Which row(s) to update

Recall from Chapter 5 that SQL does not, in general, provide a unique identifier for rows. Therefore it is not always possible to directly specify which row to update. Instead, you specify which conditions a row must meet in order to be updated. Only if you have a primary key in the table (independent of whether you declared it or not) can you reliably address individual rows by choosing a condition that matches the primary key. Graphical database access tools rely on this fact to allow you to update rows individually.

For example, this command updates all products that have a price of 5 to have a price of 10:

```
UPDATE products SET price = 10 WHERE price = 5;
```

This might cause zero, one, or many rows to be updated. It is not an error to attempt an update that does not match any rows.

Let's look at that command in detail. First is the key word UPDATE followed by the table name. As usual, the table name can be schema-qualified, otherwise it is looked up in the path. Next is the key word SET followed by the column name, an equal sign, and the new column value. The new column value can be any scalar expression, not just a constant. For example, if you want to raise the price of all products by 10% you could use:

```
UPDATE products SET price = price * 1.10;
```

As you see, the expression for the new value can refer to the existing value(s) in the row. We also left out the `WHERE` clause. If it is omitted, it means that all rows in the table are updated. If it is present, only those rows that match the `WHERE` condition are updated. Note that the equals sign in the `SET` clause is an assignment while the one in the `WHERE` clause is a comparison, but this does not create any ambiguity. Of course, the `WHERE` condition does not have to be an equality test. Many other operators are available (see Chapter 9). But the expression needs to evaluate to a Boolean result.

You can update more than one column in an `UPDATE` command by listing more than one assignment in the `SET` clause. For example:

```
UPDATE mytable SET a = 5, b = 3, c = 1 WHERE a > 0;
```

6.3. Deleting Data

So far we have explained how to add data to tables and how to change data. What remains is to discuss how to remove data that is no longer needed. Just as adding data is only possible in whole rows, you can only remove entire rows from a table. In the previous section we explained that SQL does not provide a way to directly address individual rows. Therefore, removing rows can only be done by specifying conditions that the rows to be removed have to match. If you have a primary key in the table then you can specify the exact row. But you can also remove groups of rows matching a condition, or you can remove all rows in the table at once.

You use the `DELETE` command to remove rows; the syntax is very similar to the `UPDATE` command. For instance, to remove all rows from the `products` table that have a price of 10, use:

```
DELETE FROM products WHERE price = 10;
```

If you simply write:

```
DELETE FROM products;
```

then all rows in the table will be deleted! Caveat programmer.

6.4. Returning Data from Modified Rows

Sometimes it is useful to obtain data from modified rows while they are being manipulated. The `INSERT`, `UPDATE`, `DELETE`, and `MERGE` commands all have an optional `RETURNING` clause that supports this. Use of `RETURNING` avoids performing an extra database query to collect the data, and is especially valuable when it would otherwise be difficult to identify the modified rows reliably.

The allowed contents of a `RETURNING` clause are the same as a `SELECT` command's output list (see Section 7.3). It can contain column names of the command's target table, or value expressions using those columns. A common shorthand is `RETURNING *`, which selects all columns of the target table in order.

In an `INSERT`, the default data available to `RETURNING` is the row as it was inserted. This is not so useful in trivial inserts, since it would just repeat the data provided by the client. But it can be very handy when relying on computed default values. For example, when using a `serial` column to provide unique identifiers, `RETURNING` can return the ID assigned to a new row:

```
CREATE TABLE users (firstname text, lastname text, id serial  
primary key);
```

```
INSERT INTO users (firstname, lastname) VALUES ('Joe', 'Cool')
RETURNING id;
```

The RETURNING clause is also very useful with INSERT ... SELECT.

In an UPDATE, the default data available to RETURNING is the new content of the modified row. For example:

```
UPDATE products SET price = price * 1.10
WHERE price <= 99.99
RETURNING name, price AS new_price;
```

In a DELETE, the default data available to RETURNING is the content of the deleted row. For example:

```
DELETE FROM products
WHERE obsolescence_date = 'today'
RETURNING *;
```

In a MERGE, the default data available to RETURNING is the content of the source row plus the content of the inserted, updated, or deleted target row. Since it is quite common for the source and target to have many of the same columns, specifying RETURNING * can lead to a lot of duplicated columns, so it is often more useful to qualify it so as to return just the source or target row. For example:

```
MERGE INTO products p USING new_products n ON p.product_no =
n.product_no
WHEN NOT MATCHED THEN INSERT VALUES (n.product_no, n.name,
n.price)
WHEN MATCHED THEN UPDATE SET name = n.name, price = n.price
RETURNING p.*;
```

In each of these commands, it is also possible to explicitly return the old and new content of the modified row. For example:

```
UPDATE products SET price = price * 1.10
WHERE price <= 99.99
RETURNING name, old.price AS old_price, new.price AS new_price,
new.price - old.price AS price_change;
```

In this example, writing new.price is the same as just writing price, but it makes the meaning clearer.

This syntax for returning old and new values is available in INSERT, UPDATE, DELETE, and MERGE commands, but typically old values will be NULL for an INSERT, and new values will be NULL for a DELETE. However, there are situations where it can still be useful for those commands. For example, in an INSERT with an ON CONFLICT DO UPDATE clause, the old values will be non-NULL for conflicting rows. Similarly, if a DELETE is turned into an UPDATE by a rewrite rule, the new values may be non-NULL.

If there are triggers (Chapter 37) on the target table, the data available to RETURNING is the row as modified by the triggers. Thus, inspecting columns computed by triggers is another common use-case for RETURNING.

Chapter 7. Queries

The previous chapters explained how to create tables, how to fill them with data, and how to manipulate that data. Now we finally discuss how to retrieve the data from the database.

7.1. Overview

The process of retrieving or the command to retrieve data from a database is called a *query*. In SQL the `SELECT` command is used to specify queries. The general syntax of the `SELECT` command is

```
[WITH with_queries] SELECT select_list FROM table_expression
[sort_specification]
```

The following sections describe the details of the select list, the table expression, and the sort specification. `WITH` queries are treated last since they are an advanced feature.

A simple kind of query has the form:

```
SELECT * FROM table1;
```

Assuming that there is a table called `table1`, this command would retrieve all rows and all user-defined columns from `table1`. (The method of retrieval depends on the client application. For example, the `psql` program will display an ASCII-art table on the screen, while client libraries will offer functions to extract individual values from the query result.) The select list specification `*` means all columns that the table expression happens to provide. A select list can also select a subset of the available columns or make calculations using the columns. For example, if `table1` has columns named `a`, `b`, and `c` (and perhaps others) you can make the following query:

```
SELECT a, b + c FROM table1;
```

(assuming that `b` and `c` are of a numerical data type). See Section 7.3 for more details.

`FROM table1` is a simple kind of table expression: it reads just one table. In general, table expressions can be complex constructs of base tables, joins, and subqueries. But you can also omit the table expression entirely and use the `SELECT` command as a calculator:

```
SELECT 3 * 4;
```

This is more useful if the expressions in the select list return varying results. For example, you could call a function this way:

```
SELECT random();
```

7.2. Table Expressions

A *table expression* computes a table. The table expression contains a `FROM` clause that is optionally followed by `WHERE`, `GROUP BY`, and `HAVING` clauses. Trivial table expressions simply refer to a table on disk, a so-called base table, but more complex expressions can be used to modify or combine base tables in various ways.

The optional `WHERE`, `GROUP BY`, and `HAVING` clauses in the table expression specify a pipeline of successive transformations performed on the table derived in the `FROM` clause. All these

transformations produce a virtual table that provides the rows that are passed to the select list to compute the output rows of the query.

7.2.1. The FROM Clause

The FROM clause derives a table from one or more other tables given in a comma-separated table reference list.

```
FROM table_reference [, table_reference [, ...]]
```

A table reference can be a table name (possibly schema-qualified), or a derived table such as a subquery, a JOIN construct, or complex combinations of these. If more than one table reference is listed in the FROM clause, the tables are cross-joined (that is, the Cartesian product of their rows is formed; see below). The result of the FROM list is an intermediate virtual table that can then be subject to transformations by the WHERE, GROUP BY, and HAVING clauses and is finally the result of the overall table expression.

When a table reference names a table that is the parent of a table inheritance hierarchy, the table reference produces rows of not only that table but all of its descendant tables, unless the key word ONLY precedes the table name. However, the reference produces only the columns that appear in the named table — any columns added in subtables are ignored.

Instead of writing ONLY before the table name, you can write * after the table name to explicitly specify that descendant tables are included. There is no real reason to use this syntax any more, because searching descendant tables is now always the default behavior. However, it is supported for compatibility with older releases.

7.2.1.1. Joined Tables

A joined table is a table derived from two other (real or derived) tables according to the rules of the particular join type. Inner, outer, and cross-joins are available. The general syntax of a joined table is

```
T1 join_type T2 [ join_condition ]
```

Joins of all types can be chained together, or nested: either or both *T1* and *T2* can be joined tables. Parentheses can be used around JOIN clauses to control the join order. In the absence of parentheses, JOIN clauses nest left-to-right.

Join Types

Cross join

```
T1 CROSS JOIN T2
```

For every possible combination of rows from *T1* and *T2* (i.e., a Cartesian product), the joined table will contain a row consisting of all columns in *T1* followed by all columns in *T2*. If the tables have N and M rows respectively, the joined table will have N * M rows.

FROM *T1* CROSS JOIN *T2* is equivalent to FROM *T1* INNER JOIN *T2* ON TRUE (see below). It is also equivalent to FROM *T1*, *T2*.

Note

This latter equivalence does not hold exactly when more than two tables appear, because JOIN binds more tightly than comma. For example FROM *T1* CROSS JOIN *T2* INNER JOIN *T3* ON *condition* is not the same as FROM *T1*, *T2* INNER JOIN

T3 ON *condition* because the *condition* can reference *T1* in the first case but not the second.

Qualified joins

```

T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2
  ON boolean_expression
T1 { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2 USING
  ( join column list )
T1 NATURAL { [INNER] | { LEFT | RIGHT | FULL } [OUTER] } JOIN T2

```

The words **INNER** and **OUTER** are optional in all forms. **INNER** is the default; **LEFT**, **RIGHT**, and **FULL** imply an outer join.

The *join condition* is specified in the **ON** or **USING** clause, or implicitly by the word **NATURAL**. The join condition determines which rows from the two source tables are considered to “match”, as explained in detail below.

The possible types of qualified join are:

INNER JOIN

For each row *R1* of *T1*, the joined table has a row for each row in *T2* that satisfies the join condition with *R1*.

LEFT OUTER JOIN

First, an inner join is performed. Then, for each row in *T1* that does not satisfy the join condition with any row in *T2*, a joined row is added with null values in columns of *T2*. Thus, the joined table always has at least one row for each row in *T1*.

RIGHT OUTER JOIN

First, an inner join is performed. Then, for each row in *T2* that does not satisfy the join condition with any row in *T1*, a joined row is added with null values in columns of *T1*. This is the converse of a left join: the result table will always have a row for each row in *T2*.

FULL OUTER JOIN

First, an inner join is performed. Then, for each row in *T1* that does not satisfy the join condition with any row in *T2*, a joined row is added with null values in columns of *T2*. Also, for each row of *T2* that does not satisfy the join condition with any row in *T1*, a joined row with null values in the columns of *T1* is added.

The **ON** clause is the most general kind of join condition: it takes a Boolean value expression of the same kind as is used in a **WHERE** clause. A pair of rows from *T1* and *T2* match if the **ON** expression evaluates to true.

The **USING** clause is a shorthand that allows you to take advantage of the specific situation where both sides of the join use the same name for the joining column(s). It takes a comma-separated list of the shared column names and forms a join condition that includes an equality comparison for each one. For example, joining *T1* and *T2* with **USING (a, b)** produces the join condition **ON T1.a = T2.a AND T1.b = T2.b**.

Furthermore, the output of **JOIN USING** suppresses redundant columns: there is no need to print both of the matched columns, since they must have equal values. While **JOIN ON** produces all columns from *T1* followed by all columns from *T2*, **JOIN USING** produces one output column for each of the listed column pairs (in the listed order), followed by any remaining columns from *T1*, followed by any remaining columns from *T2*.

Finally, **NATURAL** is a shorthand form of **USING**: it forms a **USING** list consisting of all column names that appear in both input tables. As with **USING**, these columns appear only once in the output table. If there are no common column names, **NATURAL JOIN** behaves like **CROSS JOIN**.

Note

USING is reasonably safe from column changes in the joined relations since only the listed columns are combined. **NATURAL** is considerably more risky since any schema changes to either relation that cause a new matching column name to be present will cause the join to combine that new column as well.

To put this together, assume we have tables **t1**:

num	name
1	a
2	b
3	c

and **t2**:

num	value
1	xxx
3	yyy
5	zzz

then we get the following results for the various joins:

=> **SELECT * FROM t1 CROSS JOIN t2;**

num	name	num	value
1	a	1	xxx
1	a	3	yyy
1	a	5	zzz
2	b	1	xxx
2	b	3	yyy
2	b	5	zzz
3	c	1	xxx
3	c	3	yyy
3	c	5	zzz

(9 rows)

=> **SELECT * FROM t1 INNER JOIN t2 ON t1.num = t2.num;**

num	name	num	value
1	a	1	xxx
3	c	3	yyy

(2 rows)

=> **SELECT * FROM t1 INNER JOIN t2 USING (num);**

num	name	value
1	a	xxx

```

    3 | c      | yyy
(2 rows)

```

=> **SELECT * FROM t1 NATURAL INNER JOIN t2;**

```

num | name | value
-----+-----+-----
    1 | a     | xxx
    3 | c     | yyy
(2 rows)

```

=> **SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num;**

```

num | name | num | value
-----+-----+-----+-----
    1 | a     |    1 | xxx
    2 | b     |     | 
    3 | c     |    3 | yyy
(3 rows)

```

=> **SELECT * FROM t1 LEFT JOIN t2 USING (num);**

```

num | name | value
-----+-----+-----
    1 | a     | xxx
    2 | b     | 
    3 | c     | yyy
(3 rows)

```

=> **SELECT * FROM t1 RIGHT JOIN t2 ON t1.num = t2.num;**

```

num | name | num | value
-----+-----+-----+-----
    1 | a     |    1 | xxx
    3 | c     |    3 | yyy
     |      |    5 | zzz
(3 rows)

```

=> **SELECT * FROM t1 FULL JOIN t2 ON t1.num = t2.num;**

```

num | name | num | value
-----+-----+-----+-----
    1 | a     |    1 | xxx
    2 | b     |     | 
    3 | c     |    3 | yyy
     |      |    5 | zzz
(4 rows)

```

The join condition specified with **ON** can also contain conditions that do not relate directly to the join. This can prove useful for some queries but needs to be thought out carefully. For example:

=> **SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num AND t2.value = 'xxx';**

```

num | name | num | value
-----+-----+-----+-----
    1 | a     |    1 | xxx
    2 | b     |     | 
    3 | c     |     | 
(3 rows)

```

Notice that placing the restriction in the **WHERE** clause produces a different result:

```
=> SELECT * FROM t1 LEFT JOIN t2 ON t1.num = t2.num WHERE t2.value
    = 'xxx';
   num | name | num | value
-----+-----+-----+-----
    1 | a    |    1 | xxx
(1 row)
```

This is because a restriction placed in the `ON` clause is processed *before* the join, while a restriction placed in the `WHERE` clause is processed *after* the join. That does not matter with inner joins, but it matters a lot with outer joins.

7.2.1.2. Table and Column Aliases

A temporary name can be given to tables and complex table references to be used for references to the derived table in the rest of the query. This is called a *table alias*.

To create a table alias, write

```
FROM table_reference AS alias
```

or

```
FROM table_reference alias
```

The `AS` key word is optional noise. *alias* can be any identifier.

A typical application of table aliases is to assign short identifiers to long table names to keep the join clauses readable. For example:

```
SELECT * FROM some_very_long_table_name s JOIN
another_fairly_long_name a ON s.id = a.num;
```

The alias becomes the new name of the table reference so far as the current query is concerned — it is not allowed to refer to the table by the original name elsewhere in the query. Thus, this is not valid:

```
SELECT * FROM my_table AS m WHERE my_table.a > 5;    -- wrong
```

Table aliases are mainly for notational convenience, but it is necessary to use them when joining a table to itself, e.g.:

```
SELECT * FROM people AS mother JOIN people AS child ON mother.id =
child.mother_id;
```

Parentheses are used to resolve ambiguities. In the following example, the first statement assigns the alias `b` to the second instance of `my_table`, but the second statement assigns the alias to the result of the join:

```
SELECT * FROM my_table AS a CROSS JOIN my_table AS b ...
SELECT * FROM (my_table AS a CROSS JOIN my_table) AS b ...
```

Another form of table aliasing gives temporary names to the columns of the table, as well as the table itself:

```
FROM table_reference [AS] alias ( column1 [, column2 [, ...]] )
```

If fewer column aliases are specified than the actual table has columns, the remaining columns are not renamed. This syntax is especially useful for self-joins or subqueries.

When an alias is applied to the output of a `JOIN` clause, the alias hides the original name(s) within the `JOIN`. For example:

```
SELECT a.* FROM my_table AS a JOIN your_table AS b ON ...
```

is valid SQL, but:

```
SELECT a.* FROM (my_table AS a JOIN your_table AS b ON ...) AS c
```

is not valid; the table alias `a` is not visible outside the alias `c`.

7.2.1.3. Subqueries

Subqueries specifying a derived table must be enclosed in parentheses. They may be assigned a table alias name, and optionally column alias names (as in Section 7.2.1.2). For example:

```
FROM (SELECT * FROM table1) AS alias_name
```

This example is equivalent to `FROM table1 AS alias_name`. More interesting cases, which cannot be reduced to a plain join, arise when the subquery involves grouping or aggregation.

A subquery can also be a `VALUES` list:

```
FROM (VALUES ('anne', 'smith'), ('bob', 'jones'), ('joe', 'blow'))
      AS names(first, last)
```

Again, a table alias is optional. Assigning alias names to the columns of the `VALUES` list is optional, but is good practice. For more information see Section 7.7.

According to the SQL standard, a table alias name must be supplied for a subquery. PostgreSQL allows `AS` and the alias to be omitted, but writing one is good practice in SQL code that might be ported to another system.

7.2.1.4. Table Functions

Table functions are functions that produce a set of rows, made up of either base data types (scalar types) or composite data types (table rows). They are used like a table, view, or subquery in the `FROM` clause of a query. Columns returned by table functions can be included in `SELECT`, `JOIN`, or `WHERE` clauses in the same manner as columns of a table, view, or subquery.

Table functions may also be combined using the `ROWS FROM` syntax, with the results returned in parallel columns; the number of result rows in this case is that of the largest function result, with smaller results padded with null values to match.

```
function_call [WITH ORDINALITY] [[AS] table_alias [(column_alias
[, ... ])] ]
ROWS FROM( function_call [, ... ] ) [WITH ORDINALITY]
[[AS] table_alias [(column_alias [, ... ])] ]
```

If the `WITH ORDINALITY` clause is specified, an additional column of type `bigint` will be added to the function result columns. This column numbers the rows of the function result set, starting from 1. (This is a generalization of the SQL-standard syntax for `UNNEST ... WITH ORDINALITY`.)

By default, the ordinal column is called *ordinality*, but a different column name can be assigned to it using an AS clause.

The special table function *UNNEST* may be called with any number of array parameters, and it returns a corresponding number of columns, as if *UNNEST* (Section 9.19) had been called on each parameter separately and combined using the *ROWS FROM* construct.

```
UNNEST( array_expression [, ... ] ) [WITH ORDINALITY]
[[AS] table_alias [(column_alias [, ... ])]]
```

If no *table_alias* is specified, the function name is used as the table name; in the case of a *ROWS FROM()* construct, the first function's name is used.

If column aliases are not supplied, then for a function returning a base data type, the column name is also the same as the function name. For a function returning a composite type, the result columns get the names of the individual attributes of the type.

Some examples:

```
CREATE TABLE foo (fooid int, foosubid int, fooname text);
```

```
CREATE FUNCTION getfoo(int) RETURNS SETOF foo AS $$
    SELECT * FROM foo WHERE fooid = $1;
$$ LANGUAGE SQL;
```

```
SELECT * FROM getfoo(1) AS t1;
```

```
SELECT * FROM foo
    WHERE foosubid IN (
        SELECT foosubid
        FROM getfoo(foo.fooid) z
        WHERE z.fooid = foo.fooid
    );
```

```
CREATE VIEW vw_getfoo AS SELECT * FROM getfoo(1);
```

```
SELECT * FROM vw_getfoo;
```

In some cases it is useful to define table functions that can return different column sets depending on how they are invoked. To support this, the table function can be declared as returning the pseudo-type *record* with no OUT parameters. When such a function is used in a query, the expected row structure must be specified in the query itself, so that the system can know how to parse and plan the query. This syntax looks like:

```
function_call [AS] alias (column_definition [, ... ])
function_call AS [alias] (column_definition [, ... ])
ROWS FROM( ... function_call AS (column_definition [, ... ])
    [, ... ] )
```

When not using the *ROWS FROM()* syntax, the *column_definition* list replaces the column alias list that could otherwise be attached to the FROM item; the names in the column definitions serve as column aliases. When using the *ROWS FROM()* syntax, a *column_definition* list can be attached to each member function separately; or if there is only one member function and no WITH ORDINALITY clause, a *column_definition* list can be written in place of a column alias list following *ROWS FROM()*.

Consider this example:

```
SELECT *
FROM dblink('dbname=mydb', 'SELECT proname, prosrc FROM
pg_proc')
AS t1(proname name, prosrc text)
WHERE proname LIKE 'bytea%';
```

The `dblink` function (part of the `dblink` module) executes a remote query. It is declared to return `record` since it might be used for any kind of query. The actual column set must be specified in the calling query so that the parser knows, for example, what `*` should expand to.

This example uses `ROWS FROM`:

```
SELECT *
FROM ROWS FROM
(
    json_to_recordset(' [{"a":40,"b":"foo"},
{"a":"100","b":"bar"}]')
    AS (a INTEGER, b TEXT),
    generate_series(1, 3)
) AS x (p, q, s)
ORDER BY p;
```

p	q	s
40	foo	1
100	bar	2
		3

It joins two functions into a single `FROM` target. `json_to_recordset()` is instructed to return two columns, the first `integer` and the second `text`. The result of `generate_series()` is used directly. The `ORDER BY` clause sorts the column values as integers.

7.2.1.5. LATERAL Subqueries

Subqueries appearing in `FROM` can be preceded by the key word `LATERAL`. This allows them to reference columns provided by preceding `FROM` items. (Without `LATERAL`, each subquery is evaluated independently and so cannot cross-reference any other `FROM` item.)

Table functions appearing in `FROM` can also be preceded by the key word `LATERAL`, but for functions the key word is optional; the function's arguments can contain references to columns provided by preceding `FROM` items in any case.

A `LATERAL` item can appear at the top level in the `FROM` list, or within a `JOIN` tree. In the latter case it can also refer to any items that are on the left-hand side of a `JOIN` that it is on the right-hand side of.

When a `FROM` item contains `LATERAL` cross-references, evaluation proceeds as follows: for each row of the `FROM` item providing the cross-referenced column(s), or set of rows of multiple `FROM` items providing the columns, the `LATERAL` item is evaluated using that row or row set's values of the columns. The resulting row(s) are joined as usual with the rows they were computed from. This is repeated for each row or set of rows from the column source table(s).

A trivial example of `LATERAL` is

```
SELECT * FROM foo, LATERAL (SELECT * FROM bar WHERE bar.id =
foo.bar_id) ss;
```

This is not especially useful since it has exactly the same result as the more conventional

```
SELECT * FROM foo, bar WHERE bar.id = foo.bar_id;
```

`LATERAL` is primarily useful when the cross-referenced column is necessary for computing the row(s) to be joined. A common application is providing an argument value for a set-returning function. For example, supposing that `vertices(polygon)` returns the set of vertices of a polygon, we could identify close-together vertices of polygons stored in a table with:

```
SELECT p1.id, p2.id, v1, v2
FROM polygons p1, polygons p2,
     LATERAL vertices(p1.poly) v1,
     LATERAL vertices(p2.poly) v2
WHERE (v1 <-> v2) < 10 AND p1.id != p2.id;
```

This query could also be written

```
SELECT p1.id, p2.id, v1, v2
FROM polygons p1 CROSS JOIN LATERAL vertices(p1.poly) v1,
     polygons p2 CROSS JOIN LATERAL vertices(p2.poly) v2
WHERE (v1 <-> v2) < 10 AND p1.id != p2.id;
```

or in several other equivalent formulations. (As already mentioned, the `LATERAL` key word is unnecessary in this example, but we use it for clarity.)

It is often particularly handy to `LEFT JOIN` to a `LATERAL` subquery, so that source rows will appear in the result even if the `LATERAL` subquery produces no rows for them. For example, if `get_product_names()` returns the names of products made by a manufacturer, but some manufacturers in our table currently produce no products, we could find out which ones those are like this:

```
SELECT m.name
FROM manufacturers m LEFT JOIN LATERAL get_product_names(m.id)
     pname ON true
WHERE pname IS NULL;
```

7.2.2. The WHERE Clause

The syntax of the `WHERE` clause is

```
WHERE search_condition
```

where *search_condition* is any value expression (see Section 4.2) that returns a value of type `boolean`.

After the processing of the `FROM` clause is done, each row of the derived virtual table is checked against the search condition. If the result of the condition is true, the row is kept in the output table, otherwise (i.e., if the result is false or null) it is discarded. The search condition typically references at least one column of the table generated in the `FROM` clause; this is not required, but otherwise the `WHERE` clause will be fairly useless.

Note

The join condition of an inner join can be written either in the `WHERE` clause or in the `JOIN` clause. For example, these table expressions are equivalent:

```
FROM a, b WHERE a.id = b.id AND b.val > 5
```

and:

```
FROM a INNER JOIN b ON (a.id = b.id) WHERE b.val > 5
```

or perhaps even:

```
FROM a NATURAL JOIN b WHERE b.val > 5
```

Which one of these you use is mainly a matter of style. The **JOIN** syntax in the **FROM** clause is probably not as portable to other SQL database management systems, even though it is in the SQL standard. For outer joins there is no choice: they must be done in the **FROM** clause. The **ON** or **USING** clause of an outer join is *not* equivalent to a **WHERE** condition, because it results in the addition of rows (for unmatched input rows) as well as the removal of rows in the final result.

Here are some examples of **WHERE** clauses:

```
SELECT ... FROM fdt WHERE c1 > 5
```

```
SELECT ... FROM fdt WHERE c1 IN (1, 2, 3)
```

```
SELECT ... FROM fdt WHERE c1 IN (SELECT c1 FROM t2)
```

```
SELECT ... FROM fdt WHERE c1 IN (SELECT c3 FROM t2 WHERE c2 =  
    fdt.c1 + 10)
```

```
SELECT ... FROM fdt WHERE c1 BETWEEN (SELECT c3 FROM t2 WHERE c2 =  
    fdt.c1 + 10) AND 100
```

```
SELECT ... FROM fdt WHERE EXISTS (SELECT c1 FROM t2 WHERE c2 >  
    fdt.c1)
```

fdt is the table derived in the **FROM** clause. Rows that do not meet the search condition of the **WHERE** clause are eliminated from **fdt**. Notice the use of scalar subqueries as value expressions. Just like any other query, the subqueries can employ complex table expressions. Notice also how **fdt** is referenced in the subqueries. Qualifying **c1** as **fdt.c1** is only necessary if **c1** is also the name of a column in the derived input table of the subquery. But qualifying the column name adds clarity even when it is not needed. This example shows how the column naming scope of an outer query extends into its inner queries.

7.2.3. The **GROUP BY** and **HAVING** Clauses

After passing the **WHERE** filter, the derived input table might be subject to grouping, using the **GROUP BY** clause, and elimination of group rows using the **HAVING** clause.

```
SELECT select_list  
    FROM ...  
    [WHERE ...]  
    GROUP BY grouping_column_reference  
    [, grouping_column_reference]...
```

The **GROUP BY** clause is used to group together those rows in a table that have the same values in all the columns listed. The order in which the columns are listed does not matter. The effect is to combine each set of rows having common values into one group row that represents all rows in the group. This is done to eliminate redundancy in the output and/or compute aggregates that apply to these groups. For instance:

```
=> SELECT * FROM test1;
```

```
x | y
---+---
a | 3
c | 2
b | 5
a | 1
(4 rows)
```

```
=> SELECT x FROM test1 GROUP BY x;
```

```
x
---
a
b
c
(3 rows)
```

In the second query, we could not have written **SELECT * FROM test1 GROUP BY x**, because there is no single value for the column **y** that could be associated with each group. The grouped-by columns can be referenced in the select list since they have a single value in each group.

In general, if a table is grouped, columns that are not listed in **GROUP BY** cannot be referenced except in aggregate expressions. An example with aggregate expressions is:

```
=> SELECT x, sum(y) FROM test1 GROUP BY x;
```

```
x | sum
---+-----
a | 4
b | 5
c | 2
(3 rows)
```

Here **SUM** is an aggregate function that computes a single value over the entire group. More information about the available aggregate functions can be found in Section 9.21.

Tip

Grouping without aggregate expressions effectively calculates the set of distinct values in a column. This can also be achieved using the **DISTINCT** clause (see Section 7.3.3).

Here is another example: it calculates the total sales for each product (rather than the total sales of all products):

```
SELECT product_id, p.name, (sum(s.units) * p.price) AS sales
FROM products p LEFT JOIN sales s USING (product_id)
GROUP BY product_id, p.name, p.price;
```

In this example, the columns **product_id**, **p.name**, and **p.price** must be in the **GROUP BY** clause since they are referenced in the query select list (but see below). The column **s.units** does

not have to be in the **GROUP BY** list since it is only used in an aggregate expression (`sum( . . . )`), which represents the sales of a product. For each product, the query returns a summary row about all sales of the product.

If the products table is set up so that, say, `product_id` is the primary key, then it would be enough to group by `product_id` in the above example, since name and price would be *functionally dependent* on the product ID, and so there would be no ambiguity about which name and price value to return for each product ID group.

In strict SQL, **GROUP BY** can only group by columns of the source table but PostgreSQL extends this to also allow **GROUP BY** to group by columns in the select list. Grouping by value expressions instead of simple column names is also allowed.

If a table has been grouped using **GROUP BY**, but only certain groups are of interest, the **HAVING** clause can be used, much like a **WHERE** clause, to eliminate groups from the result. The syntax is:

```
SELECT select_list FROM ... [WHERE ...] GROUP BY ...
HAVING boolean_expression
```

Expressions in the **HAVING** clause can refer both to grouped expressions and to ungrouped expressions (which necessarily involve an aggregate function).

Example:

```
=> SELECT x, sum(y) FROM test1 GROUP BY x HAVING sum(y) > 3;
```

```
x | sum
---+-----
a |    4
b |    5
(2 rows)
```

```
=> SELECT x, sum(y) FROM test1 GROUP BY x HAVING x < 'c';
```

```
x | sum
---+-----
a |    4
b |    5
(2 rows)
```

Again, a more realistic example:

```
SELECT product_id, p.name, (sum(s.units) * (p.price - p.cost)) AS
profit
FROM products p LEFT JOIN sales s USING (product_id)
WHERE s.date > CURRENT_DATE - INTERVAL '4 weeks'
GROUP BY product_id, p.name, p.price, p.cost
HAVING sum(p.price * s.units) > 5000;
```

In the example above, the **WHERE** clause is selecting rows by a column that is not grouped (the expression is only true for sales during the last four weeks), while the **HAVING** clause restricts the output to groups with total gross sales over 5000. Note that the aggregate expressions do not necessarily need to be the same in all parts of the query.

If a query contains aggregate function calls, but no **GROUP BY** clause, grouping still occurs: the result is a single group row (or perhaps no rows at all, if the single row is then eliminated by **HAVING**). The same is true if it contains a **HAVING** clause, even without any aggregate function calls or **GROUP BY** clause.

7.2.4. GROUPING SETS, CUBE, and ROLLUP

More complex grouping operations than those described above are possible using the concept of *grouping sets*. The data selected by the **FROM** and **WHERE** clauses is grouped separately by each specified grouping set, aggregates computed for each group just as for simple **GROUP BY** clauses, and then the results returned. For example:

```
=> SELECT * FROM items_sold;
```

brand	size	sales
-----+-----+-----		
Foo	L	10
Foo	M	20
Bar	M	15
Bar	L	5

(4 rows)

```
=> SELECT brand, size, sum(sales) FROM items_sold GROUP BY GROUPING
SETS ((brand), (size), ());
```

brand	size	sum
-----+-----+-----		
Foo		30
Bar		20
	L	15
	M	35
		50

(5 rows)

Each sublist of **GROUPING SETS** may specify zero or more columns or expressions and is interpreted the same way as though it were directly in the **GROUP BY** clause. An empty grouping set means that all rows are aggregated down to a single group (which is output even if no input rows were present), as described above for the case of aggregate functions with no **GROUP BY** clause.

References to the grouping columns or expressions are replaced by null values in result rows for grouping sets in which those columns do not appear. To distinguish which grouping a particular output row resulted from, see Table 9.66.

A shorthand notation is provided for specifying two common types of grouping set. A clause of the form

```
ROLLUP ( e1, e2, e3, ... )
```

represents the given list of expressions and all prefixes of the list including the empty list; thus it is equivalent to

```
GROUPING SETS (
    ( e1, e2, e3, ... ),
    ...
    ( e1, e2 ),
    ( e1 ),
    ( )
)
```

This is commonly used for analysis over hierarchical data; e.g., total salary by department, division, and company-wide total.

A clause of the form

CUBE (e1, e2, ...)

represents the given list and all of its possible subsets (i.e., the power set). Thus

CUBE (a, b, c)

is equivalent to

```
GROUPING SETS (
  ( a, b, c ),
  ( a, b   ),
  ( a,    c ),
  ( a     ),
  (    b, c ),
  (    b   ),
  (     c ),
  (      )
)
```

The individual elements of a CUBE or ROLLUP clause may be either individual expressions, or sublists of elements in parentheses. In the latter case, the sublists are treated as single units for the purposes of generating the individual grouping sets. For example:

CUBE ((a, b), (c, d))

is equivalent to

```
GROUPING SETS (
  ( a, b, c, d ),
  ( a, b       ),
  (      c, d ),
  (           )
)
```

and

ROLLUP (a, (b, c), d)

is equivalent to

```
GROUPING SETS (
  ( a, b, c, d ),
  ( a, b, c     ),
  ( a          ),
  (           )
)
```

The CUBE and ROLLUP constructs can be used either directly in the GROUP BY clause, or nested inside a GROUPING SETS clause. If one GROUPING SETS clause is nested inside another, the effect is the same as if all the elements of the inner clause had been written directly in the outer clause.

If multiple grouping items are specified in a single GROUP BY clause, then the final list of grouping sets is the Cartesian product of the individual items. For example:

`GROUP BY a, CUBE (b, c), GROUPING SETS ((d), (e))`

is equivalent to

```
GROUP BY GROUPING SETS (
    (a, b, c, d), (a, b, c, e),
    (a, b, d),   (a, b, e),
    (a, c, d),   (a, c, e),
    (a, d),      (a, e)
)
```

When specifying multiple grouping items together, the final set of grouping sets might contain duplicates. For example:

`GROUP BY ROLLUP (a, b), ROLLUP (a, c)`

is equivalent to

```
GROUP BY GROUPING SETS (
    (a, b, c),
    (a, b),
    (a, b),
    (a, c),
    (a),
    (a),
    (a, c),
    (a),
    ()
)
```

If these duplicates are undesirable, they can be removed using the **DISTINCT** clause directly on the **GROUP BY**. Therefore:

`GROUP BY DISTINCT ROLLUP (a, b), ROLLUP (a, c)`

is equivalent to

```
GROUP BY GROUPING SETS (
    (a, b, c),
    (a, b),
    (a, c),
    (a),
    ()
)
```

This is not the same as using **SELECT DISTINCT** because the output rows may still contain duplicates. If any of the ungrouped columns contains **NULL**, it will be indistinguishable from the **NULL** used when that same column is grouped.

Note

The construct `(a, b)` is normally recognized in expressions as a row constructor. Within the **GROUP BY** clause, this does not apply at the top levels of expressions, and `(a, b)` is parsed

as a list of expressions as described above. If for some reason you *need* a row constructor in a grouping expression, use `ROW(a, b)`.

7.2.5. Window Function Processing

If the query contains any window functions (see Section 3.5, Section 9.22 and Section 4.2.8), these functions are evaluated after any grouping, aggregation, and `HAVING` filtering is performed. That is, if the query uses any aggregates, `GROUP BY`, or `HAVING`, then the rows seen by the window functions are the group rows instead of the original table rows from `FROM/WHERE`.

When multiple window functions are used, all the window functions having equivalent `PARTITION BY` and `ORDER BY` clauses in their window definitions are guaranteed to see the same ordering of the input rows, even if the `ORDER BY` does not uniquely determine the ordering. However, no guarantees are made about the evaluation of functions having different `PARTITION BY` or `ORDER BY` specifications. (In such cases a sort step is typically required between the passes of window function evaluations, and the sort is not guaranteed to preserve ordering of rows that its `ORDER BY` sees as equivalent.)

Currently, window functions always require presorted data, and so the query output will be ordered according to one or another of the window functions' `PARTITION BY/ORDER BY` clauses. It is not recommended to rely on this, however. Use an explicit top-level `ORDER BY` clause if you want to be sure the results are sorted in a particular way.

7.3. Select Lists

As shown in the previous section, the table expression in the `SELECT` command constructs an intermediate virtual table by possibly combining tables, views, eliminating rows, grouping, etc. This table is finally passed on to processing by the *select list*. The select list determines which *columns* of the intermediate table are actually output.

7.3.1. Select-List Items

The simplest kind of select list is `*` which emits all columns that the table expression produces. Otherwise, a select list is a comma-separated list of value expressions (as defined in Section 4.2). For instance, it could be a list of column names:

```
SELECT a, b, c FROM ...
```

The columns names `a`, `b`, and `c` are either the actual names of the columns of tables referenced in the `FROM` clause, or the aliases given to them as explained in Section 7.2.1.2. The name space available in the select list is the same as in the `WHERE` clause, unless grouping is used, in which case it is the same as in the `HAVING` clause.

If more than one table has a column of the same name, the table name must also be given, as in:

```
SELECT tbl1.a, tbl2.a, tbl1.b FROM ...
```

When working with multiple tables, it can also be useful to ask for all the columns of a particular table:

```
SELECT tbl1.*, tbl2.a FROM ...
```

See Section 8.16.5 for more about the *table_name.** notation.

If an arbitrary value expression is used in the select list, it conceptually adds a new virtual column to the returned table. The value expression is evaluated once for each result row, with the row's values

substituted for any column references. But the expressions in the select list do not have to reference any columns in the table expression of the **FROM** clause; they can be constant arithmetic expressions, for instance.

7.3.2. Column Labels

The entries in the select list can be assigned names for subsequent processing, such as for use in an **ORDER BY** clause or for display by the client application. For example:

```
SELECT a AS value, b + c AS sum FROM ...
```

If no output column name is specified using **AS**, the system assigns a default column name. For simple column references, this is the name of the referenced column. For function calls, this is the name of the function. For complex expressions, the system will generate a generic name.

The **AS** key word is usually optional, but in some cases where the desired column name matches a PostgreSQL key word, you must write **AS** or double-quote the column name in order to avoid ambiguity. (Appendix C shows which key words require **AS** to be used as a column label.) For example, **FROM** is one such key word, so this does not work:

```
SELECT a from, b + c AS sum FROM ...
```

but either of these do:

```
SELECT a AS from, b + c AS sum FROM ...
SELECT a "from", b + c AS sum FROM ...
```

For greatest safety against possible future key word additions, it is recommended that you always either write **AS** or double-quote the output column name.

Note

The naming of output columns here is different from that done in the **FROM** clause (see Section 7.2.1.2). It is possible to rename the same column twice, but the name assigned in the select list is the one that will be passed on.

7.3.3. DISTINCT

After the select list has been processed, the result table can optionally be subject to the elimination of duplicate rows. The **DISTINCT** key word is written directly after **SELECT** to specify this:

```
SELECT DISTINCT select_list ...
```

(Instead of **DISTINCT** the key word **ALL** can be used to specify the default behavior of retaining all rows.)

Obviously, two rows are considered distinct if they differ in at least one column value. Null values are considered equal in this comparison.

Alternatively, an arbitrary expression can determine what rows are to be considered distinct:

```
SELECT DISTINCT ON (expression [, expression ...]) select_list ...
```

Here *expression* is an arbitrary value expression that is evaluated for all rows. A set of rows for which all the expressions are equal are considered duplicates, and only the first row of the set is kept in the output. Note that the “first row” of a set is unpredictable unless the query is sorted on enough columns to guarantee a unique ordering of the rows arriving at the `DISTINCT` filter. (`DISTINCT ON` processing occurs after `ORDER BY` sorting.)

The `DISTINCT ON` clause is not part of the SQL standard and is sometimes considered bad style because of the potentially indeterminate nature of its results. With judicious use of `GROUP BY` and subqueries in `FROM`, this construct can be avoided, but it is often the most convenient alternative.

7.4. Combining Queries (`UNION`, `INTERSECT`, `EXCEPT`)

The results of two queries can be combined using the set operations union, intersection, and difference. The syntax is

```
query1 UNION [ALL] query2
query1 INTERSECT [ALL] query2
query1 EXCEPT [ALL] query2
```

where *query1* and *query2* are queries that can use any of the features discussed up to this point.

`UNION` effectively appends the result of *query2* to the result of *query1* (although there is no guarantee that this is the order in which the rows are actually returned). Furthermore, it eliminates duplicate rows from its result, in the same way as `DISTINCT`, unless `UNION ALL` is used.

`INTERSECT` returns all rows that are both in the result of *query1* and in the result of *query2*. Duplicate rows are eliminated unless `INTERSECT ALL` is used.

`EXCEPT` returns all rows that are in the result of *query1* but not in the result of *query2*. (This is sometimes called the *difference* between two queries.) Again, duplicates are eliminated unless `EXCEPT ALL` is used.

In order to calculate the union, intersection, or difference of two queries, the two queries must be “union compatible”, which means that they return the same number of columns and the corresponding columns have compatible data types, as described in Section 10.5.

Set operations can be combined, for example

```
query1 UNION query2 EXCEPT query3
```

which is equivalent to

```
(query1 UNION query2) EXCEPT query3
```

As shown here, you can use parentheses to control the order of evaluation. Without parentheses, `UNION` and `EXCEPT` associate left-to-right, but `INTERSECT` binds more tightly than those two operators. Thus

```
query1 UNION query2 INTERSECT query3
```

means

```
query1 UNION (query2 INTERSECT query3)
```

You can also surround an individual *query* with parentheses. This is important if the *query* needs to use any of the clauses discussed in following sections, such as `LIMIT`. Without parentheses, you'll get a syntax error, or else the clause will be understood as applying to the output of the set operation rather than one of its inputs. For example,

```
SELECT a FROM b UNION SELECT x FROM y LIMIT 10
```

is accepted, but it means

```
(SELECT a FROM b UNION SELECT x FROM y) LIMIT 10
```

not

```
SELECT a FROM b UNION (SELECT x FROM y LIMIT 10)
```

7.5. Sorting Rows (**ORDER BY**)

After a query has produced an output table (after the select list has been processed) it can optionally be sorted. If sorting is not chosen, the rows will be returned in an unspecified order. The actual order in that case will depend on the scan and join plan types and the order on disk, but it must not be relied on. A particular output ordering can only be guaranteed if the sort step is explicitly chosen.

The `ORDER BY` clause specifies the sort order:

```
SELECT select_list
      FROM table_expression
      ORDER BY sort_expression1 [ASC | DESC] [NULLS { FIRST | LAST }]
              [, sort_expression2 [ASC | DESC] [NULLS { FIRST |
LAST }] ...]
```

The sort expression(s) can be any expression that would be valid in the query's select list. An example is:

```
SELECT a, b FROM table1 ORDER BY a + b, c;
```

When more than one expression is specified, the later values are used to sort rows that are equal according to the earlier values. Each expression can be followed by an optional `ASC` or `DESC` keyword to set the sort direction to ascending or descending. `ASC` order is the default. Ascending order puts smaller values first, where “smaller” is defined in terms of the `<` operator. Similarly, descending order is determined with the `>` operator.¹

The `NULLS FIRST` and `NULLS LAST` options can be used to determine whether nulls appear before or after non-null values in the sort ordering. By default, null values sort as if larger than any non-null value; that is, `NULLS FIRST` is the default for `DESC` order, and `NULLS LAST` otherwise.

Note that the ordering options are considered independently for each sort column. For example `ORDER BY x, y DESC` means `ORDER BY x ASC, y DESC`, which is not the same as `ORDER BY x DESC, y DESC`.

¹ Actually, PostgreSQL uses the *default B-tree operator class* for the expression's data type to determine the sort ordering for `ASC` and `DESC`. Conventionally, data types will be set up so that the `<` and `>` operators correspond to this sort ordering, but a user-defined data type's designer could choose to do something different.

A *sort_expression* can also be the column label or number of an output column, as in:

```
SELECT a + b AS sum, c FROM table1 ORDER BY sum;
SELECT a, max(b) FROM table1 GROUP BY a ORDER BY 1;
```

both of which sort by the first output column. Note that an output column name has to stand alone, that is, it cannot be used in an expression — for example, this is *not* correct:

```
SELECT a + b AS sum, c FROM table1 ORDER BY sum + c;      --
wrong
```

This restriction is made to reduce ambiguity. There is still ambiguity if an `ORDER BY` item is a simple name that could match either an output column name or a column from the table expression. The output column is used in such cases. This would only cause confusion if you use `AS` to rename an output column to match some other table column's name.

`ORDER BY` can be applied to the result of a `UNION`, `INTERSECT`, or `EXCEPT` combination, but in this case it is only permitted to sort by output column names or numbers, not by expressions.

7.6. LIMIT and OFFSET

`LIMIT` and `OFFSET` allow you to retrieve just a portion of the rows that are generated by the rest of the query:

```
SELECT select_list
      FROM table_expression
      [ ORDER BY ... ]
      [ LIMIT { count | ALL } ]
      [ OFFSET start ]
```

If a limit count is given, no more than that many rows will be returned (but possibly fewer, if the query itself yields fewer rows). `LIMIT ALL` is the same as omitting the `LIMIT` clause, as is `LIMIT` with a `NULL` argument.

`OFFSET` says to skip that many rows before beginning to return rows. `OFFSET 0` is the same as omitting the `OFFSET` clause, as is `OFFSET` with a `NULL` argument.

If both `OFFSET` and `LIMIT` appear, then `OFFSET` rows are skipped before starting to count the `LIMIT` rows that are returned.

When using `LIMIT`, it is important to use an `ORDER BY` clause that constrains the result rows into a unique order. Otherwise you will get an unpredictable subset of the query's rows. You might be asking for the tenth through twentieth rows, but tenth through twentieth in what ordering? The ordering is unknown, unless you specified `ORDER BY`.

The query optimizer takes `LIMIT` into account when generating query plans, so you are very likely to get different plans (yielding different row orders) depending on what you give for `LIMIT` and `OFFSET`. Thus, using different `LIMIT/OFFSET` values to select different subsets of a query result *will give inconsistent results* unless you enforce a predictable result ordering with `ORDER BY`. This is not a bug; it is an inherent consequence of the fact that SQL does not promise to deliver the results of a query in any particular order unless `ORDER BY` is used to constrain the order.

The rows skipped by an `OFFSET` clause still have to be computed inside the server; therefore a large `OFFSET` might be inefficient.

7.7. VALUES Lists

VALUES provides a way to generate a “constant table” that can be used in a query without having to actually create and populate a table on-disk. The syntax is

```
VALUES ( expression [, ...] ) [, ...]
```

Each parenthesized list of expressions generates a row in the table. The lists must all have the same number of elements (i.e., the number of columns in the table), and corresponding entries in each list must have compatible data types. The actual data type assigned to each column of the result is determined using the same rules as for UNION (see Section 10.5).

As an example:

```
VALUES (1, 'one'), (2, 'two'), (3, 'three');
```

will return a table of two columns and three rows. It's effectively equivalent to:

```
SELECT 1 AS column1, 'one' AS column2
UNION ALL
SELECT 2, 'two'
UNION ALL
SELECT 3, 'three';
```

By default, PostgreSQL assigns the names `column1`, `column2`, etc. to the columns of a VALUES table. The column names are not specified by the SQL standard and different database systems do it differently, so it's usually better to override the default names with a table alias list, like this:

```
=> SELECT * FROM (VALUES (1, 'one'), (2, 'two'), (3, 'three')) AS t
   (num, letter);
 num | letter
-----+-----
   1 | one
   2 | two
   3 | three
(3 rows)
```

Syntactically, VALUES followed by expression lists is treated as equivalent to:

```
SELECT select_list FROM table_expression
```

and can appear anywhere a SELECT can. For example, you can use it as part of a UNION, or attach a *sort_specification* (ORDER BY, LIMIT, and/or OFFSET) to it. VALUES is most commonly used as the data source in an INSERT command, and next most commonly as a subquery.

For more information see VALUES.

7.8. WITH Queries (Common Table Expressions)

WITH provides a way to write auxiliary statements for use in a larger query. These statements, which are often referred to as Common Table Expressions or CTEs, can be thought of as defining temporary tables that exist just for one query. Each auxiliary statement in a WITH clause can be a SELECT, INSERT, UPDATE, DELETE, or MERGE; and the WITH clause itself is attached to a primary statement that can also be a SELECT, INSERT, UPDATE, DELETE, or MERGE.